

UNIVERSITÀ
POLITECNICA
DELLE MARCHE

Corso di «Ingegneria del Software»

A.A. 2024/2025

Parte III: Analisi

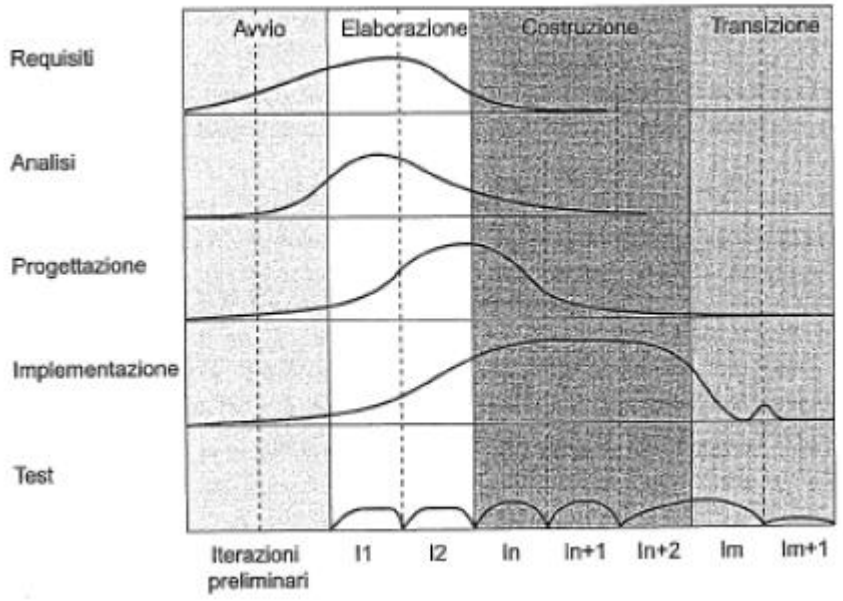
Prof. Domenico Ursino
d.ursino@univpm.it

Il flusso di lavoro dell'analisi – Caratteristiche generali

- Le attività di analisi entrano nel vivo verso la fine della fase di Avvio e diventano preponderanti durante la fase di Elaborazione.

- La maggior parte delle attività della fase di Elaborazione sono finalizzate alla creazione di modelli che descrivono il comportamento desiderato del sistema.

- Come si evince dalla seguente figura, il lavoro di analisi si sovrappone esattamente alla raccolta dei requisiti; difatti, queste attività procedono spesso mano nella mano:



Il flusso di lavoro dell'analisi – Caratteristiche generali

- Il flusso di lavoro dell'analisi mira a produrre un modello di analisi.
- Questo modello si concentra su che cosa il sistema debba fare, mentre il dettaglio del come lo farà compete al flusso di lavoro della progettazione.
- Il confine tra analisi e progettazione può essere piuttosto sfumato e, fino a un certo punto, tocca all'analista porre la linea di confine dove lo ritiene opportuno.
- Nel flusso di lavoro dell'analisi vengono prodotti due manufatti principali:
  - Le classi di analisi, che modellano i concetti chiave del dominio del problema.
  - Le realizzazioni di caso d'uso, che mostrano come le istanze delle classi di analisi possono interagire per realizzare il comportamento del sistema specificato da un caso d'uso.

Il flusso di lavoro dell'analisi – Caratteristiche generali

- Il flusso di lavoro dell'analisi previsto da UP viene riportato nella seguente figura:



Il flusso di lavoro dell'analisi – Regole pratiche per la costruzione di un modello di analisi

- Ogni sistema è diverso ed è difficile generalizzare sui modelli di analisi.
- Per un sistema di medie dimensioni e complessità il modello di analisi può contenere tra le 50 e le 100 classi.
- Nel modello di analisi è di importanza vitale limitarsi esclusivamente a quelle classi che fanno parte del vocabolario del dominio del problema.
- Si deve sempre evitare di aggiungere classi di progettazione (come quelle per le telecomunicazioni o per l'accesso ai database) nel modello di analisi.
- Ci si autolimita in questa maniera per fare del modello di analisi un'enunciazione semplice e concisa della struttura e del comportamento del sistema.
- Alcune regole pratiche per l'efficace modellazione dell'analisi sono le seguenti:
  - Esprimere sempre il modello di analisi nel linguaggio del problema.
  - Creare modelli che "raccontano una storia".
  - Concentrarsi sulla descrizione della vista d'insieme.

Il flusso di lavoro dell'analisi – Regole pratiche per la costruzione di un modello di analisi

- Mantenere una netta distinzione tra il dominio del problema (i requisiti di business) e il dominio delle soluzioni (considerazioni sui dettagli di progettazione).
- Impegnarsi sempre per minimizzare le interdipendenze.
- Esplorare l'uso dell'ereditarietà se sembra che esista un'ovvia e naturale gerarchia tra le astrazioni.
- Chiedersi sempre: "questo modello sarà utile a tutte le parti interessate?".
- Le strategie chiave con cui cautelarsi sono: conferire la massima visibilità al modello e alle attività dell'analisi, coinvolgere le parti interessate nel processo ogniqualvolta possibile, e indire frequenti riunioni aperte a tutti.

- Infine, mantenere semplice il modello!
- Un modo per semplificare consiste nel guardare al caso generale piuttosto che a quelli particolari.

Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono gli oggetti?

- Un oggetto può essere definito come:

  Un'entità discreta, con confini ben definiti che incapsula stato e comportamento; un'istanza di una classe

- Un oggetto può essere visto come un pacchetto coeso di dati e funzioni.
- L'unico modo di avere accesso alla parte dati di un oggetto consiste nel chiamare una delle funzioni rese disponibili dall'oggetto stesso.
- Nell'analisi ci si riferisce a queste funzioni come operazioni.
- Il mascherare la parte di un oggetto sotto uno strato di funzioni si chiama incapsulamento ovvero opacità dei dati.
- Ogni oggetto è di per sé un'istanza di una classe;.
- Una classe definisce l'insieme di caratteristiche (attributi e operazioni/metodi) comuni a tutti i corrispettivi oggetti.

Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono gli oggetti?

- È possibile notare certe proprietà comuni a tutti gli oggetti:
  - Identità: la peculiarità dell'esistenza dell'oggetto nel tempo e nello spazio, ciò che lo distingue da tutti gli altri oggetti.
  - Stato: viene stabilito dai valori degli attributi di un oggetto e dalle relazioni che l'oggetto ha con gli altri oggetti in un dato istante.
    - Come esempio di stato, nella seguente tabella viene riportata una lista parziale degli stati in cui si può trovare una stampante.

Stato dell'oggetto	*Attributo della classe*	*Valore dell'attributo dell'oggetto*	*Relazione*
On	alimentazione	On	N/A
Off	alimentazione	Off	N/A
FineInchiostroNero	cartucciaNero	Vuoto	N/A
FineInchiostroColore	cartucciaColore	Vuoto	N/A
Collegato	N/A	N/A	Collegata a un oggetto calcolatore
NonCollegato	N/A	N/A	Non collegata a un oggetto calcolatore

Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono gli oggetti?

  - Comportamento: ci sono azioni che un oggetto può eseguire (ad esempio, per una stampante: `accendi`(), `spegni`(), `stampaDocumento`(), `avanzamentoPagina`(), `puliziaTestina`(), `cambiaCartucciaInchiostro`()).

- L'esecuzione di un'operazione o di un modello di un oggetto provoca spesso una variazione di valore di uno o più dei suoi attributi o delle relazioni ad altri oggetti.

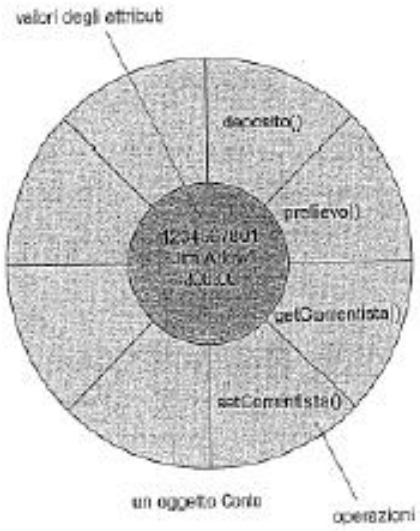
- Questo potrebbe portare ad una transizione di stato.

- Lo stato di un oggetto può anche influenzare il comportamento.

- Un'operazione è la specifica di un comportamento; un'implementazione di quel comportamento è definita metodo.

Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono gli oggetti? - Incapsulamento

- L'identità di un oggetto è una forma di identificatore univoco stabilito dal linguaggio di implementazione.
- Nel seguito chiameremo questi identificatori riferimenti all'oggetto; durante l'analisi non ci preoccuperemo di come essi sono realizzati.
- È possibile che durante la progettazione si debba tenere conto dell'implementazione dei riferimenti agli oggetti.
- Nella seguente figura viene riportata una rappresentazione concettuale di un oggetto:



Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono gli oggetti? - Incapsulamento

- Lo stato dell'oggetto è l'insieme dei valori degli attributi posseduti dall'oggetto in un dato istante.
- Tipicamente, alcuni di questi valori cambieranno nel tempo mentre altri rimarranno invariati.
- Ad esempio, in un conto corrente al variare del saldo nel tempo può anche cambiare lo stato dell'oggetto.
- L'insieme delle transazioni possibili può essere molto ampio.
- UML fornisce i diagrammi delle macchine a stati per rappresentare i passaggi di stato.
- Il comportamento di un oggetto rappresenta ciò che esso può fare per noi.
  - Ad esempio, nella seguente tabella vengono riportate le possibili operazioni per l'oggetto raffigurato in precedenza:

Operazione	*Semantica*
`deposito()`	Deposita del denaro nell'oggetto `Conto`. Incrementa il valore dell'attributo `Saldo`.
`prelievo()`	Preleva del denaro dall'oggetto `Conto`. Decrementa il valore dell'attributo `Saldo`.
`getCorrentista()`	Restituisce il correntista dell'oggetto `Conto`. Rappresenta un'operazione di query.
`setCorrentista()`	Cambia il correntista dell'oggetto `Conto`.

Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono gli oggetti? - Incapsulamento

- Questo insieme di operazioni specifica il comportamento dell'oggetto.

- L'esecuzione di alcune di queste operazioni modifica i valori di qualche attributo e può generare delle transazioni da uno stato ad un altro.

- Altre operazioni non cambiano il valore di alcun attributo e quindi non scatenano alcuna transazione di stato.

- L'incapsulamento è uno dei vantaggi più importanti della programmazione OO e può consentire la creazione di software robusto ed affidabile.

Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono gli oggetti? - Messaggi

- Come si combinano gli oggetti per creare dei sistemi?
- Gli oggetti collaborano stabilendo dei collegamenti tra loro e scambiandosi messaggi lungo questi collegamenti.
- Quando un oggetto riceve un messaggio, esso esamina le sue operazioni per trovare una la cui segnatura corrisponda alla segnatura del messaggio; se tale operazione esiste allora l'oggetto la esegue.
- Un esempio di questo comportamento viene riportato nella seguente figura:

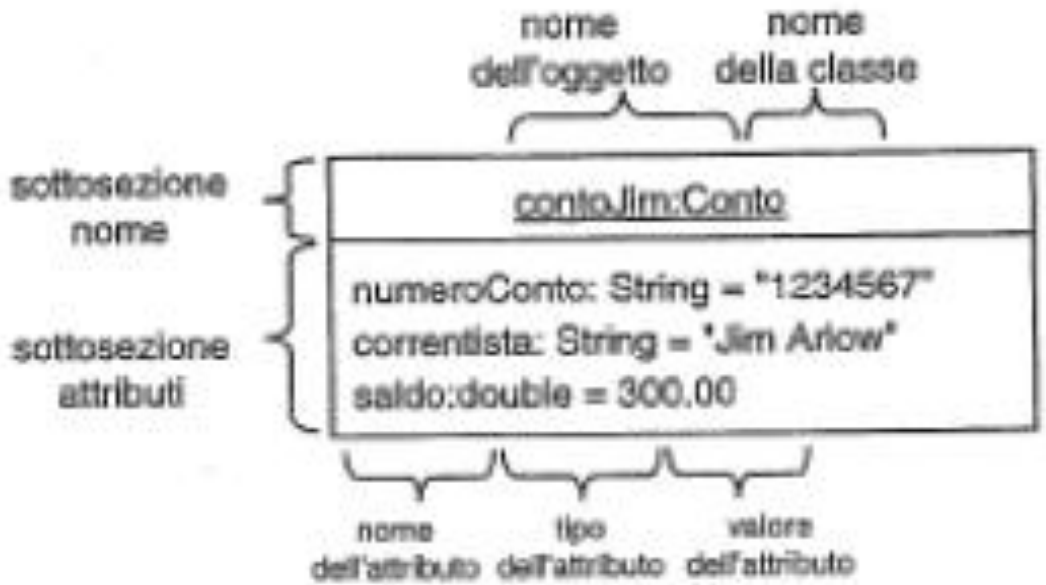


Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono gli oggetti? - Messaggi

- Queste segnature sono composte dal nome del messaggio, dai tipi dei parametri e dal valore restituito.
- La struttura a run-time di un sistema OO è costituita da numerosi oggetti che vengono creati, esistono per un certo tempo e poi, forse, vengono distrutti.
- Questi oggetti si scambiano messaggi per utilizzare i rispettivi servizi.
- Questa struttura è profondamente diversa da quella dei sistemi software procedurali.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per gli oggetti

- Nella seguente figura viene riportata una rappresentazione di un oggetto in UML:



- La sottosezione superiore contiene l'identificatore dell'oggetto, che è sempre sottolineato.

- UML consente di rappresentare un oggetto in modo molto flessibile.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per gli oggetti

- L'identificatore di un oggetto può essere uno qualunque dei seguenti:
  - Il solo nome della classe, per esempio `Conto`.
    - Questo indica un oggetto anonimo.
    - Gli oggetti anonimi si usano spesso quando in un diagramma è presente un solo oggetto di una data classe.
    - Se occorre mostrare due oggetti della stessa classe è meglio distinguerli.
  - Il solo nome dell'oggetto, per esempio `contoJim`.
    - Questo identifica un oggetto specifico, ma non dice a quale classe esso appartenga.
    - Questa notazione è utile per un'analisi molto preliminare.
  - Il nome dell'oggetto, concatenato al nome della classe, separati da due punti, per esempio `contoJim:Conto`.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per gli oggetti

- Normalmente i nomi degli oggetti sono scritti in lowerCamelCase.
- Poiché tutti gli oggetti di una classe hanno le stesse operazioni queste sono elencate nell'icona della sola classe ma non nell'icona dell'oggetto.
- Non è obbligatorio mostrare gli attributi nella sottosezione inferiore dell'icona di un oggetto.
- Anche i nomi degli attributi vengono scritti in lowerCamelCase.
- Ogni valore di un attributo di un oggetto si rappresenta come `nome:tipo=valore`.
- Si può scegliere di mostrare tutti, solo alcuni o nessuno dei valori degli attributi di un oggetto.
- Per mantenere i diagrammi semplici e chiari si può voler omettere i tipi degli attributi.

Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono le classi?

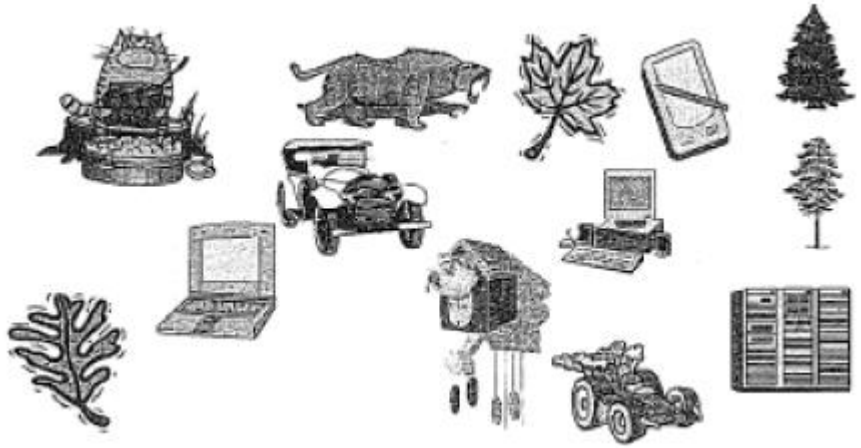
- Una classe può essere definita come:

> Il descrittore di un insieme di oggetti che condividono gli stessi attributi, le stesse operazioni, gli stessi metodi, le stesse relazioni e lo stesso comportamento

- In pratica, una classe descrive un insieme di oggetti che hanno le stesse caratteristiche.
- Ogni oggetto è un'istanza di una classe.
- Classi e oggetti sono gli esempi più comuni della distinzione tra classificatore e istanza.
- Una classe è una specifica o formato a cui devono corrispondere tutti i suoi oggetti.
- Ogni oggetto della classe ha specifici valori per gli attributi definiti dalla classe.
- A seconda del loro stato, oggetti diversi possono rispondere diversamente allo stesso messaggio.
- La classificazione è uno dei concetti OO più importanti; le classi consentono di descrivere l'insieme di caratteristiche che i corrispettivi oggetti devono possedere senza che si debba descrivere ogni singolo oggetto.

Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono le classi?

- Consideriamo la seguente figura e chiediamoci quante classi ci sono in essa:



- In realtà a questa domanda non c'è risposta!
- Avendo così tante possibilità uno degli aspetti più importanti dell'analisi e progettazione OO è la scelta dello schema di classificazione più appropriato.
- Esaminando la figura attentamente si iniziano a vedere altri tipi di relazioni, oltre a quelle di classificatore e istanza.
- Per esempio si delineano diversi livelli di classificazione (ereditarietà).

Il flusso di lavoro dell'analisi – Classi e oggetti – Cosa sono le classi?

- Gli oggetti albero possiedono ognuno un gruppo di oggetti foglia (composizione).

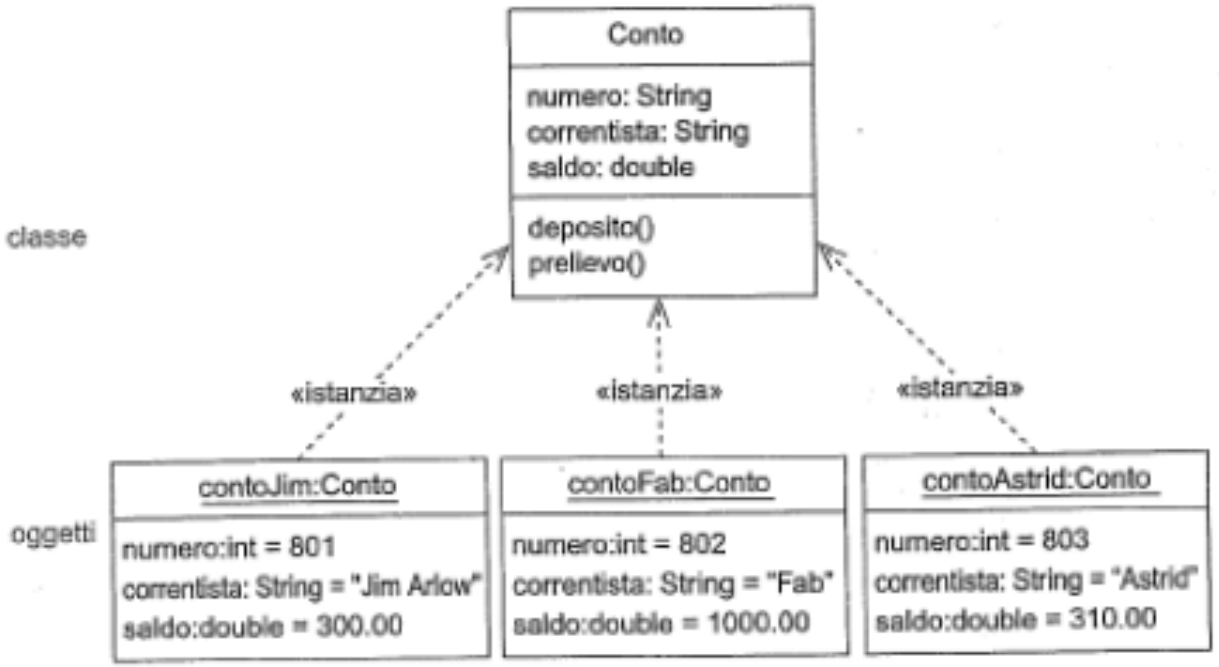
- Un calcolatore può avere associate delle periferiche che mantengono una loro indipendenza dal calcolatore (aggregazione).

Il flusso di lavoro dell'analisi – Classi e oggetti – Relazioni tra classi e oggetti

- La relazione esistente tra una classe e i corrispettivi oggetti è una relazione <<`istanzia`>>.
- Una relazione può essere definita come:

Una connessione tra elementi del modello

- Un esempio della relazione <<`istanzia`>> viene riportato nella seguente figura:



Il flusso di lavoro dell'analisi – Classi e oggetti – Relazioni tra classi e oggetti

- La freccia tratteggiata è in realtà una relazione di dipendenza a cui viene assegnato un significato speciale mediante lo stereotipo <<`istanzia`>>.

- Una dipendenza può essere definita come:

> Una relazione tra due elementi in cui un cambiamento in un elemento (il fornitore) può influenzare o fornire informazione di cui necessita l'altro elemento (il cliente)

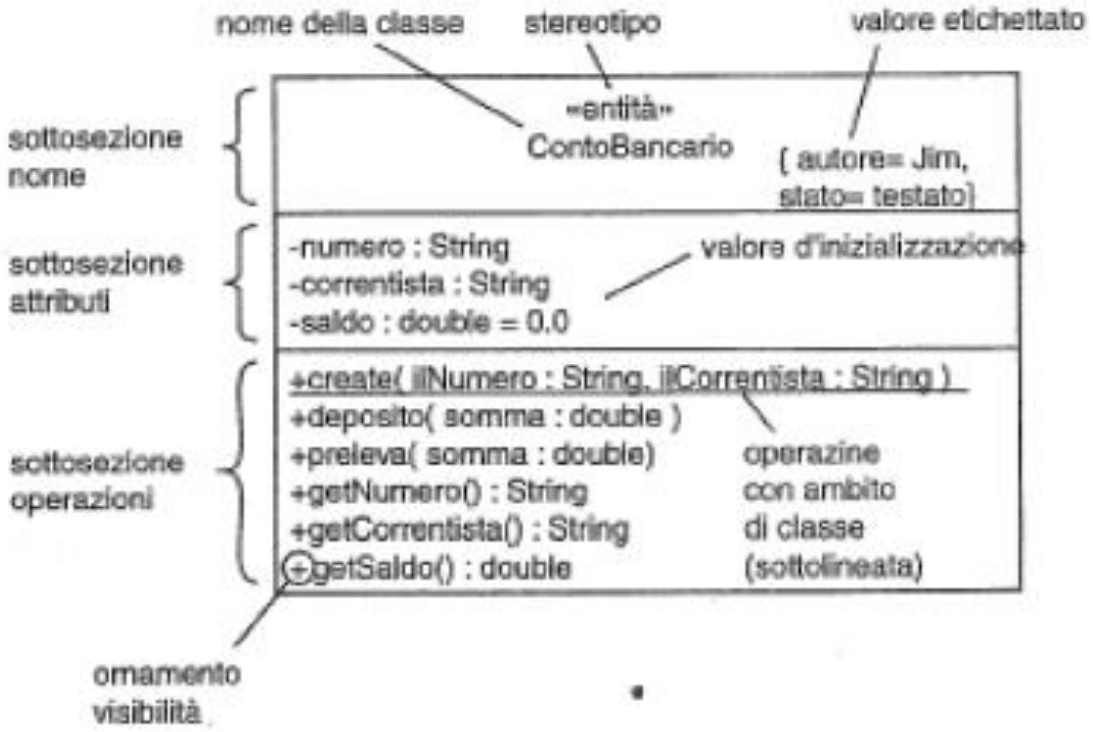
- Nella figura precedente la classe `Conto` deve chiaramente essere il fornitore.

Il flusso di lavoro dell'analisi – Classi e oggetti – Istanziazione di una classe

- L'istanziazione è la creazione di nuove istanze di elementi del modello.
- L'istanziazione si applica anche ad altri elementi del modello, così come alle classi e agli oggetti.
- L'istanziazione applicata alle classi porta alla creazione degli oggetti; tale operazione avviene attraverso i costruttori.
- I costruttori nella programmazione OO hanno ambito di classe e non di oggetto proprio perché devono creare un nuovo oggetto.
- Il costruttore alloca della memoria per il nuovo oggetto, assegna ad esso un'identità unica e stabilisce i valori iniziali degli attributi dell'oggetto.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi

- La sintassi UML per le classi diviene più gestibile applicando il concetto di ornamenti opzionali di UML.
- Il nome è l'unica sottosezione obbligatoria nella sintassi grafica UML.
- Un esempio completo di classe viene riportato nella seguente figura:



Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi

- Lo scopo per cui si produce il diagramma delle classi determina quali sottosezioni e ornamenti vengono indicati in una classe.
- Se si è interessati a mostrare le sole relazioni tra le classi allora può bastare la sola sottosezione nome.
- Se il diagramma deve illustrare il comportamento delle classi, probabilmente occorre far vedere anche la sottosezione delle operazioni.
- Se invece il diagramma è un modello più "orientato ai dati" conviene tentare di mostrare la corrispondenza tra classi e tabelle relazionali aggiungendo alla sottosezione nome anche le sottosezioni operazioni e attributi ed, eventualmente, anche i tipi degli attributi.
- È necessario comunicare la giusta quantità di informazione.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi

- In un modello di analisi, generalmente, è necessario indicare i seguenti dettagli:
  - Nome della classe
  - Attributi fondamentali
  - Operazioni fondamentali
  - Stereotipi (se sono particolarmente significativi per il business)
- Solitamente non è necessario indicare i seguenti dettagli:
  - Valori etichettati
  - Parametri delle operazioni
  - Visibilità
  - Valori d'inizializzazione

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione nome

- Esiste una convenzione usata quasi universalmente per i nomi delle classi:
- Il nome della classe viene scritto in UpperCamelCase.
  - I caratteri speciali, quali i segni di interpunzione e di sottolineatura, i vari simboli, le lineette, gli apici e le sbarrette, non sono ammessi.
  - Si osservi che, tra i vari simboli vietati ad UML, si devono includere anche le lettere accentate.
- Si evitano le abbreviazioni a tutti i costi.
  - I nomi delle classi dovrebbero sempre riflettere i nomi di entità del mondo reale, senza alcuna abbreviazione.
  - Le abbreviazioni rendono il modello difficile da leggere.
- Se esistono degli acronimi specifici del dominio (per esempio CRM – Customer Relationship Management), la forma estesa delle parole può essere comunque preferibile se rende il modello più chiaro.
- Le classi rappresentano "cose", quindi dovrebbero avere un nome costituito da un sostantivo o da una frase nominale, per esempio `Persona`, `Denaro`, `ContoBancario`.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione attributi

- Il nome dell'attributo è l'unica parte obbligatoria della sintassi UML per gli attributi.
- La struttura di un attributo è la seguente:



- Un attributo viene scritto in lowerCamelCase.
- Il nome degli attributi di solito è un sostantivo o una frase nominale, perché gli attributi indicano una "cosa".
- Si devono evitare caratteri speciali e abbreviazioni.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione attributi – Visibilità

- L'ornamento di visibilità si applica agli attributi e alle operazioni della classe.
- I possibili ornamenti di visibilità sono riportati nella seguente tabella:

Ornamento	*Tipo di visibilità*	*Semantica*
+	Visibilità pubblica	Ogni elemento che può accedere alla classe può anche accedere a ogni suo membro con visibilità pubblica.
-	Visibilità privata	Solo le operazioni della classe possono accedere ai membri con visibilità privata.
#	Visibilità protetta	Solo operazioni appartenenti alla classe o ai suoi discendenti possono accedere ai membri con visibilità protetta.
~	Visibilità package	Ogni elemento dello stesso package della classe (o suo sotto-package annidato) può accedere ai membri della classe con visibilità package.

- Durante l'analisi normalmente non si affollano i diagrammi con indicatori di visibilità.
- Tutti i tipi di visibilità, a eccezione di pubblica, sono dipendenti dal linguaggio di implementazione.
- Questa precisazione è importante perché i diversi linguaggi possono definire tipi differenti di visibilità non tutti direttamente supportati da UML.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione attributi – Visibilità

- Per dare un'idea di quanto stiamo dicendo, la seguente tabella prende in esame due linguaggio OO; in particolare, essa confronta la semantica della visibilità UML con quella di Java e C#:

Visibilità	*Semantica UML*	*Semantica Java*	*Semantica C#*
pubblica	Ogni elemento che può accedere alla classe, può accedere ad ogni suo membro con visibilità pubblica	come in UML	come in UML
privata	Solo le operazioni della classe possono accedere a membri con visibilità privata	come in UML	come in UML
protetta	Solo operazioni appartenenti alla classe o ai suoi discendenti possono accedere ai membri con visibilità protetta	come in UML, ma accessibile solo alle classi dello stesso package Java della classe in cui è definito	come in UML
package	Ogni elemento che sia nello stesso package della classe, o in un suo sotto-package annidato, può accedere ai membri della classe con visibilità package	La visibilità di default in Java Classi annidate in sotto-package annidati non hanno automaticamente accesso agli elementi del loro package genitore	-
privata	-	Lo stesso che protetta in UML	-
interna	-	-	Accessibile da ogni elemento dello stesso programma
protetta interna	-	-	Assomma le semantiche di interna e di protetta; si applica solo agli attributi

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione attributi – Visibilità

- La definizione della visibilità dipende totalmente dal linguaggio.
- L'esatto tipo di visibilità da utilizzare è una decisione implementativa di basso livello e dovrebbe essere effettuata dal programmatore piuttosto che dall'analista.
- Dal punto di vista della modellazione in generale le definizioni standard di UML sono adeguate e sufficienti.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione attributi – Tipo

- Il tipo di un attributo può essere un'altra classe o un tipo primitivo.
- La specifica di UML definisce quattro tipi primitivi che vengono usati in essa.
- La tabella seguente riporta i tipi primitivi in UML e in OCL:

	Tipo primitivo	*Semantica*
UML	Integer	Un numero intero
	UnlimitedNatural	Un numero intero ≥ 0. L'infinito può essere indicato come $*$
	Boolean	Può assumere il valore true o false
	String	Una sequenza di caratteri. Le string literal sono sempre tra virgolette: per esempio "Jim"
OCL	Real	Un numero a virgola mobile

- Object Constraint Language (OCL) è un linguaggio formale che permette di descrivere i vincoli nei modelli UML.
- OCL definisce operazioni standard per i tipi primitivi UML e inserisce un nuovo tipo chiamato `Real`.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione attributi – Tipo

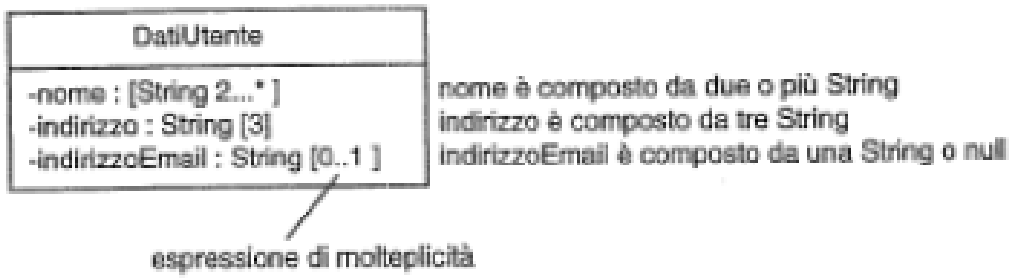
- Se il modello ha come obiettivo un linguaggio specifico, per esempio Java o C#, è possibile usare i tipi primitivi di quest'ultimo.

- Talvolta è necessario aggiungere a un modello UML un insieme di tipi primitivi per poterli usare in altre classi come tipi di parametri di operazioni e attributi.

- Normalmente la classe non ha attributi o operazioni perché agisce semplicemente come segnaposto del tipo primitivo.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione attributi – Molteplicità

- La molteplicità è ampiamente usata nella progettazione ma può anche essere utilizzata per i modelli di analisi.
- Alcuni esempi di sintassi per la molteplicità vengono riportati nella figura seguente:



- La molteplicità consente di modellare due aspetti prettamente distinti con una sola specifica di molteplicità su un attributo:
  - Classi contenitore: se la specifica di molteplicità è un intero più grande di 1, allora si sta definendo una raccolta del tipo; per esempio, colori[7]

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione attributi – Molteplicità

- Valori nulli: molti linguaggi distinguono l'attributo che contiene un valore vuoto o non inizializzato dall'attributo che non si riferisce a nulla, ovvero il riferimento null a oggetto
  - Quando un attributo contiene un riferimento a null vuol dire che l'oggetto a cui si riferisce non è ancora stato creato o che ha già cessato di esistere.
  - Nella progettazione dettagliata capita a volte che sia importante evidenziare che null è un possibile valore di un certo attributo; si esprime questa informazione con la speciale espressione di molteplicità [0..1].

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione attributi – Valore iniziale

- Il valore iniziale consente di specificare il valore che l'attributo assume quando si istanzia un oggetto della classe.
- Definire ogniqualvolta possibile i valori iniziali è considerato un ottimo approccio nella progettazione.
- Nell'analisi si usano i valori iniziali solo quando possono esprimere o evidenziare un importante vincolo del problema; questa cosa, tuttavia, accade di rado.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione attributi

- Come per ogni altro elemento dei modelli di UML, la sintassi per gli attributi si può estendere facendoli precedere da stereotipi.

- Si può anche estendere la specifica di un attributo posponendo ad esso dei valori etichettati:

```
"stereotipo" attributo {etichetta1=valore1, etichetta2=valore2, ...}
```

- Nei valori etichettati è possibile registrare ciò che si vuole; vengono spesso utilizzati per contenere informazioni come le seguenti:

```
Indirizzo {aggiuntoDa = Jim Arlow, data = 20MAR2004}
```

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione operazioni

- Le operazioni sono funzioni associate ad una certa classe.
- Esse hanno tutte le caratteristiche delle funzioni, ovvero il nome, la lista dei parametri e il tipo del valore restituito.
- La possibile segnatura di un'operazione viene riportata nella seguente figura:



- Ogni operazione di una classe deve avere una sua unica segnatura.
- Quando si invia un messaggio a un oggetto la segnatura del messaggio viene confrontata con quella delle operazioni definite nella classe dell'oggetto.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione operazioni

- I vari linguaggi di implementazione interpretano il concetto di segnatura di un'operazione in modo leggermente diverso.
- I nomi delle operazioni vengono scritti in lowerCamelCase; di solito sono verbi o predicati verbali.
- La forma dei parametri delle operazioni è la seguente:

```
Direzione nomeParametro : tipoParametro = valorePredefinito
```

- Un'operazione può non avere parametri o averne molti.
- I nomi dei parametri vengono scritti in lowerCamelCase e normalmente sono sostantivi o frasi sostantivate.
- Ogni parametro ha un tipo che è una classe o un tipo primitivo.
- Ai parametri delle operazioni può essere assegnata una direzione.
- Se la direzione non è specificata assume il valore di default di `in`.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione operazioni

- Si consideri il seguente esempio:

```
operazione(in p1:Integer, inout p2:Integer, out p3:Integer,
return p4:Integer, return p5:Integer)
```

- La semantica della direzione dei parametri viene riportata nella seguente tabella:

Parametro	*Semantica*
in p1:Integer	Predefinito L'operazione usa p1 come un parametro di input Il valore di p1 viene usato dall'operazione p1 non viene modificato dall'operazione
inout p2:Integer	L'operazione accetta p2 come un parametro di input/output Il valore di p2 viene usato dall'operazione e serve per accettare un valore prodotto dall'operazione p2 può essere modificato dall'operazione
out p3:Integer	L'operatore usa p3 come un parametro di output Il parametro serve come un repository per accettare un valore prodotto dall'operazione p3 può essere modificato dall'operazione
return p4:Integer	L'operazione usa p4 come un parametro di ritorno L'operazione restituisce p4 come uno dei suoi valori di ritorno
return p5:Integer	L'operazione usa p5 come un parametro di ritorno L'operazione restituisce p5 come uno dei suoi valori di ritorno

- La semantica di `in`, `inout` e `out` è piuttosto semplice.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione operazioni

- La semantica del `return` forse va approfondita.
- In genere ci si aspetta che le funzioni restituiscano un oggetto esattamente come mostrato di seguito:

```
valoreMassimo = max(a,b)
valoreMinimo = min(a,b)
```

- Ogni operazione può avere un valore di ritorno.
- In UML è possibile modellare queste operazioni "normali" nel modo seguente:

```
valoreMassimo(a:Integer, b:Integer):Integer
```

- Alcuni linguaggi permettono alle operazioni di restituire più di un valore; per esempio, Python consente alle operazioni di avere valori di ritorno multipli:

```
valoreMassimo, valoreMinimo = maxMin(a,b)
```

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione operazioni

- In UML è possibile modellare questa operazione come segue:

```
maxMin(in a:Integer, in b:Integer, return valoreMax:Integer,
return valoreMin:Integer)
```

- La direzione del parametro `return` permette di modellare situazioni in cui un'operazione ha più di un valore di ritorno.

- È possibile elencare i parametri di `return` dopo il nome dell'operazione come segue:

```
maxMin(in a:Integer, in b:Integer):Integer,Integer
```

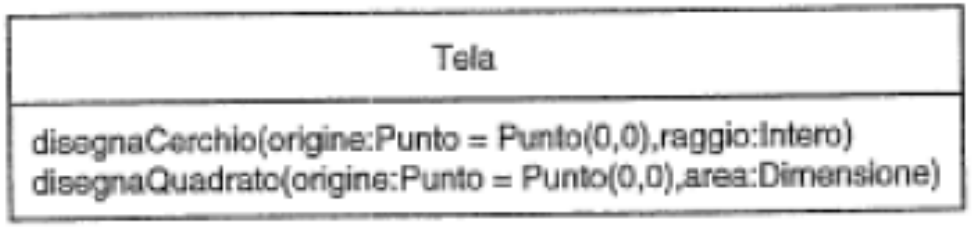
- In questo caso è sconsigliabile usare la parola chiave `return` poiché i parametri di return si distinguono già con chiarezza.

- Le direzioni dei parametri sono un problema di progettazione e quindi, normalmente, non vengono considerate nell'analisi.

- Le direzioni dei parametri possono essere molto importanti nella progettazione, soprattutto se si genera codice.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione operazioni

- A un parametro dell'operazione si può assegnare un valore di default.
- È possibile specificare valori di default nel seguente modo:



- I valori di default sono un problema di progettazione e raramente vengono usati nell'analisi.
- La sintassi per le operazioni può anche essere estesa sia preponendo uno stereotipo sia posponendo dei valori etichettati:

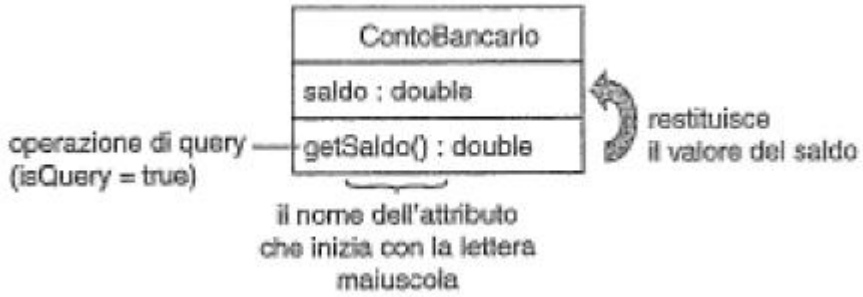
```
"stereotipo" operazione(...) {etichetta1=valore1, etichetta2=valore2, ...}
```

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione operazioni

- Ogni operazione ha una proprietà chiamata `isQuery`.
- Essa non ha effetti collaterali e non cambia lo stato dell'oggetto su cui è chiamata.
- Un'operazione che restituisce il valore di un attributo dovrebbe avere la proprietà `isQuery` impostata a `true`.
- Il valore di default per `isQuery` è `false`.
- Normalmente l'impostazione di `isQuery` è un problema di progettazione.
- Tuttavia, specificare le operazioni di query può essere importante se si usa OCL nei modelli UML.
- Uno standard di denominazione più o meno generale per le operazioni di query è creare il loro nome premettendo get al nome di ciò che si vuole ottenere dall'interrogazione.

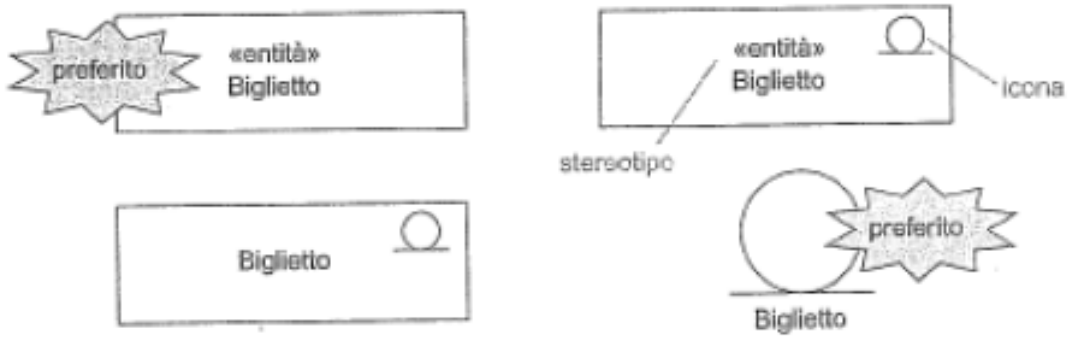
Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sottosezione operazioni

- Una possibile sintassi per le operazioni di query viene riportata nella seguente figura:



Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Sintassi per le classi con stereotipo

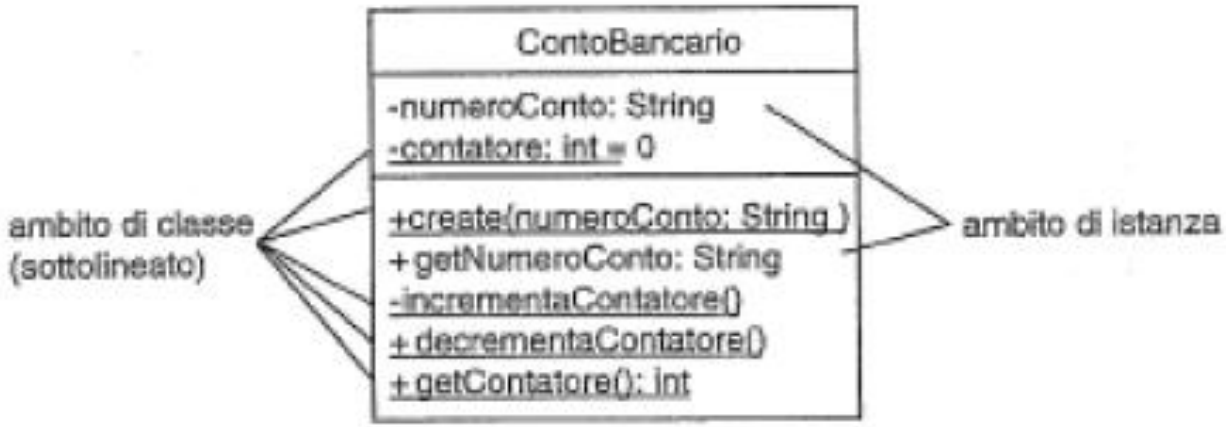
- Sono previsti modi diversi per indicare uno stereotipo in un modello.
- Molti modellatori preferiscono limitarsi a usare il nome tra apici ("nomeStereotipo") o l'icona.
- I modi di rappresentazione degli stereotipi vengono riportati nella seguente figura:



- Uno stereotipo potrebbe anche essere associato a un colore o a un retino, ma questa è una pessima idea.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Ambito

- Generalmente gli oggetti hanno una propria copia degli attributi e delle operazioni definiti dalla loro classe; in questo caso si dice che hanno ambito di istanza.
- Si possono definire attributi e operazioni condivisi tra tutti gli oggetti di una stessa classe; in questo caso si dice che essi hanno ambito di classe.
- Gli elementi che hanno ambito di classe sono condivisi da tutti gli oggetti di una stessa classe.
- Le notazioni per l'ambito di istanza e di classe vengono riportate nella seguente figura:



Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Ambito

- La semantica dell'ambito di istanza e di classe degli attributi e delle operazioni viene riportata nella seguente tabella:

	Ambito di istanza	*Ambito di classe*
Attributi	Per default, gli attributi hanno ambito di istanza Gli attributi possono essere definiti con ambito di classe Quindi, ciascun oggetto può avere valori diversi per un attributo con ambito di istanza	Ogni oggetto della classe contiene una sua copia di un dato attributo con ambito di istanza Ogni oggetto della classe condivide un'unica copia di un attributo con ambito di classe Quindi, tutti gli oggetti avranno lo stesso valore per un attributo con ambito di classe
Operazioni	Se non indicato altrimenti, le operazioni hanno ambito di istanza Ogni chiamata a un'operazione con ambito di istanza si applica ad una specifica istanza della classe Non si può chiamare un'operazione con ambito di istanza se non si ha già un'istanza della classe. Chiaramente, questo implica che non si può usare questo tipo di operazione per creare oggetti della classe, perchè non si potrebbe mai crearne il primo	Le operazioni possono essere definite con ambito di classe La chiamata a un'operazione con ambito di classe non si applica a un'istanza specifica di una classe, ma invece si può pensare che l'operazione si applichi alla classe stessa Si può chiamare un'operazione con ambito di classe anche se non esiste un'istanza della classe. Questo consente di avere operazioni per la creazione degli oggetti

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Ambito

- Se un'operazione possa o meno accedere a un altro membro della classe viene stabilito dall'ambito dell'operazione e da quella del membro a cui essa cerca di accedere.

- Le operazioni con ambito di istanza possono accedere ad altre operazioni o attributi con ambito di istanza, come anche a tutti gli attributi e le operazioni con ambito di classe.

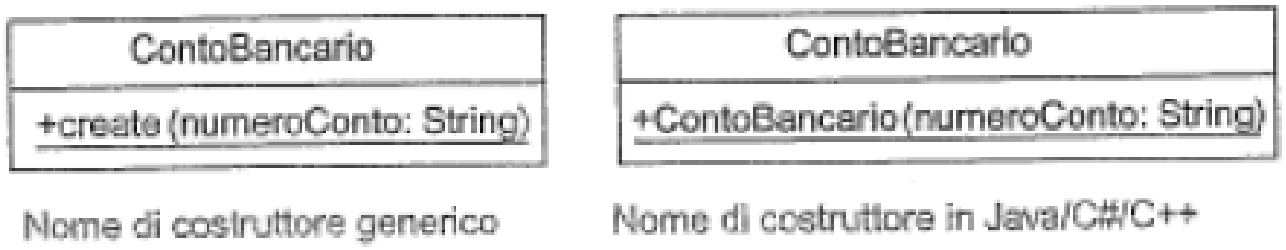
- Le operazioni con ambito di classe possono accedere soltanto ad altre operazioni e attributi con ambito di classe ma non possono accedere alle operazioni con ambito di istanza.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Creazione e distruzione degli oggetti

- I costruttori sono operazioni speciali che creano nuove istanze delle classi.
- Queste operazioni devono avere ambito di classe.
- I costruttori vanno presi in considerazione in fase di progettazione e generalmente non compaiono nei modelli dell'analisi.
- Ogni linguaggio ha uno standard diverso per i nomi dei costruttori.
- Un approccio completamente generico è quello di chiamare il costruttore semplicemente `create`(…).
- I linguaggi Java, C# e C++ richiedono tutti che il nome del costruttore sia uguale a quello della classe.
- Una classe può possedere molti costruttori, ognuno reso distinguibile da una diversa lista di parametri.
- Il costruttore privo di parametri è noto come costruttore di default.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Creazione e distruzione degli oggetti

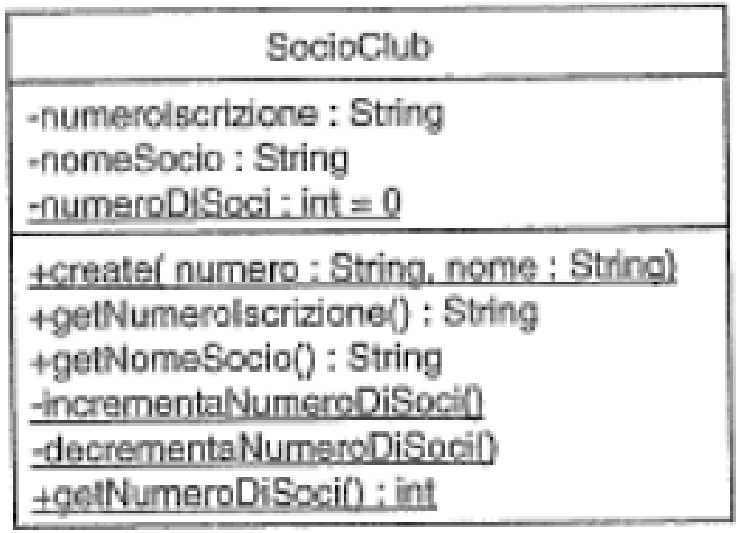
- Un esempio di costruttore viene riportato nella seguente figura:



- Non è possibile creare un conto senza specificare questo parametro.
- Nel modello dell'analisi non ci si preoccupa di specificare i costruttori (o i distruttori) poiché normalmente non hanno alcuna influenza o relazione con la parte di semantica del problema descritta dalla classe.
- Durante la progettazione, invece, si devono specificare il nome e il tipo di parametri.
- La distruzione degli oggetti è un argomento più complesso della loro costruzione.
- I linguaggi OO hanno semantiche differenti per la distruzione degli oggetti.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Creazione e distruzione degli oggetti

- Un tipico uso di attributi e di operazioni con ambito di classe viene riportato nella seguente figura:



- Al momento della creazione l'attributo `numeroDiSoci` viene inizializzato a zero.

- Se questo fosse stato un attributo con ambito di istanza, al momento della creazione ogni oggetto avrebbe ricevuto una sua propria copia dell'attributo.

- Il momento esatto in cui un attributo viene inizializzato dipende dall'implementazione; si può comunque assumere che avvenga all'avvio del programma.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Creazione e distruzione degli oggetti

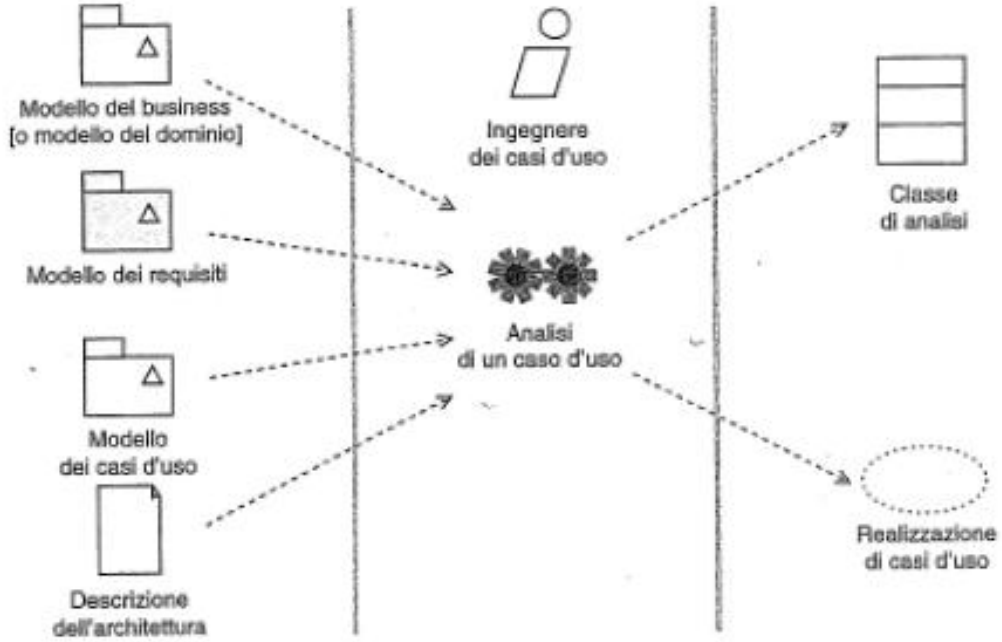
- Si supponga che nell'operazione `create(…)` si chiami l'operazione `incrementaNumeroDiSoci` che ha ambito di classe.
- L'attributo `numeroDiSoci` si incrementa a ogni nuova creazione di un'istanza della classe.
- Si può ottenere il valore di `numeroDiSoci` mediante l'operazione `getNumeroDiSoci()`.
- Cosa accade se il programma, oltre a creare oggetti `SocioClub`, li distrugge anche?
- È necessario introdurre nella classe un'operazione che decrementa `numeroDiSoci` e che viene chiamata a ogni distruzione di un'istanza di `SocioClub`.
- Alcuni linguaggi OO possiedono metodi speciali con ambito di classe, detti distruttori.
- In C++ il distruttore ha sempre la forma `~NomeDellaClasse(listaDiParametri)` e viene chiamato al momento dell'eliminazione dell'oggetto.

Il flusso di lavoro dell'analisi – Classi e oggetti – Notazione UML per le classi – Creazione e distruzione degli oggetti

- In Java ogni classe possiede un metodo, di nome `finalize`(), chiamato alla terminazione dell'oggetto.
- Gli oggetti non vengono però eliminati subito, ma piuttosto sono passati a un garbage collector automatico.
- In questo caso occorre decrementare esplicitamente `numeroDiSoci`, chiamando l'operazione con ambito di classe `decrementaNumeroDiSoci`() prima di affidarlo al garbage collector.
- Il distruttore di C# ha la stessa semantica di Java, eccetto per il nome del metodo, che si chiama `Finalize`().

Individuazione delle classi di analisi – Attività UP: analizzare un caso d'uso

- Il flusso di lavoro UP "Analizzare un caso d'uso" viene schematizzato nella seguente figura:



- I prodotti del flusso di lavoro sono le classi di analisi e le realizzazioni dei casi d'uso.

- Le realizzazioni dei casi d'uso sono collaborazioni tra oggetti che mostrano come sistemi di oggetti che interagiscono tra di loro possono realizzare il comportamento del sistema descritto nei casi d'uso.

Individuazione delle classi di analisi – Attività UP: analizzare un caso d'uso

- Input dell'attività è rappresentato da:
  - Modello del business.
  - Modello dei requisiti; questi forniscono l'input per il processo di modellazione dei casi d'uso.
  - Modello dei casi d'uso: riguarda la creazione del modello dei casi d'uso.
  - Descrizione dell'architettura: riguarda un'istantanea degli aspetti significativi dell'architettura del sistema.

Individuazione delle classi di analisi – Cosa sono le classi di analisi?

- Le classi di analisi:
  - Rappresentano un'astrazione ben definita nel dominio del problema.
  - Dovrebbero corrispondere a concetti concreti del mondo del business.
- Il dominio del problema è quello in cui si è inizialmente manifestata l'esigenza di un sistema software.
- Normalmente si tratta di una specifica area di attività, quali le vendite telematiche o la gestione delle relazioni con i clienti.
- Il dominio del problema potrebbe anche non essere un'attività gestionale ma avere origine da un dispositivo software (sistema embedded).
- Tutto lo sviluppo di software gestionale ha per fine ultimo una qualche necessità aziendale.
- L'aspetto più importante di una classe di analisi è che deve corrispondere chiaramente e senza ambiguità a un concetto della realtà di business.
- Sebbene questa affermazione presuma che i concetti del business siano essi stessi chiari e non ambigui, questo succede di rado.

Individuazione delle classi di analisi – Cosa sono le classi di analisi?

- Il primo passo nella costruzione di un sistema OO è l'analisi del dominio del problema.
- La maggior parte di questo sforzo deve essere speso nel flusso di lavoro dei requisiti.
- Di contro, molti altri chiarimenti vengono prodotti durante la costruzione delle classi di analisi e delle realizzazioni dei casi d'uso.
- È importante che tutte le classi nel modello dell'analisi siano classi di analisi e non classi originate da aspetti di progettazione.
- Ciascuna classe di analisi verrà alla fine raffinata in una o più classi di progettazione.
- Individuare le giuste classi di analisi è la chiave dell'analisi e della progettazione OO.
- Pertanto, nel flusso di lavoro dell'analisi è essenziale dedicare il tempo necessario a garantire che sia stato individuato il corretto insieme di classi di analisi.
- Nel seguito del corso ci focalizzeremo soprattutto sullo sviluppo di sistemi gestionali.
- Tuttavia, lo sviluppo di sistemi embedded non è altro che un caso specifico del normale processo di sviluppo.

Individuazione delle classi di analisi – Cosa sono le classi di analisi?

- I sistemi gestionali sono normalmente dominati dai requisiti funzionali.
- Nei sistemi embedded spesso sono preponderanti i requisiti non funzionali che derivano dall'hardware in cui il sistema è incorporato.
- I requisiti sono importanti per tutti i tipi di sistema e anche per alcuni sistemi embedded, quali i sistemi di controllo di apparecchiature radiografiche a raggi x.

Individuazione delle classi di analisi – Cosa sono le classi di analisi? – Anatomia di una classe di analisi

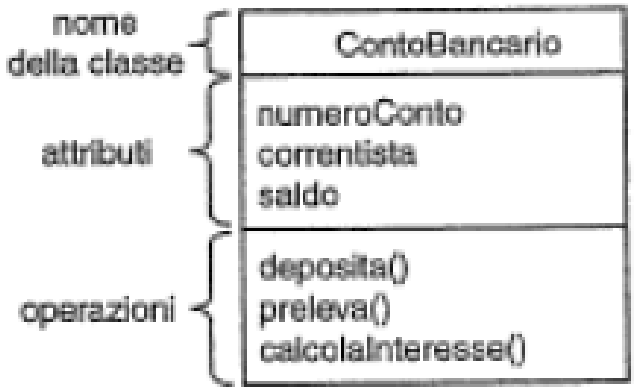
- Le classi di analisi dovrebbero contenere attributi molto ad “alto livello”.
- Le classi di analisi fissano gli attributi candidati per le classi di progettazione.
- Le operazioni di una classe di analisi specificano, ad alto livello, i servizi principali offerti dalla classe che, al momento della progettazione, diventeranno i metodi effettivamente implementabili.
- Può avvenire che un’operazione a livello di analisi venga scomposta in più di un’operazione a livello di progettazione.
- Nell’analisi si usa solo un piccolo sottoinsieme della sintassi UML per le classi.

Individuazione delle classi di analisi – Cosa sono le classi di analisi? – Anatomia di una classe di analisi

- Un formato minimale per una classe di analisi comprende quanto segue:
  - Nome: obbligatorio.
  - Attributi: il loro nome è obbligatorio, il loro tipo è opzionale.
  - Operazioni: possono anche essere solo delle affermazioni ad alto livello sulle responsabilità delle classe.
  - Visibilità: generalmente non la si esplicita.
  - Stereotipi: si possono indicare se migliorano il modello.
  - Valori etichettati: si possono indicare se migliorano il modello.

Individuazione delle classi di analisi – Cosa sono le classi di analisi? – Anatomia di una classe di analisi

- Un esempio di classi di analisi viene riportato nella seguente figura:



- L'idea è quella di fissare l'essenza di un'astrazione nella classe di analisi e di lasciare i dettagli relativi all'implementazione per quando si arriverà alla progettazione.

Individuazione delle classi di analisi – Cosa sono le classi di analisi? – Cosa contraddistingue una buona classe di analisi

- Le qualità di una buona classe di analisi si riassumono nei seguenti punti:
  - Il suo nome ne rispecchia l'intento.
  - È un'astrazione ben definita che modella uno specifico elemento nel dominio del problema.
  - Corrisponde a una caratteristica chiaramente identificabile del dominio del problema.
  - Ha un insieme ridotto e ben definito di responsabilità.
  - Ha la massima coesione interna.
  - Ha la minima interdipendenza con le altre classi.
- Nell'analisi si vuole modellare accuratamente e concisamente un aspetto del dominio del problema.
- Per esempio, nel modellare un cliente per un sistema bancario si vorranno includere il nome, l'indirizzo, etc.

Individuazione delle classi di analisi – Cosa sono le classi di analisi? – Cosa contraddistingue una buona classe di analisi

- Molte volte si può stabilire dal solo nome della classe se essa sia “buona” o no.
- Si deve sempre ricercare un’ “astrazione ben definita”, un qualcosa che possegga una semantica chiara ed evidente.
- Una responsabilità è un contratto o un obbligo che la classe ha verso i propri clienti.
- È cruciale che la classe abbia un insieme ben coeso di responsabilità in stretto accordo con il suo intento espresso dal nome.
- Ad esempio, la classe `CarrelloSpesa` ci sia aspetta che abbia responsabilità quali l’aggiunta di un prodotto al carrello, la rimozione di un prodotto dal carrello, la visualizzazione dei prodotti nel carrello.
- Questo è un insieme coeso di responsabilità.
- Queste tre responsabilità sono così coese che potrebbero essere riassunte con una di più alto livello chiamata “gestione del carrello”.

Individuazione delle classi di analisi – Cosa sono le classi di analisi? – Cosa contraddistingue una buona classe di analisi

- Si potrebbero anche aggiungere a `CarrelloSpesa` le responsabilità di verifica della carta di credito, di accettazione del pagamento e di stampa di una ricevuta.

- Queste nuove responsabilità non sembrano però accordarsi con l'intento e la semantica intuitiva dei carrelli della spesa.

- Le classi migliori hanno la minima interdipendenza con le altre classi.

- Concentrare responsabilità di controllo o assegnare troppe responsabilità a un'unica classe aumenta l'interdipendenza verso di essa.

Individuazione delle classi di analisi – Cosa sono le classi di analisi? – Regole pratiche per le classi di analisi

- Alcune regole pratiche per creare delle classi di analisi ben formate sono le seguenti:
  - Associare a ciascuna classe tra le tre e le cinque responsabilità.
  - Nessuna classe può essere isolata.
  - Evitare di avere tante classi troppo semplici.
  - Evitare di avere poche classi troppo complesse.
  - Evitare i “functoid”; un functoid è una normale funzione procedurale travestita da classe.
  - Evitare le classi onnipotenti.
  - Evitare gli alberi di ereditarietà profondi; un errore comune è quello di usare l’ereditarietà per creare una specie di decomposizione funzionale dove ogni livello di astrazione ha un’unica responsabilità. Cosa si intende per profondo?
    - Nell’analisi si considerano «profondi» tre livelli o più di ereditarietà.
    - Nella progettazione la definizione di «profondo» dipende dal linguaggio di programmazione.

Individuazione delle classi di analisi – Come si individuano le classi di analisi?

- Non esiste un semplice algoritmo di individuazione delle classi.
- Ciò nonostante esistono tecniche provate e sperimentate per trovare delle buone soluzioni.
- Individuare le classi “giuste” dipende comunque molto dalla percezione, dall’abilità e dall’esperienza del singolo analista.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – La tecnica dell'analisi nome/verbo

- L'analisi nome/verbo è un modo molto semplice di analizzare un testo al fine di individuare le classi.
- Nomi e frasi nominali indicano le classi o gli attributi delle classi mentre i verbi e le frasi verbali indicano le responsabilità o le operazioni di una classe.
- L'analisi nome/verbo viene usata da molti anni.
- Bisogna comunque fare attenzione a problemi di sinonimia e omonima che potrebbero dare origine a classi spurie.
- Se il dominio del problema è poco noto e mal definito occorre essere molto cauti.
- È opportuno, anche, studiare domini di problemi simili, esistenti al di fuori della propria organizzazione.
- L'aspetto forse più ingannevole dell'analisi nome/verbo è l'individuazione delle classi "nascoste".
- Un esempio di classe nascosta è la classe `Ordine` in un sistema di prenotazioni di una compagnia di viaggi.
- Ci si rende conto di aver individuato una classe nascosta perché introducendo questa nuova astrazione l'intero modello si assesta all'improvviso.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – La tecnica dell'analisi nome/verbo

- Come primo passo dell'analisi nome/verbo si raccolgono quante più informazioni rilevanti possibili.
- Le potenziali fonti di informazioni sono:
  - Il modello dei requisiti.
  - Il modello dei casi d'uso.
  - Il glossario di progetto.
  - Qualunque altro documento (architettura, visione aziendale, etc.).
- Dopo aver raccolto la documentazione, la si analizza semplicemente evidenziando i seguenti elementi:
  - I nomi, per esempio volo.
  - I predicati nominali, per esempio numero del volo.
  - I verbi, per esempio assegnare.
  - I predicati verbali, per esempio verificare la carta di credito.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – La tecnica dell'analisi nome/verbo

- I nomi e le classi nominali possono indicare classi o attributi di classe; i verbi e le frasi verbali possono indicare responsabilità di classe.
- Se, durante questo processo, si incontrano termini sconosciuti, è necessario farseli chiarire immediatamente e aggiungerli al glossario di progetto.
- A questo punto si prende la lista di nomi, frasi e verbi e si usa il glossario di progetto per risolvere tutti i casi di omonimia e di sinonimia.
- In questo modo si ottiene una lista di candidati per classi, attributi e responsabilità.
- Con la lista di candidati si predispone una prima assegnazione di attributi e responsabilità alle classi.
- Nel caso ci si sia fatta qualche idea sulle possibili relazioni tra le classi si possono anche inserire queste potenziali associazioni.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – L'analisi CRC

- CRC significa Classi, Responsabilità e Collaboratori (Class, Responsilities and Collaborators)
- Questa tecnica usa il più potente strumento di analisi del mondo, ovvero il post-it!
- Si incomincia preparando alcuni post-it, ciascuno suddiviso in tre scomparti; in quello superiore si scrive il nome della classe candidata, a sinistra le sue responsabilità e a destra i suoi collaboratori:



- I collaboratori di una classe sono le classi che possono lavorare con essa per realizzare una parte delle funzionalità del sistema.
- Un altro modo per fissare queste relazioni è quello di appiccicare i post-it in una lavagna e tracciare delle linee tra le classi che collaborano.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – L'analisi CRC

- L'analisi CRC dovrebbe essere sempre usata congiuntamente all'analisi nome/verbo.
- L'analisi CRC consente di separare la raccolta delle informazioni dall'analisi delle stesse.
- Conviene, quindi, eseguirla in due fasi:
  - Fase 1: Brainstorming – raccogliere informazioni.
  - Fase 2: Analizzare le informazioni raccolte.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – L'analisi CRC – Fase 1: Brainstorming

- I partecipanti sono gli analisti OO, i committenti e le altre parti interessate e gli esperti del dominio.
- La procedura è la seguente:
  - Spiegare ai partecipanti che si tratta di una sessione di brainstorming:
    - Tutte le idee saranno considerate buone idee.
    - Le idee vengono registrate ma non dibattute: non si discute nulla, ci si limita a scrivere tutto e a proseguire. Verrà tutto analizzato successivamente.
  - Chiedere ai partecipanti di fare il nome di "cose" che agiscono nel loro dominio del problema. Per esempio, cliente o prodotto.
    - Scrivere ogni cosa su un post-it: si tratta di una potenziale classe o di un potenziale attributo di una classe.
    - Appiccicare il post-it a una parete o a una lavagna.
  - Chiedere ai partecipanti di indicare le responsabilità che quelle cose potrebbero avere e scriverle nella sottosezione delle responsabilità del post-it.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – L'analisi CRC – Fase 1: Brainstorming

- Lavorando assieme ai partecipanti, cercare di individuare le classi che potrebbero interagire reciprocamente.

- Risistemare i post-it sulla lavagna per illustrare questa organizzazione e tracciare linee per congiungerle, oppure scrivere i collaboratori nell'apposita sottosezione dei post-it.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – L'analisi CRC – Fase 2: Analisi dell'informazione

- I partecipanti sono gli analisti OO e gli esperti del dominio.
- Certi post-it rappresenteranno dei concetti chiave del business e dovranno chiaramente diventare delle classi.
- Per altri post-it potrebbe essere più difficile prendere una decisione; se un post-it sembra essere logicamente una parte di un altro, questo è un buon indizio che forse si tratta di un attributo.
- Anche nel caso in cui un post-it non sembri essere particolarmente importante, oppure abbia un comportamento di scarso interesse, è necessario chiedersi se non sia possibile che si tratti di un attributo di un'altra classe.
- I post-it su cui restano dei dubbi vengono trattati come classi.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – Ricercare altre fonti di classi

- Oltre alle analisi nome/verbo e CRC e agli stereotipi RUP esistono anche altre fonti di potenziali classi che dovrebbero essere prese in considerazione.
- Queste possono essere cercate nel mondo reale.
- Alcuni esempi sono i seguenti:
  - Oggetti fisici.
  - Modulistica; questa deve essere esaminata con molta cautela.
  - Interfacce tra il sistema e il mondo esterno.
  - Entità concettuali essenziali per il funzionamento dell'organizzazione ma che non si manifestano in cose concrete; ad esempio una classe `IniziativaFedeltà` per gli sconti fedeltà ai clienti.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – Ricercare altre fonti di classi – Pattern archetipo

- I pattern archetipo sono pattern di concetti così pervasivi nei sistemi di business che si ritiene che siano veramente archetipi naturali.
- Per questa ragione possono essere modellati una volta e poi riutilizzati.
- Questi pattern possono essere usati così come sono oppure possono essere modificati per creare il proprio modello di analisi a partire dai componenti di base.
- Alcuni pattern archetipo sono i seguenti:
  - Customer Relationship Management.
  - Inventory.
  - Money.
  - Order.
  - Party.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – Ricercare altre fonti di classi – Pattern archetipo

  - Party relationship.
  - Product.
  - Quantity.
  - Rule.

- Ciascuno di questi pattern è molto dettagliato e complesso.
- Se si riesce a riutilizzare uno di questi pattern è possibile risparmiare giorni-uomo o anche mesi-uomo di lavoro.
- Questo probabilmente è il modo più efficace di trovare classi per i propri modelli: prenderle già fatte!

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – Creazione di una prima bozza di modello di analisi

- Per creare una prima bozza di modello di analisi occorre utilizzare uno strumento di modellazione per consolidare in un unico documento UML i prodotti dell'analisi nome/verbo, CRC e gli stereotipi RUP, nonché le considerazioni ricavate da altre fonti di classi.

- A tal fine è opportuno effettuare le seguenti attività:

  - Confrontare le tre fonti di informazioni.

  - Consolidare le classi di analisi; più specificatamente occorre:

    - Eliminare sinonimi e omonimi utilizzando il glossario di progetto.

    - Cercare le differenze tra i risultati delle tre tecniche.

  - I collaboratori (o le linee tra i post-it) rappresentano relazioni tra le classi; esse andranno modellate secondo quanto si vedrà nel seguito.

  - Normalizzare i nomi delle classi, degli attributi e delle responsabilità.

Individuazione delle classi di analisi – Come si individuano le classi di analisi? – Creazione di una prima bozza di modello di analisi

- Il prodotto di questa attività è un insieme di classi di analisi dove ognuna potrebbe avere alcuni attributi chiave e dovrebbe avere da tre a cinque responsabilità.

- Questa è la prima bozza del modello di analisi.

Relazioni – Cos'è una relazione?

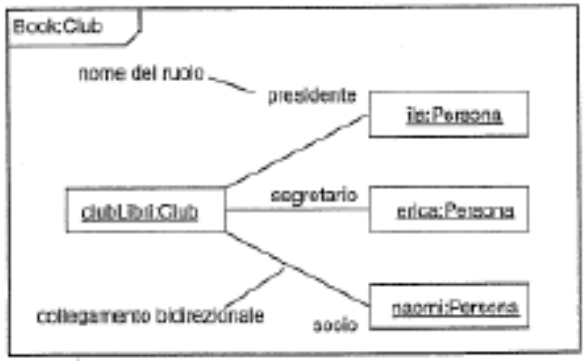
- Le relazioni sono connessioni semanticamente significative tra elementi di modellazione.
- Precedentemente sono stati già visti alcuni tipi di relazione:
  - Tra attore e caso d'uso (associazione).
  - Tra caso d'uso e caso d'uso (generalizzazione, <<include>>, <<extends>>).
  - Tra attore e attore (generalizzazione).
- Esaminiamo ora le connessioni tra oggetti, denominate collegamenti, e le connessioni tra classi, denominate associazioni.
- Le connessioni tra oggetti si chiamano collegamenti; quando gli oggetti operano insieme si dice che essi collaborano.
- Affinchè due oggetti possano comunicare direttamente tra di loro, le loro rispettive classi devono avere una qualche conoscenza l'una dell'altra.
- Le connessioni tra classi si chiamano associazioni.
- I collegamenti tra oggetti sono istanze delle associazioni tra le rispettive classi.

Relazioni – Cos'è un collegamento?

- Un programma OO in esecuzione non è altro che una comunità di oggetti che comunicano tra loro.
- Un collegamento è una connessione semantica tra due oggetti e consente loro di scambiarsi messaggi.
- Un sistema OO in esecuzione contiene molti oggetti che nascono e che muoiono, e molti collegamenti che nascono e muoiono, e che connettono questi oggetti tra di loro.
- Quando un oggetto riceve un messaggio invoca il corrispondente metodo.
- I linguaggi di programmazione OO realizzano i collegamenti in modi diversi.
- Il requisito minimo per stabilire un collegamento è che almeno uno dei due oggetti disponga di un riferimento dell'altro.

Relazioni – Cos'è un collegamento? – Diagrammi degli oggetti

- Un diagramma degli oggetti mostra gli oggetti, con le loro relazioni, in un dato istante.
- Gli oggetti collegati possono assumere dei ruoli l'uno rispetto all'altro.
- Un esempio di diagramma degli oggetti viene riportato nella seguente figura:



- L'oggetto `ila` assume il ruolo di presidente nel suo collegamento con l'oggetto `clubLibri`.
- L'oggetto `clubLibri` svolge sempre il ruolo di "club" e sarebbe inutile indicarlo nel diagramma.

Relazioni – Cos'è un collegamento? – Diagrammi degli oggetti

- I collegamenti sono connessioni dinamiche tra oggetti.
- Normalmente un singolo collegamento connette esattamente due oggetti.
- UML, tuttavia, consente anche di utilizzare un unico collegamento per connettere più di due oggetti.
- Nella figura precedente vi sono tre collegamenti tra quattro oggetti.
- Nella figura i collegamenti sono bidirezionali.
- Se un collegamento è unidirezionale si usa la navigabilità per specificare in quale direzione i messaggi possono passare lungo il collegamento.
- Si può specificare la navigabilità aggiungendo una freccia (navigabile) o una croce (non navigabile) all'estremità di un collegamento.
- La specifica UML 2 consente tre diversi idiomi di modellazione per visualizzare la navigabilità; essi verranno discussi dettagliatamente nel seguito.

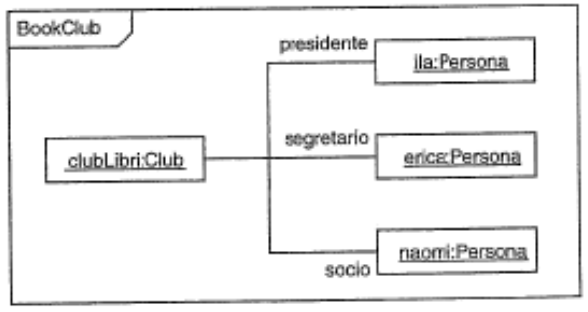
Relazioni – Cos'è un collegamento? – Diagrammi degli oggetti

- Nel nostro caso utilizzeremo l'idioma più comune che prevede che:
  - Tutte le croci vengano eliminate.
  - Le associazioni bidirezionali siano prive di frecce.
  - Le associazioni unidirezionali abbiano una sola freccia.
- Un esempio di collegamento unidirezionale:



Relazioni – Cos'è un collegamento? – Cammini

- Un cammino è una "sequenza connessa di segmenti" che connette due o più simboli.
- Ci sono tre stili per tracciare i cammini:
  - Ortogonale, in cui il cammino è costituito da segmenti orizzontali e verticali.
  - Obliquo, in cui il cammino è costituito da una o più linee inclinate.
  - Curvo, in cui il cammino è costituito da una curva.
- I gusti personali dettano quale stile usare; molti modellatori preferiscono lo stile ortogonale.
- Un esempio di stile ortogonale viene riportato nella seguente figura:



- L'ordine, la leggibilità e l'aspetto generale dei diagrammi sono di importanza cruciale.

Relazioni – Cos'è un'associazione?

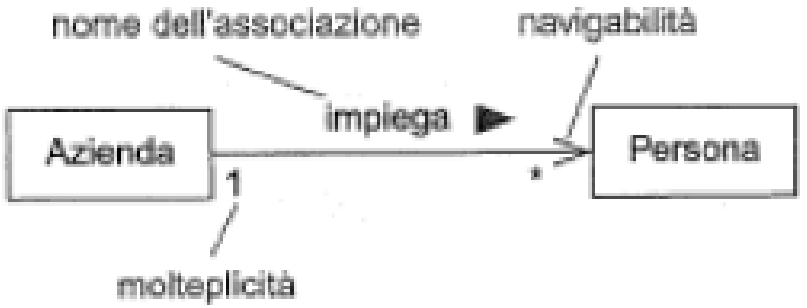
- Così come i collegamenti connettono gli oggetti, le associazioni connettono le classi.
- Le relazioni tra oggetti e classi e tra collegamenti e associazioni sono riportati nella seguente figura:



- La semantica delle associazioni, nella loro forma base, non raffinata, è semplice: un'associazione tra classi indica che si possono avere dei collegamenti tra i corrispettivi oggetti.
- Esistono forme di associazione più complesse (aggregazione e composizione).

Relazioni – Cos'è un'associazione? – Sintassi per le associazioni

- Per ciascuna associazione è possibile definire un nome, i nomi dei ruoli, le molteplicità e la navigabilità.
- I nomi delle associazioni dovrebbero essere verbi o frasi verbali poiché indicano un'azione che l'oggetto origine esegue sull'oggetto destinazione.
- Il nome può essere preceduto o seguito da una freccia nera che indica in quale direzione si deve leggere.
- I nomi sono scritti in lowerCamelCase, iniziando con una lettera minuscola.
- L'associazione nella figura seguente si legge "un'azienda impiega molte persone" e "ogni persona è impiegata da una sola azienda":

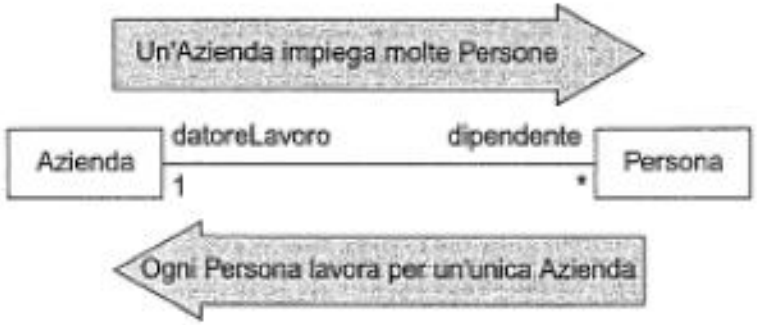


Relazioni – Cos'è un'associazione? – Sintassi per le associazioni

- In alternativa si possono assegnare nomi ai ruoli delle classi a una, o entrambe, le estremità dell'associazione.
- Nella figura precedente l'oggetto `Azienda` avrà il ruolo *datoreLavoro* mentre l'oggetto `Persona` avrà il ruolo *dipendente*.
- I ruoli dovrebbero essere nomi o frasi nominali.
- L'utilizzo di entrambe le notazioni è fortemente sconsigliato.
- Il segreto per scegliere buoni nomi di associazioni e ruoli è che essi devono suonare bene quando vengono letti.

Relazioni – Cos'è un'associazione? – Molteplicità

- I vincoli sono uno dei tre meccanismi di estendibilità in UML.
- La molteplicità è di gran lunga il vincolo più usato.
- La molteplicità limita il numero di oggetti di una classe che possono partecipare in una relazione in un dato istante.
- Nella figura seguente in ogni istante un oggetto `Persona` è dipendente da esattamente un oggetto `Azienda`; tuttavia, nel corso del tempo, uno stesso oggetto `Persona` potrebbe essere dipendente da una successione di diversi oggetti `Azienda`.



- Nella figura un oggetto `Persona` non può mai essere disoccupato: è sempre dipendente di esattamente un oggetto `Azienda`.

Relazioni – Cos'è un'associazione? – Molteplicità

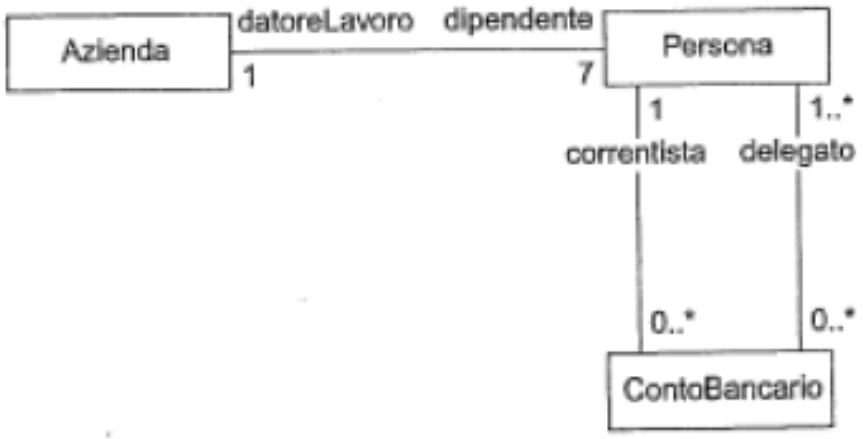
- Tale vincolo esprime due regole di business:
  - Gli oggetti `Persona` possono essere dipendenti di un'`Azienda` per volta.
  - Gli oggetti `Persona` devono sempre essere dei dipendenti.
- Se questi siano dei vincoli ragionevoli o meno dipende esclusivamente dai requisiti del sistema che viene modellato
- I vincoli di molteplicità sono importantissimi perché codificano nel modello delle regole di business fondamentali che sono "sepolte" nel modello.
- La molteplicità viene specificata come una lista di intervalli separati da virgole, ognuno della forma `minimo..massimo`.
- Se la molteplicità non viene indicata esplicitamente allora è indefinita; in UML non esiste un valore predefinito per la molteplicità.

Relazioni – Cos'è un'associazione? – Molteplicità

- Alcuni esempi di sintassi per la molteplicità sono mostrati nella seguente tabella:

Ornamento	*Significato*
0..1	zero o 1
1	esattamente 1
0..*	zero o più
*	zero o più
1..*	1 o più
1..6	da 1 a 6
1..3, 7..10, 15, 19..*	da 1 a 3 *oppure* da 7 a 10 *opppure* esattamente 15 *oppure* 19 o più

- La molteplicità è un vincolo potente che di solito ha un impatto notevole sulla semantica di business del modello.
- Consideriamo la seguente figura:



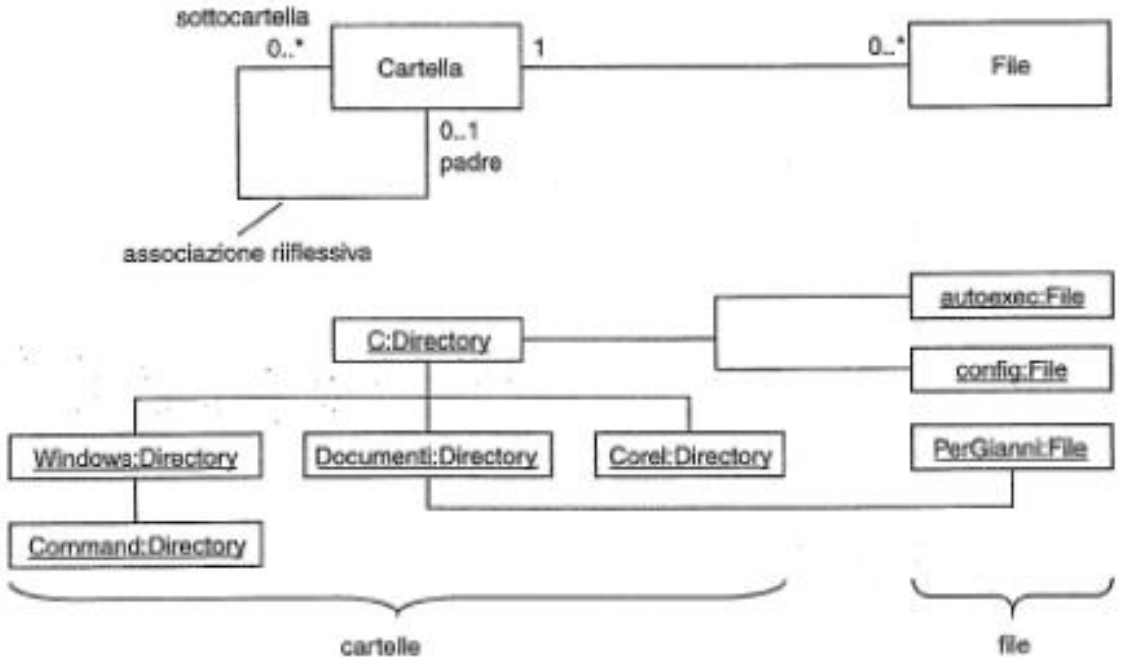
Relazioni – Cos'è un'associazione? – Molteplicità

- Leggendo attentamente l'esempio si osserva che:
  - Un'`Azienda` ha esattamente sette `dipendenti`.
  - Una `Persona` può essere dipendente di esattamente un'`Azienda`.
  - Un `ContoBancario` può avere un solo `correntista`.
  - Un `ContoBancario` può avere uno o più `delegati`.
  - Una `Persona` può essere correntista di zero o più di un `ContoBancario`.
  - Una `Persona` può essere delegato di zero o più di un `ContoBancario`.
- Leggendo un modello è vitale ricavarne esattamente il significato letterale, piuttosto che fare assunzioni o inventarsi significati inesistenti.
- È oggetto di discussione se la molteplicità debba o meno essere indicata nei modelli di analisi.

Relazioni – Cos'è un'associazione? – Molteplicità – Associazioni riflessive

- È piuttosto comune che una classe abbia un'associazione con se stessa; questa associazione viene chiamata associazione riflessiva.

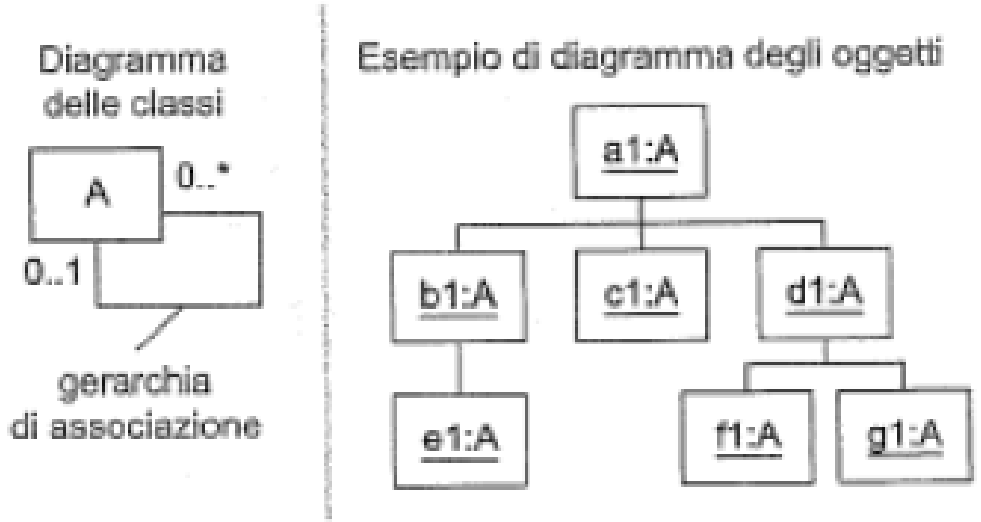
- Un esempio di associazione riflessiva viene riportata nella seguente figura:



- Nella metà superiore della figura si vede il diagramma delle classi mentre la metà inferiore illustra un esempio di diagramma degli oggetti corrispondente a quello delle classi.

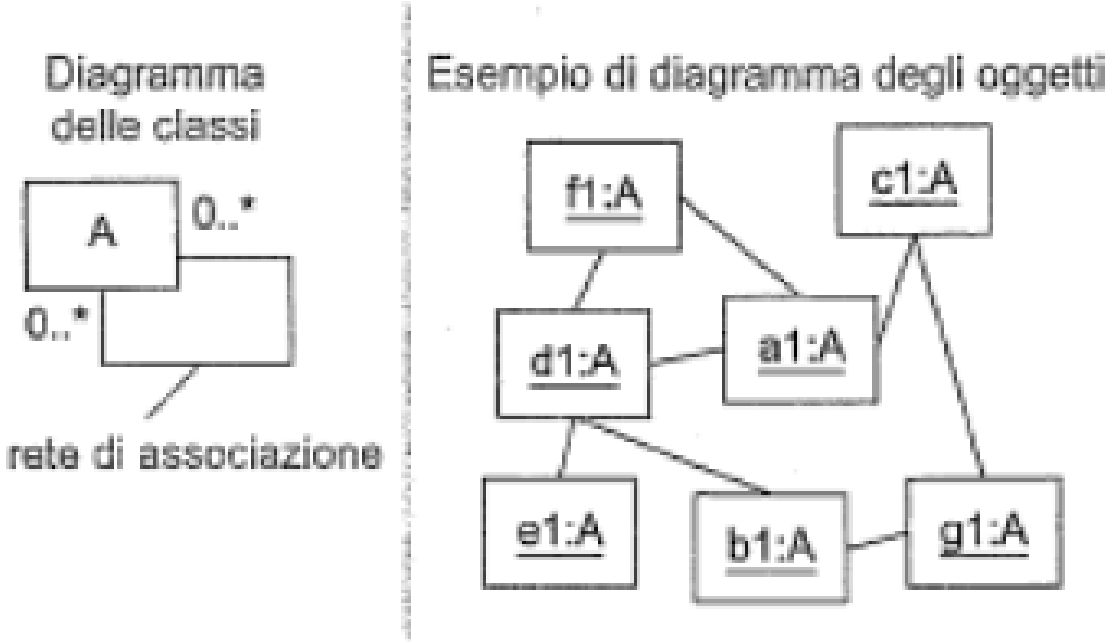
Relazioni – Cos'è un'associazione? – Molteplicità – Gerarchie e reti

- Gli oggetti si organizzano in gerarchie o reti.
- Le gerarchie organizzano gli oggetti in modo molto ordinato e strutturato ma piuttosto rigido.
- Un esempio di gerarchia viene riportato nella seguente figura:



Relazioni – Cos'è un'associazione? – Molteplicità – Gerarchie e reti

- Con le reti, invece, spesso non si ha un oggetto radice; ogni oggetto può essere direttamente connesso a molti altri.
- La rete è una struttura molto più flessibile, in cui non esiste un concetto implicito di livello "superiore" o "inferiore", e può accadere che nessun nodo abbia la supremazia sugli altri.
- Un esempio di rete è riportato nella seguente figura:



Relazioni – Cos'è un'associazione? – Navigabilità

- La navigabilità indica che è possibile spostarsi da un qualsiasi oggetto della classe origine a uno o più oggetti della classe destinazione.
- La navigabilità può essere letta come "i messaggi possono essere inviati solo nella direzione della freccia".
- Un esempio di navigabilità è riportato nella seguente figura:



- La navigabilità è un buon mezzo per ridurre l'interdipendenza tra classi.
- La navigabilità viene indicata aggiungendo una croce o una freccia all'estremità della relazione.

Relazioni – Cos'è un'associazione? – Navigabilità

- La specifica UML 2 suggerisce tre idiomi di modellazione per usare la navigabilità sui diagrammi:
  - Rendere la navigabilità completamente esplicita: tutte le frecce e le croci devono essere visualizzate.
  - Rendere la navigabilità completamente invisibile: non vengono visualizzate né croci né frecce.
  - Eliminare tutte le croci: le associazioni bidirezionali non hanno frecce, le associazioni unidirezionali ne hanno una sola.

Relazioni – Cos'è un'associazione? – Navigabilità

- Tali idiomi sono riportati nella seguente figura:

Idiomi di navigabilità UML 2			
Sintassi UML 2	Idioma 1: navigabilità precisa di UML 2	Idioma 2: nessuna navigabilità	Idioma 3: pratica standard
	A verso B è navigabile B verso A è navigabile		
	A verso B è navigabile B verso A non è navigabile		
	A verso B è navigabile B verso A è indefinito		A verso B è navigabile B verso A non è navigabile
	A verso B è indefinito B verso A è indefinito	A verso B è indefinito B verso A è indefinito	A verso B è navigabile B verso A è navigabile
	A verso B non è navigabile B verso A non è navigabile		

L'idioma 1 rende la navigabilità completamente visibile.

L'idioma 2 di solito andrebbe evitato.

L'idioma 3 è un compromesso ragionevole e, in effetti, è l'opzione quasi sempre usata.

Relazioni – Cos'è un'associazione? – Navigabilità

- I principali vantaggi dell'idioma 3 sono che non affolla i diagrammi con troppe frecce ed è compatibile con le versioni precedenti di UML.
- Tuttavia, esso ha alcuni lati negativi:
  - Non è possibile capire dal diagramma se la navigabilità è presente o se non è ancora stata definita.
  - Il significato della freccia cambia da navigabile/indefinita a navigabile/non navigabile.
  - Non è possibile visualizzare associazioni che non sono navigabili in nessuna delle due direzioni.
- È possibile che una direzione sia percorribile anche nella direzione lungo la quale non è navigabile, ma in modo probabilmente poco efficiente.
- Nell'implementazione con linguaggi di programmazione OO, la navigabilità implica che l'oggetto origine conserva un riferimento all'oggetto destinazione e che può utilizzare questo riferimento per inviare messaggi all'oggetto destinazione.
- In un diagramma degli oggetti questo può essere indicato mediante un collegamento unidirezionale associato a un oggetto.

Relazioni – Cos'è un'associazione? – Associazioni e attributi

- Esiste una correlazione forte tra le associazioni e gli attributi delle classi.
- Non essendovi nei linguaggi di programmazione OO comunemente usati dei costrutti specifici per le associazioni, quando il codice viene generato automaticamente da un modello UML, le associazioni uno-a-uno vengono trasformate in attributi della classe origine.
- Un esempio di traduzione di un'associazione uno-a-uno viene riportato nella seguente figura:



Se una relazione navigabile ha un ruolo, allora è come se la classe origine avesse uno pseudo-attributo con lo stesso nome del ruolo e con lo stesso tipo della classe destinazione.

Relazioni – Cos'è un'associazione? – Associazioni e attributi

- Il seguente codice Java è stato generato partendo dal modello della figura precedente:

```
Public class Casa
{
        private Indirizzo indirizzo
}
```

- Le molteplicità superiori a 1 si realizzano come:
  - Un attributo di tipo array.
  - Un attributo che abbia per tipo una qualche classe collezione; queste sono delle classi le cui istanze sono dedicate a contenere riferimenti ad altri oggetti.
- Il concetto di pseudo-attributo inizia a dare problemi quando si prendono in considerazione le relazioni molti-a-molti.
- Si usano le associazioni solo quando la classe destinazione è una parte importante del modello.

Relazioni – Cos'è un'associazione? – Associazioni e attributi

- Entro certi limiti la scelta tra associazioni esplicite e attributi è una questione di stile.
- Spesso è più chiaro mostrare un'associazione verso un'altra classe piuttosto che esprimere la stessa relazione mediante un attributo.
- Una molteplicità della destinazione superiore a 1 è un buon indizio che la classe destinazione è importante nel modello e, di conseguenza, si preferisce usare associazioni per rappresentare questo tipo di relazione.
- Se la molteplicità è esattamente 1, la destinazione potrebbe, in realtà, essere una mera parte dell'origine e potrebbe non valere la pena di rappresentarla come associazione.

Relazioni – Cos'è un'associazione? – Classi associazione

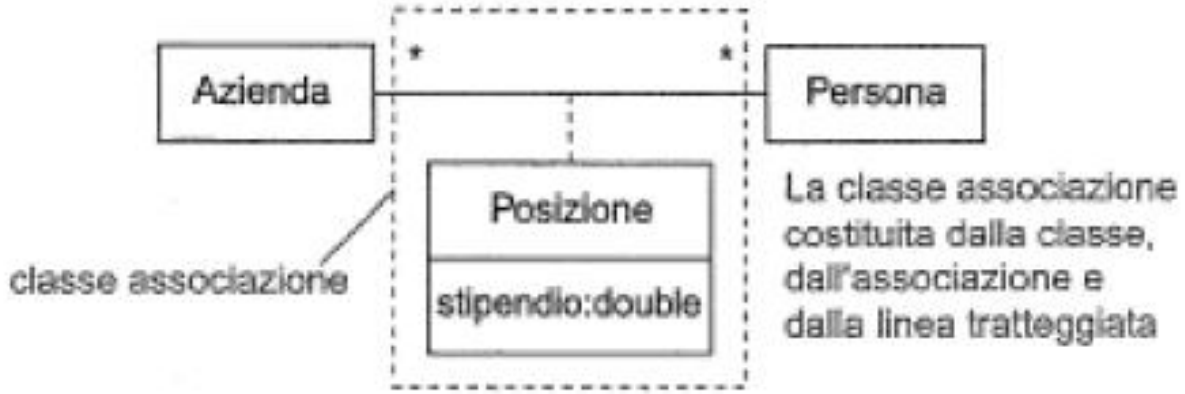
- Consideriamo la seguente relazione:



- A prima vista sembrerebbe un modello del tutto innocuo.
- Cosa accade se si aggiunge una regola per cui ogni `Persona` percepisce uno stipendio da ogni `Azienda` in cui è impiegata?
- Lo stipendio è una proprietà dell'associazione stessa.
- UML consente di modellare questa situazione con una classe associazione.

Relazioni – Cos'è un'associazione? – Classi associazione

- Un esempio di utilizzo di una classe associazione viene riportato nella seguente figura:

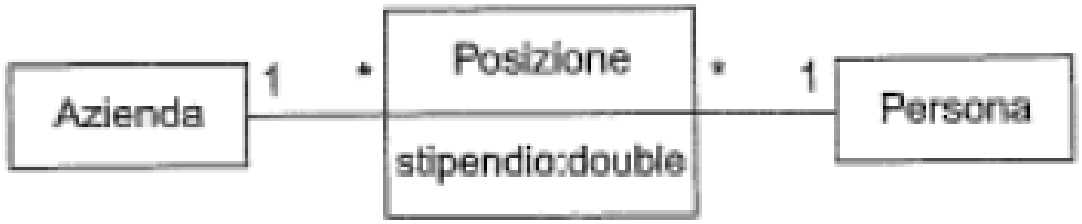


- La classe associazione è, in effetti, un'associazione che è anche una classe.
- Le istanze dell'associazione sono collegamenti, dotati di attributi e operazioni, la cui identità individuale è stabilita esclusivamente dalle identità degli oggetti alle estremità del collegamento.
- Questo implica che un'associazione può essere usata solo se tra una data coppia di oggetti si ha, in ogni momento, un solo collegamento univocamente determinato.

Relazioni – Cos'è un'associazione? – Classi associazione

- Se, invece, la situazione da modellare prevede che un dato oggetto `Persona` possa avere più di una `Posizione` in una specifica `Azienda`, allora non si può usare una classe associazione, semplicemente perché la semantica del modello non corrisponderebbe alla realtà modellata

- In questo caso è necessario effettuare un'operazione di reificazione sulla classe associazione `Posizione`, trasformandola in una classe:



- Molti modellatori non comprendono bene la differenza semantica tra classi associazione e relazioni reificate e le usano spesso in modo intercambiabile.

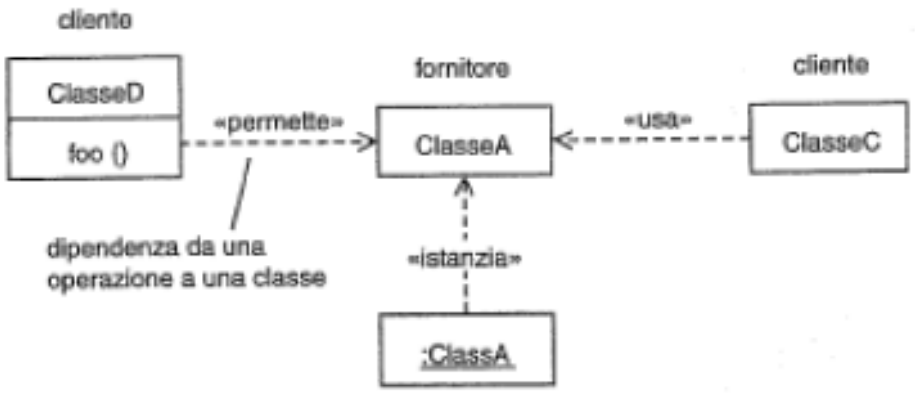
Relazioni – Cos'è un'associazione? – Cos'è una dipendenza?

- Una dipendenza indica una relazione tra due o più elementi del modello, dove un cambiamento a uno di essi (il fornitore) può influenzare o fornire delle informazioni necessarie all'altro (il cliente).
- Si usano le dipendenze per descrivere relazioni tra classificatori, in cui un classificatore dipende in qualche modo da un altro.
- Questa relazione non è propriamente un'associazione o una generalizzazione.
- Per modellare questo tipo di relazioni, si possono utilizzare le dipendenze (specializzate con stereotipi predefiniti).
- Esistono tre tipi base di dipendenze. Essi sono riportati nella seguente tabella:

Tipo	*Significato*
Uso	Il cliente utilizza alcuni dei servizi resi disponibili dal fornitore per implementare il proprio comportamento
Astrazione	Una relazione tra cliente e fornitore in cui il fornitore è più astratto del cliente
Permesso	Il fornitore assegna al cliente un qualche tipo di permesso di accesso al proprio contenuto: in questo modo il fornitore limita e controlla gli accessi al proprio contenuto

Relazioni – Cos'è un'associazione? – Cos'è una dipendenza?

- Le dipendenze esistono non solo tra le classi ma anche tra package e package e tra oggetti e classi.
- Possono anche esistere dipendenze tra un'operazione e una classe, ma è piuttosto raro vederle esplicitate in un diagramma.
- Alcuni esempi dei diversi tipi di dipendenza sono riportati nella seguente figura:



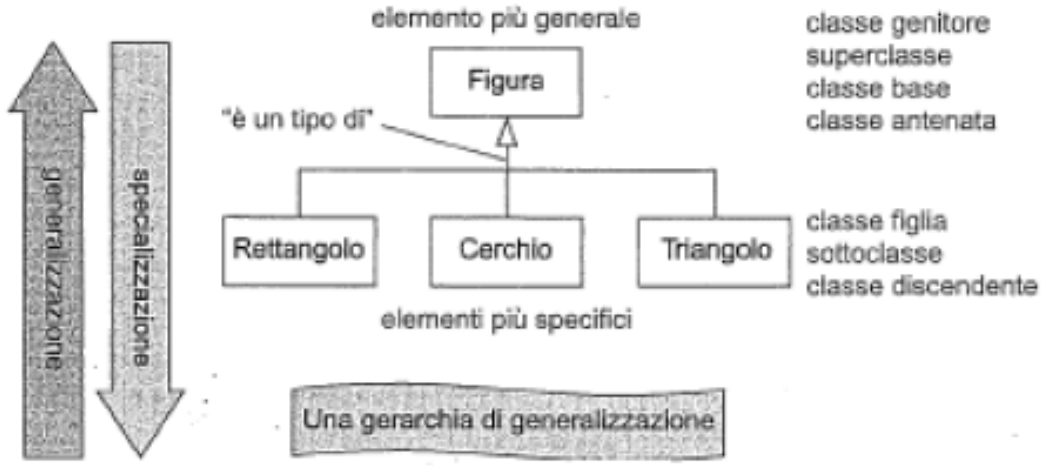
- Il più delle volte, per descrivere una dipendenza, si usa una linea tratteggiata priva di ornamenti, senza indicare il tipo.
- Qualora lo si desideri o sia necessario essere più specifici sul tipo di dipendenza, UML definisce un insieme completo di stereotipi.

Ereditarietà e polimorfismo – Generalizzazione

- La generalizzazione è una relazione tra un elemento generico e uno più specializzato in cui l'elemento più specializzato è completamente consistente con quello più generico ma contiene più informazione.
- I due elementi obbediscono al principio di sostituibilità.
- Questo tipo di relazione è molto più forte di quanto non lo sia l'associazione.

Ereditarietà e polimorfismo – Generalizzazione – Classi e generalizzazione

- La generalizzazione è un'idea concettualmente semplice.
- La generalizzazione può essere applicata a tutti i classificatori, e anche alcuni elementi di modellazione, quali le associazioni, gli stati, gli eventi e le collaborazioni.
- Un esempio di generalizzazione tra classi viene riportato nella seguente figura:



Ereditarietà e polimorfismo – Generalizzazione – Classi e generalizzazione

- Possiamo usare un'istanza di una sottoclasse di `Figura` ovunque sia ammesso l'uso di un'istanza della superclasse `Figura`.

- La gerarchia illustrata può essere ottenuta mediante un processo di specializzazione oppure tramite un processo di generalizzazione.

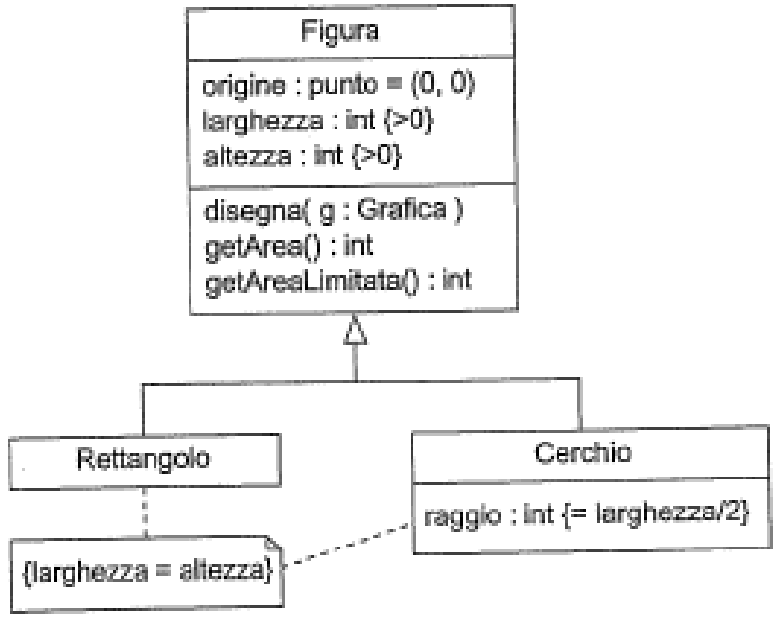
- Gli analisti OO tendono ad usare sia la specializzazione che la generalizzazione ma l'esperienza suggerisce che è meglio imparare a individuare i concetti generici quanto prima possibile.

Ereditarietà e polimorfismo – Ereditarietà di classe

- Quando si organizzano le classi in una gerarchia di generalizzazione si assume implicitamente l'esistenza di ereditarietà tra le classi.
- Le sottoclassi ereditano attributi, operazioni, relazioni e vincoli.
- Le sottoclassi possono anche aggiungere nuove caratteristiche e ridefinire le operazioni delle superclassi.

Ereditarietà e polimorfismo – Ereditarietà di classe – Ridefinizione

- Consideriamo la seguente figura:

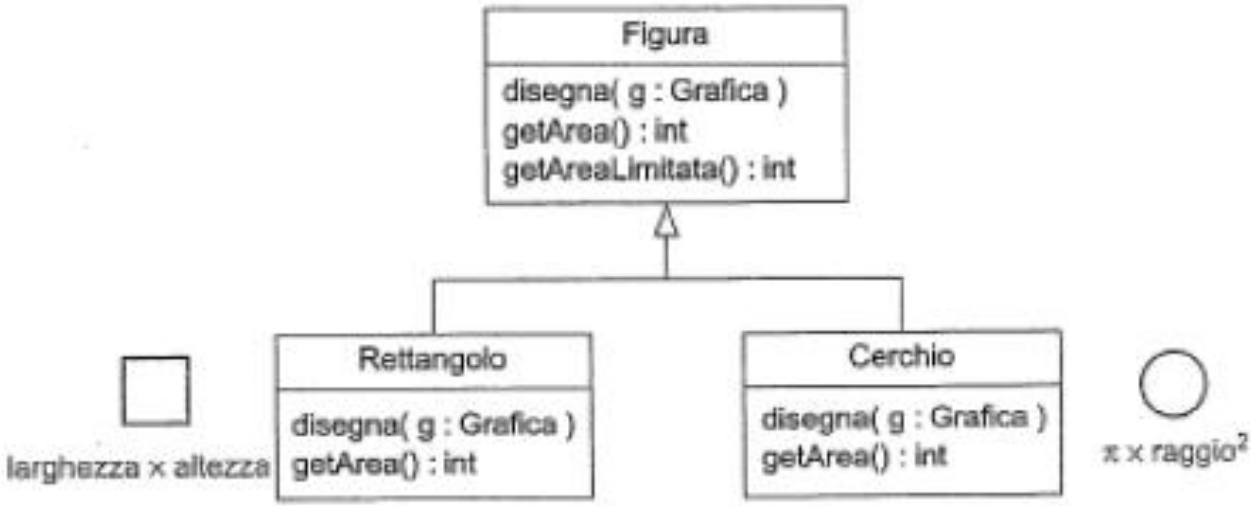


- Le sottoclassi `Rettangolo` e `Cerchio` ereditano tutti gli attributi, le operazioni e i vincoli della superclasse `Figura`.

- Queste caratteristiche sono implicitamente presenti nelle sottoclassi nonostante non siano indicate esplicitamente nel modello.

Ereditarietà e polimorfismo – Ereditarietà di classe – Ridefinizione

- Le operazioni `disegna()` e `getArea()` definite in `Figura` non sono in alcun modo appropriate per le sottoclassi.
- L'operazione `disegna()` predefinita, che entrambe le sottoclassi ereditano dalla loro classe genitore, non può essere adeguata; questa operazione potrebbe anche non disegnare alcunchè.
- Le sottoclassi devono poter modificare il comportamento della superclasse.
- La figura seguente mostra come si può risolvere questo problema: le sottoclassi `Rettangolo` e `Cerchio` hanno le loro versioni specifiche delle operazioni `disegna()` e `getArea()` che implementano comportamenti appropriati.



Ereditarietà e polimorfismo – Ereditarietà di classe – Ridefinizione

- Per ridefinire un'operazione della superclasse, una sottoclasse deve avere una sua operazione con esattamente la stessa segnatura dell'operazione nella superclasse.

- UML definisce la segnatura di un'operazione come l'insieme costituito dal nome, dal tipo restituito e dall'elenco ordinato dei tipi di tutti i parametri.

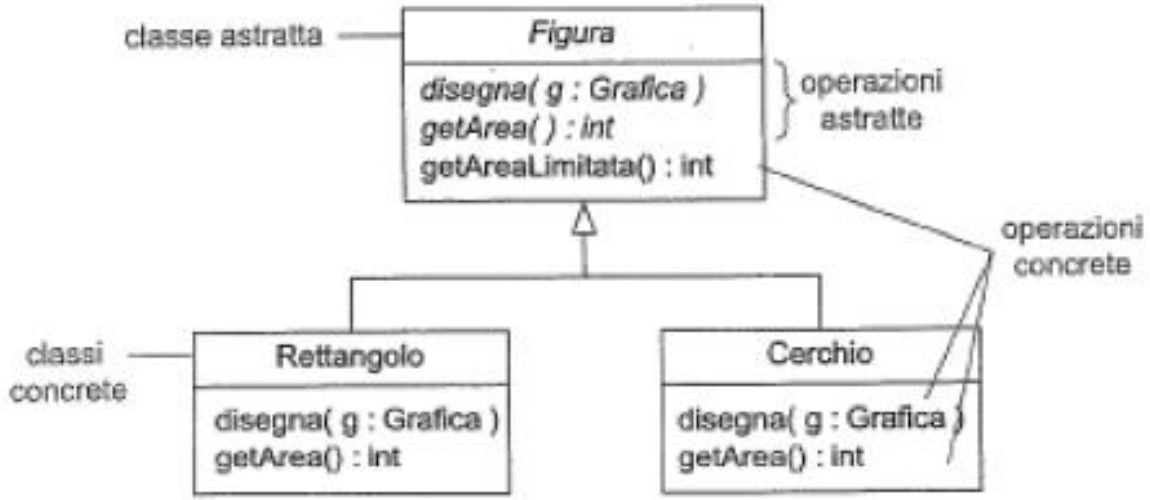
- I linguaggi di programmazione possono definire la segnatura di un'operazione in modo diverso; per esempio, in C++ e Java il tipo restituito dall'operazione non fa parte della segnatura.

Ereditarietà e polimorfismo – Ereditarietà di classe – Classi e operazioni astratte

- A volte si preferirebbe demandare direttamente l'implementazione di un'operazione alle sottoclassi specifiche; ad esempio, non sembra plausibile implementare `Figura::disegna(g:Grafica)`.
- È possibile specificare che un'operazione è priva di implementazione; in questo caso si dice che l'operazione è astratta.
- In UML le operazioni astratte sono semplicemente indicate scrivendone il nome in corsivo.
- Una classe con una o più operazioni astratte è incompleta e non può essere istanziata; la classe viene detta astratta.
- In UML le classi astratte sono semplicemente indicate scrivendone il nome in corsivo.

Ereditarietà e polimorfismo – Ereditarietà di classe – Classi e operazioni astratte

- Nella seguente figura viene riportato un esempio di classe astratta e di operazioni astratte:



- Sebbene `Figura` sia una classe incompleta e non possa essere istanziata le sue sottoclassi sono complete e possono essere istanziate.

- Le classi che possono essere istanziate sono dette concrete.

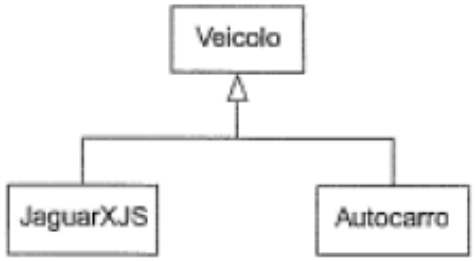
- L'operazione `getAreaLimitata()` è un'operazione concreta di `Figura` in quanto l'area limitata di qualunque `Figura` viene calcolata sempre allo stesso modo.

Ereditarietà e polimorfismo – Ereditarietà di classe – Classi e operazioni astratte

- L'utilizzo di classi e operazioni astratte offre un paio di importanti vantaggi:
  - Nella superclasse astratta `Figura` è possibile definire un insieme di operazioni astratte che tutte le sottoclassi sono tenute ad implementare.
  - È possibile scrivere del codice che manipola oggetti di classe `Figura` e poi sostituire opportunamente oggetti di classe `Cerchio`, `Rettangolo` o di altre sottoclassi di `Figura`.

Ereditarietà e polimorfismo – Ereditarietà di classe – Livelli di astrazione

- Consideriamo la seguente figura:



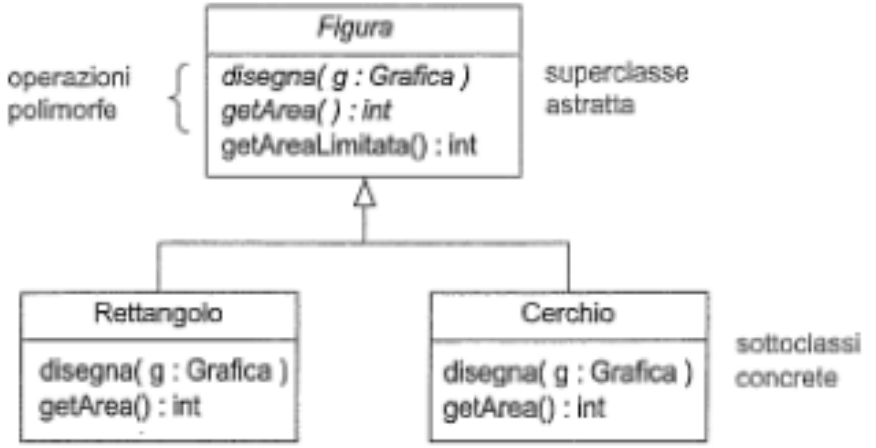
- Essa, a livello concettuale, è sbagliata.
- Una gerarchia di generalizzazione definisce un insieme di livelli di astrazione, dal più generale al più specializzato.
- Si dovrebbe sempre mantenere un livello di astrazione uniforme all'interno di ciascun livello della gerarchia di generalizzazione.
- Nella figura sopra `JaguarXSJ` è un tipo di automobile e si trova chiaramente a un livello di astrazione più basso rispetto ad `Autocarro`.
- Il modello può essere facilmente corretto introducendo una superclasse `Automobile` tra `JaguarXJS` e `Veicolo`.

Ereditarietà e polimorfismo – Ereditarietà di classe – Ereditarietà multipla

- In UML una classe può avere più di una superclasse diretta; questo viene definito ereditarietà multipla.
- La sottoclasse eredita da tutte le sue superclassi dirette.
- L'ereditarietà multipla normalmente è considerata un problema di progettazione.

Ereditarietà e polimorfismo – Polimorfismo

- Polimorfismo significa capacità di presentarsi sotto molte forme.
- Nell'esempio precedente le operazioni astratte `disegna()` e `getArea()` della classe `Figura` sono polimorfe perché possono avere due implementazioni differenti.
- Un esempio di polimorfismo viene riportato nella seguente figura:



- `Rettangolo` e `Cerchio` forniscono un'implementazione differente di `disegna()` e `getArea()`.
- L'incapsulamento, l'ereditarietà e il polimorfismo sono i tre pilastri della programmazione orientata agli oggetti.

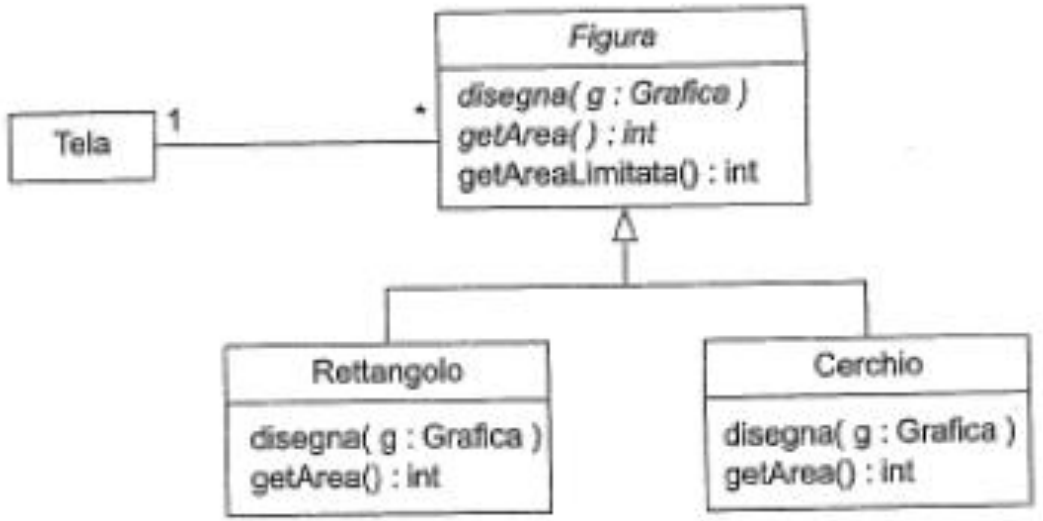
Ereditarietà e polimorfismo – Polimorfismo

- Il polimorfismo permette di progettare sistemi più semplici perché consente di trattare oggetti differenti allo stesso modo.

- Ciò che rende il polimorfismo un aspetto essenziale della programmazione OO è la possibilità di inviare lo stesso messaggio a oggetti di classi differenti pur consentendo a tali oggetti di rispondere in modo appropriato.

Ereditarietà e polimorfismo – Polimorfismo – Esempi di polimorfismo

- Consideriamo il seguente esempio:

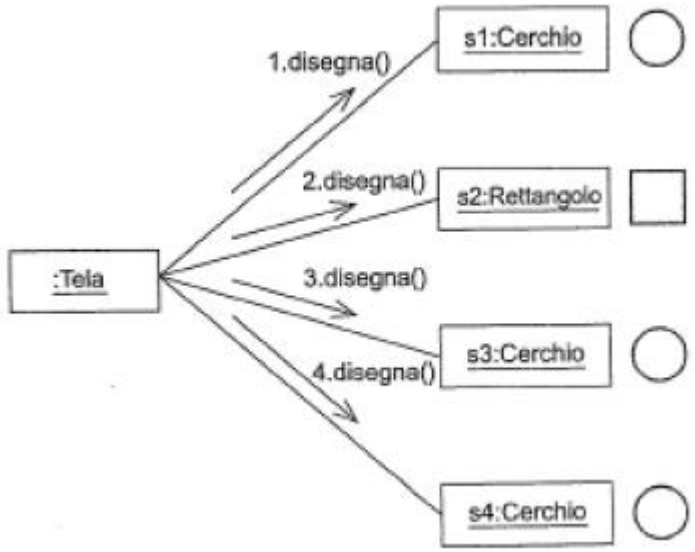


- Non è possibile creare un'istanza della classe astratta `Figura` ma si possono creare istanze delle sue sottoclassi concrete e usarle ovunque si faccia riferimento a `Figura`.

- Nella figura un oggetto `Tela` contiene un insieme di molti oggetti `Figura`; gli unici oggetti che possono essere effettivamente inseriti in questo insieme sono le istanze delle sottoclassi concrete di `Figura`.

Ereditarietà e polimorfismo – Polimorfismo – Esempi di polimorfismo

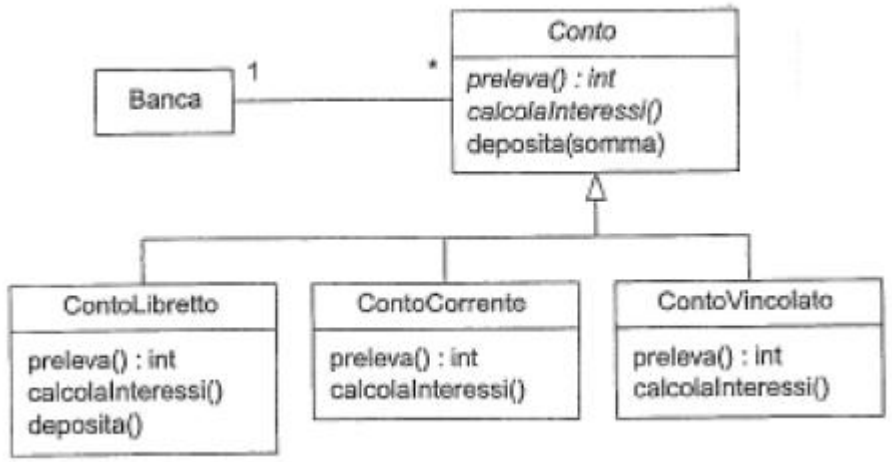
- Un modello ad oggetti del diagramma della figura precedente viene riportato nella seguente figura:



- Ogni oggetto risponde ad un messaggio invocando l'operazione corrispondente specificata dalla propria classe.
- Tutti gli oggetti della stessa classe risponderanno allo stesso messaggio invocando la stessa operazione.
- Questo non significa che tutti gli oggetti della stessa classe risponderanno allo stesso messaggio esattamente allo stesso modo in quanto i risultati di una chiamata dipendono dallo stato dell'oggetto.

Ereditarietà e polimorfismo – Polimorfismo – Esempi di polimorfismo

- Consideriamo il seguente ulteriore esempio:



- Le regole di gestione per i prelievi e il calcolo degli interessi sono diverse a seconda del tipo di conto bancario.

- Si osservi che in `Conto` viene anche definita l'operazione concreta `deposita()` di cui però la classe concreta `ContoLibretto` ridefinisce l'implementazione.

- Per ridefinire un'operazione di una classe base occorre che la sottoclasse disponga di un'operazione con identica segnatura.

Ereditarietà e polimorfismo – Polimorfismo – Esempi di polimorfismo

- Questa ridefinizione è utile perché vi sono regole di business che rendono diverso il processo di deposito di denaro su un `ContoLibretto` rispetto al deposito effettuato su altri tipi di `Conto`.
- Dal momento che esistono due implementazioni di `deposita()`, questa operazione è polimorfa.
- Anche un'operazione concreta, come `deposita`, può essere quindi polimorfa.
- È necessario dedicare particolare attenzione alla ridefinizione delle operazioni concrete.
- Le operazioni astratte possono sempre essere ridefinite con sicurezza.
- Ridefinire delle operazioni concrete può avere effetti collaterali imprevisti e potrebbe essere rischioso.
- Spesso l'operazione della sottoclasse fa qualcosa in più ma poi richiama l'operazione della superclasse.
- Questo modo di procedere è legittimo ed è generalmente sicuro.
- Alcuni linguaggi consentono di impedire alle sottoclassi di ridefinire un metodo concreto della superclasse; ad esempio, Java con la clausola `final`.

Package di analisi – Che cos'è un package?

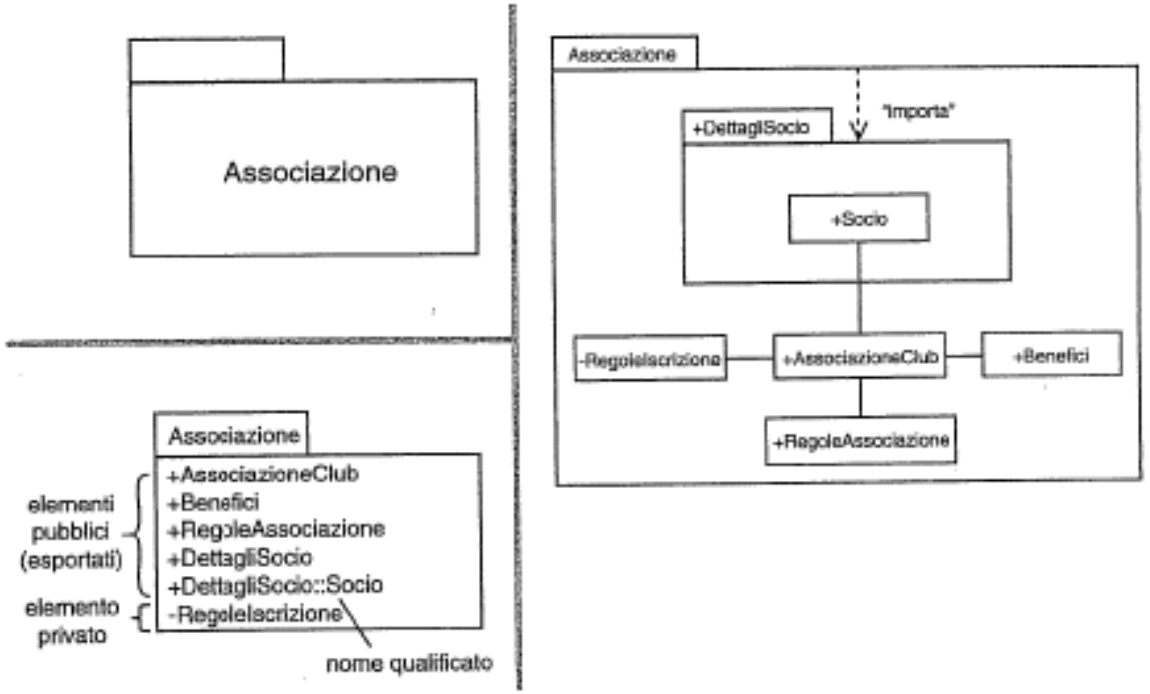
- Un package è l'entità di raggruppamento prevista da UML.
- Ogni package possiede un suo spazio dei nomi al cui interno i nomi degli elementi devono essere univoci.
- Un package è un meccanismo generalizzato per organizzare e raggruppare elementi e diagrammi.
- Un package può essere utilizzato per:
  - Fornire uno spazio dei nomi incapsulato al cui interno tutti i nomi devono essere univoci.
  - Raggruppare gli elementi semanticamente correlati.
  - Creare un "confine semantico" interno al modello.
  - Definire delle utilità per il lavoro in parallelo e la gestione della configurazione.
- I package prevedono il raggruppamento di elementi che hanno una forte correlazione semantica.
- Possono anche essere utili per creare confini semantici interni al modello.
- Un package è un meccanismo di raggruppamento logico che fornisce uno spazio dei nomi per i suoi membri.

Package di analisi – Che cos'è un package?

- Se si desidera raggruppare fisicamente gli elementi del modello è consigliabile utilizzare un componente.
- Ogni elemento del modello appartiene ad un unico package e la gerarchia di appartenenza forma un albero la cui radice è il package di livello più alto.
- UML mette a disposizione lo stereotipo <<`topLevel`>> che può essere utilizzato per identificare il package di livello più alto.
- I package di analisi possono contenere casi d'uso, casi d'analisi, realizzazioni di casi d'uso.

Package di analisi – Che cos'è un package?

- La sintassi UML per i package è abbastanza semplice ed è riportata nella seguente figura che mostra tre modi diversi di rappresentare lo stesso package:



- Gli elementi contenuti in un package possono avere una visibilità che indica se sono visibili o meno ai clienti del package.

Package di analisi – Che cos'è un package?

- Le visibilità previste da UML per gli elementi di un package sono riportate nella seguente tabella:

Simbolo	*Visibilità*	*Significato*
+	pubblica	Gli elementi con visibilità pubblica sono visibili agli elementi esterni al package; essi vengono esportati dal package
-	privata	Gli elementi con visibilità privata sono del tutto nascosti all'interno del package in cui risiedono

- Si può sfruttare la visibilità degli elementi per limitare il grado di interdipendenza tra package.
- Per dare a un package un'interfaccia piccola e semplice occorre minimizzare il numero di elementi pubblici presenti al suo interno.
- Questo potrebbe essere difficile da ottenere durante le attività di analisi a meno che non si espliciti la navigabilità delle associazioni; in caso contrario le associazioni tra classi sono tutte bidirezionali.

Package di analisi – Package e spazi dei nomi

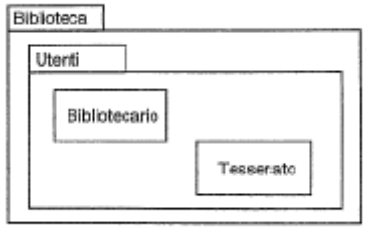
- Un package definisce ciò che viene chiamato uno spazio dei nomi incapsulato; in pratica questo vuol dire che esso crea un confine all'interno del quale tutti i nomi degli elementi devono essere univoci.

- Un elemento di uno spazio dei nomi, per referenziare un elemento di un altro spazio dei nomi, deve indicare, oltre al nome del destinatario, anche un cammino per arrivarci; tale cammino è detto pathname o nome qualificato dell'elemento.

- Si crea un nome qualificato premettendo al nome dell'elemento i nomi dei package in cui risiede, separati da una coppia di due punti (`::`).

- Prima si elenca il package più esterno e poi ogni package in ordine di annidamento finchè si arriva all'elemento più interno.

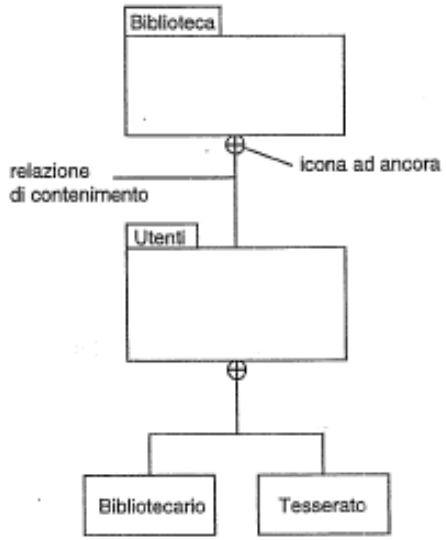
- Consideriamo la seguente figura:



- Il nome qualificato della classe `Bibliotecario` è `Biblioteca::Utenti::Bibliotecario`.

Package di analisi – Package annidati

- È possibile annidare i package all'interno di altri package raggiungendo un qualsivoglia livello di annidamento.
- Di solito sono sufficienti un livello o due di annidamento.
- UML consente di rappresentare l'annidamento mediante due distinte notazioni.
- La prima è quella mostrata nella figura precedente.
- La seconda è utile quando si hanno diversi livelli di annidamento, o annidamenti complessi. Essa viene mostrata nella seguente figura:

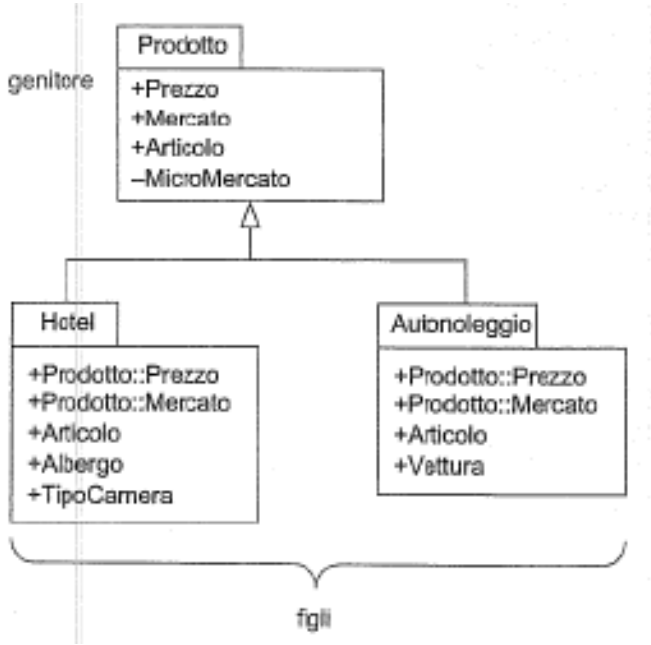


Package di analisi – Package annidati

- I package annidati hanno accesso allo spazio dei nomi dei package che li contengono.
- Nella figura precedente gli elementi nel package `Utenti` possono accedere a tutti gli elementi nel package `Biblioteca` usando nomi non qualificati.
- Tuttavia, l'inverso non è vero.
- Il package proprietario deve usare nomi qualificati per accedere al contenuto dei package che gli appartengono.
- Ad esempio, nella figura precedente, gli elementi in `Biblioteca` devono usare i nomi completamente qualificati `Utenti::Bibliotecario` e `Utenti::Tesserato` per accedere ai due elementi nel package `Utenti`.

Package di analisi – Generalizzazione tra package

- La generalizzazione tra package è simile a quella tra classi.
- I package figli sono più specializzati ed ereditano gli elementi pubblici e protetti del loro package genitore.
- I package figli possono aggiungere nuovi elementi e possono ridefinire elementi del genitore fornendone un'implementazione alternativa con lo stesso nome.
- Un esempio di generalizzazione tra package viene riportato nella seguente figura:



Package di analisi – Generalizzazione tra package

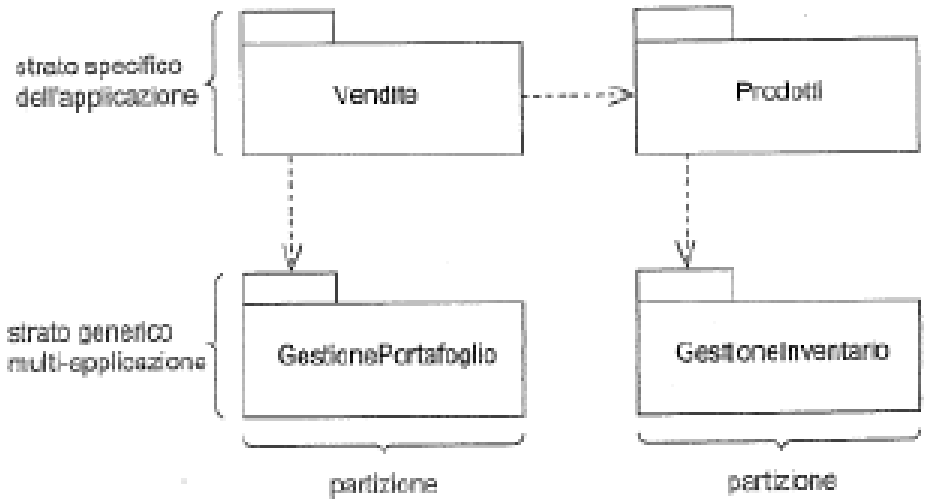
- I package `Hotel` e `Autonoleggio` ereditano tutti i membri protetti e pubblici del package `Prodotto`.
- Sia il package `Hotel` che il package `Autonoleggio` ridefiniscono la classe `Articolo` ereditata dal loro genitore fornendo una classe alternativa con lo stesso nome.
- Il package `Hotel` aggiunge le classi `Albergo` e `TipoCamera` mentre il package `Autonoleggio` aggiunge la classe `Vettura`.
- Il principio di sostituibilità vale anche per i package, proprio come per l'ereditarietà tra classi.

Package di analisi – Analisi dell'architettura

- Nell'analisi dell'architettura tutte le classi di analisi sono organizzate in un insieme coeso di package di analisi; questi sono ulteriormente organizzati in partizioni e strati.

- Ogni package di analisi in uno strato costituisce una partizione.

- Un esempio di organizzazione di package in strati viene riportata nella seguente figura:



- Uno degli obiettivi dell'analisi dell'architettura è minimizzare le interdipendenze presenti nel sistema.

Package di analisi – Analisi dell'architettura

- Esistono tre tecniche per ottenere questo risultato:
  - Minimizzare le dipendenze tra package di analisi.
  - Minimizzare il numero di elementi pubblici e protetti presenti in ciascun package di analisi.
  - Massimizzare il numero di elementi privati presenti in ciascun package di analisi.
- Ridurre le interdipendenze è una delle preoccupazioni principali dell'analisi dell'architettura.
- Il numero di strati individuati nel modello tende ad aumentare man mano che questo viene raffinato e arricchito di dettagli.
- Tuttavia, durante l'analisi ci si può limitare a organizzare i package in due soli strati: quello specifico dell'applicazione e quello generico multiapplicazione.
- Nella figura precedente `GestionePortafoglio` e `GestioneInventario` potrebbero essere riutilizzabili in diverse altre applicazioni, quindi questi package appartengono allo strato generico multiapplicazione.

Package di analisi – Analisi dell'architettura – Individuare i package di analisi

- I package di analisi possono essere individuati identificando i gruppi di elementi del modello accomunati da forti correlazioni semantiche.
- I package vengono spesso individuati gradualmente.
- È fondamentale che i package di analisi rispecchino raggruppamenti semantici reali piuttosto che una visione ideale dell'architettura logica.
- La fonte più utile di potenziali package è il modello statico; occorre cercare:
  - Aggregati coesi di classi nei diagrammi delle classi.
  - Gerarchie di ereditarietà.
- Anche il modello dei casi d'uso può essere preso in considerazione come fonte di potenziali package.
- Tuttavia capita spesso che i casi d'uso siano trasversali a diversi package di analisi.

Package di analisi – Analisi dell'architettura – Individuare i package di analisi

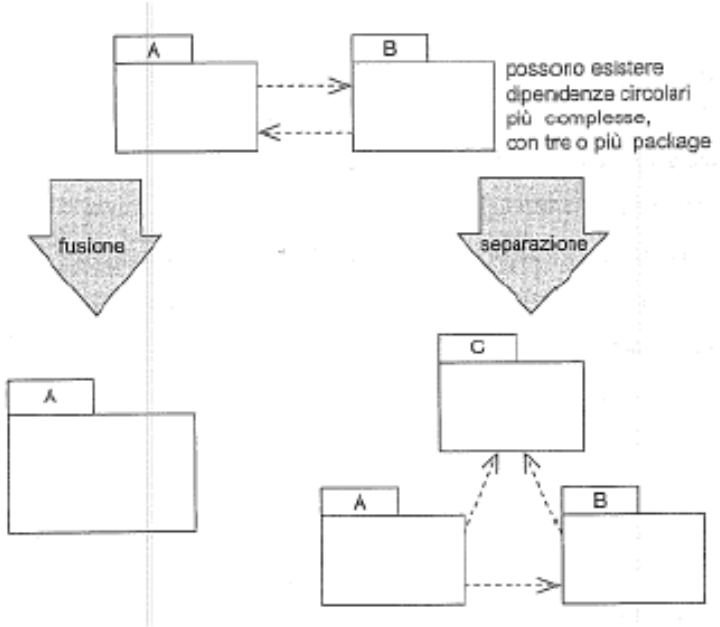
- Uno o più casi d'uso che supportino un dato processo aziendale o attore, o anche un insieme di casi d'uso collegati, potrebbero indicare un potenziale package.
- Dopo l'individuazione di un insieme di package occorre tentare di minimizzare i membri pubblici e protetti dei package e le dipendenze tra i package stessi.
- A tal fine si può procedere:
  - Spostando classi da un package ad un altro.
  - Aggiungendo package.
  - Eliminando package.
- I punti di forza di una buona struttura di package sono la massima coesione interna e la minima interdipendenza.
- Un package dovrebbe contenere un gruppo di classi strettamente correlate; le classi sono correlate nel modo più forte dall'ereditarietà, quindi dalla composizione, dall'aggregazione e, infine, dalle dipendenze.

Package di analisi – Analisi dell'architettura – Individuare i package di analisi

- Quando si crea il modello dei package di analisi si deve ricercare la semplicità.
- Un modo per mantenere il modello semplice consiste nell'evitare l'uso eccessivo di package annidati.
- Modelli con un numero eccessivo di livelli di annidamento assomigliano più ad una tipica decomposizione funzionale top-down che non ad un modello ad oggetti.
- Come regola pratica si può provare ad assegnare tra le quattro e le dieci classi di analisi per ogni package.
- Come per tutte le regole pratiche ci sono eccezioni.
- Ad esempio, a volte per spezzare una dipendenza circolare nel modello dei package, occorre introdurre un package contenente solo una o due classi.

Package di analisi – Analisi dell'architettura – Dipendenze circolari tra package

- Si deve tentare di evitare dipendenze circolari nel modello dei package di analisi.
- Se il package A dipende in qualche modo dal package B, e viceversa, forse è il caso di fonderli in un unico package.
- Un approccio migliore, che funziona molto spesso, è quello di tentare di estrarre gli elementi comuni e raggrupparli in un terzo package.
- Nella seguente figura vengono mostrate due tecniche per evitare dipendenze circolari:

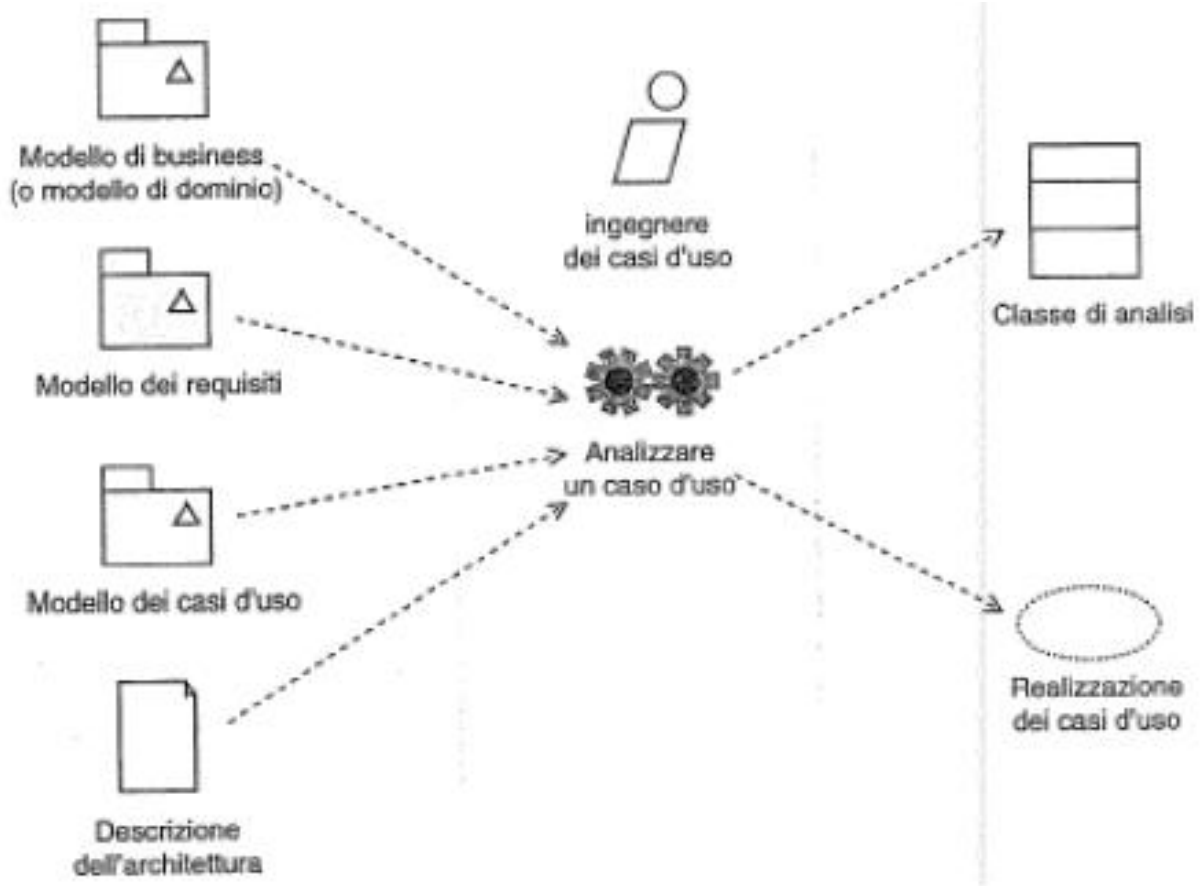


Package di analisi – Analisi dell'architettura – Dipendenze circolari tra package

- Molti strumenti di modellazione verificano automaticamente le dipendenze tra package; lo strumento crea una lista delle violazioni di accesso.
- In un modello di analisi potrebbe risultare impossibile creare un diagramma dei package privo di violazioni di accesso; questo perché nell'analisi si usano relazioni tra classi non raffinate (ovvero bidirezionali).
- Gli unici modi per eliminare questa violazione sono rifinire la relazione tra le due classi rendendola unidirezionale, oppure raggruppare le due classi in un unico package.
- Le classi che hanno dipendenze bidirezionali vere (ovvero, non dovute semplicemente all'incompletezza del modello) dovrebbero solitamente risiedere nello stesso package.

Realizzazione dei casi d'uso – Attività UP: analizzare un caso d'uso

- L'attività UP "analizzare un caso d'uso" produce un secondo manufatto, la realizzazione di caso d'uso, come illustrato nella seguente figura:

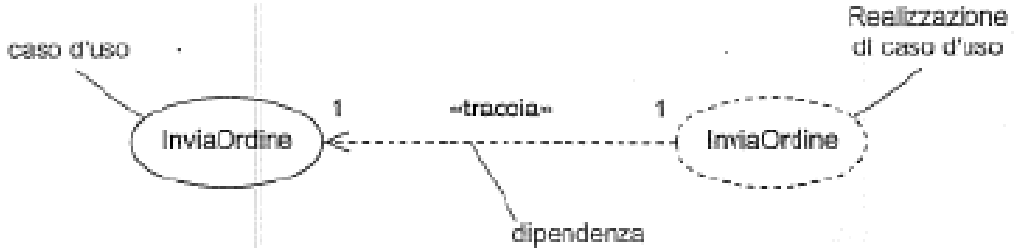


Realizzazione dei casi d'uso – Attività UP: analizzare un caso d'uso

- Le classi di analisi modellano la struttura statica del sistema; invece, le realizzazioni dei casi d'uso fanno parte della vista dinamica del sistema.
- Gli obiettivi per la realizzazione dei casi d'uso nell'analisi sono i seguenti:
  - Individuare quali classi di analisi interagiscono per realizzare il comportamento specificato da un caso d'uso.
  - Individuare quali istanze dei loro messaggi queste classi si devono inviare l'un l'altra per realizzare il comportamento specificato.
- Nell'analisi della realizzazione dei casi d'uso è essenziale fissare gli attributi, le operazioni e le relazioni fondamentali tra le classi di analisi.
- Non è necessario creare una realizzazione per ogni caso d'uso; basta scegliere i casi d'uso più importanti e lavorare su questi.
- Alla fine della realizzazione dei casi d'uso nell'analisi si avrà un modello di analisi che fornisce un'immagine di alto livello del comportamento dinamico del sistema.

Realizzazione dei casi d'uso – Cosa sono le realizzazioni dei casi d'uso

- Con il termine «realizzazioni dei casi d'uso» si intende un insieme di classi che realizzano il comportamento specificato da un caso d'uso.
- Esempio: verificare come interagiscono le classi `Libro`, `Scheda`, `Debitore` e l'attore `Bibliotecario` per realizzare il comportamento descritto in `PrestitoLibro`.
- In questo modo si trasforma un caso d'uso in un insieme di diagrammi delle classi e di diagrammi di interazione.
- Nella seguente figura viene mostrata la rappresentazione delle realizzazioni dei casi d'uso:



- Sebbene UML preveda un apposito simbolo per rappresentare le realizzazioni dei casi d'uso questi ultimi sono modellati in modo esplicito molto raramente.
- Di solito si aggiungono gli elementi appropriati in uno strumento di modellazione e si lascia che le realizzazioni dei casi d'uso siano un elemento implicito nell'archivio del modello.

Realizzazione dei casi d'uso – Cosa sono le realizzazioni dei casi d'uso

- Nella seguente tabella vengono riportati gli elementi tipici delle realizzazioni dei casi d'uso:

Elemento	*Scopo*
Diagrammi delle classi di analisi	Mostrare le classi di analisi che interagiscono per realizzare il caso d'uso.
Diagrammi di interazione	Mostrare le collaborazioni e le interazioni tra istanze specifiche che realizzano il caso d'uso; sono delle "istantanee" del sistema in esecuzione.
Requisiti speciali	Formulare i requisiti specifici del caso d'uso eventualmente emersi durante il processo di realizzazione dei casi d'uso.
Raffinamento del caso d'uso	Aggiornare il caso d'uso originario, qualora si ottengano nuove informazioni durante la sua realizzazione.

Realizzazione dei casi d'uso – Elementi

- Le realizzazioni dei casi d'uso sono costituite dagli elementi elencati nella tabella precedente.
- La realizzazione di un caso d'uso è un processo di raffinamento; partendo da una specifica del comportamento richiesto si arriva ad una descrizione piuttosto dettagliata delle interazioni tra classi e oggetti che consentiranno di realizzare tale comportamento.
- I diagrammi delle classi di analisi sono una parte fondamentale della realizzazione dei casi d'uso.
- Oltre ai diagrammi delle classi si possono creare diagrammi che esplicitano come oggetti specifici di queste classi collaborano e interagiscono tra di loro.
- Questi diagrammi sono noti come diagrammi di interazione; ne esistono di quattro tipi: diagramma di sequenza, diagramma di comunicazione, diagrammi di interazione generale e diagramma di temporizzazione.
- La modellazione OO è un processo iterativo; possono venire alla luce nuovi requisiti oppure si può rendere necessario rivedere casi d'uso esistenti.
- Se si rende necessario bisogna aggiornare i documenti dei requisiti ed, eventualmente, anche gli stessi casi d'uso.
- Qualora si renda necessario, bisogna aggiornare il modello dei casi d'uso, il modello dei requisiti e le classi di analisi per mantenerli tutti coerenti.

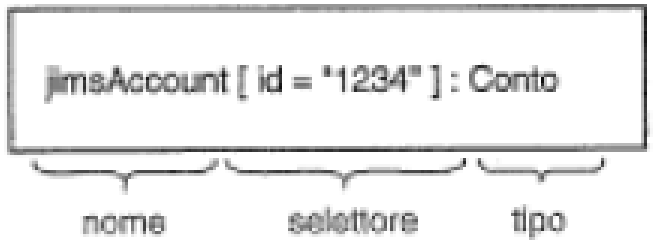
Realizzazione dei casi d'uso – Interazioni

- Le interazioni sono unità di comportamento di un classificatore che è noto come classificatore di contesto e che fornisce il contesto per l'interazione.
- Un'interazione può usare una delle funzioni del suo classificatore di contesto oppure una delle funzioni a cui il classificatore di contesto ha accesso.
- Nella realizzazione dei casi d'uso il classificatore di contesto è un caso d'uso e si creano una o più interazioni per dimostrare come il comportamento specificato dal caso d'uso può essere realizzato da istanze di classificatori che si scambiano messaggi.
- Quando si lavora su diagrammi di interazione si scoprono nuove operazioni e attributi delle classi di analisi.
- I diagrammi delle classi di analisi dovrebbero essere aggiornati con queste informazioni come parte del processo di realizzazione dei casi d'uso.
- Gli elementi fondamentali nei diagrammi di interazione sono le linee di vita e i messaggi; questi vengono analizzati in modo approfondito nelle prossime sezioni.

Realizzazione dei casi d'uso – Linee di vita

- Una linea di vita rappresenta un singolo partecipante a un'interazione, ovvero raffigura come un'istanza di uno specifico classificatore partecipa all'interazione.

- La sintassi della linea di vita viene riportata nella seguente figura:



- Ogni linea di vita ha un nome opzionale, un tipo e un selettore opzionale:

  - Il nome viene usato per far riferimento alla linea di vita all'interno dell'interazione.

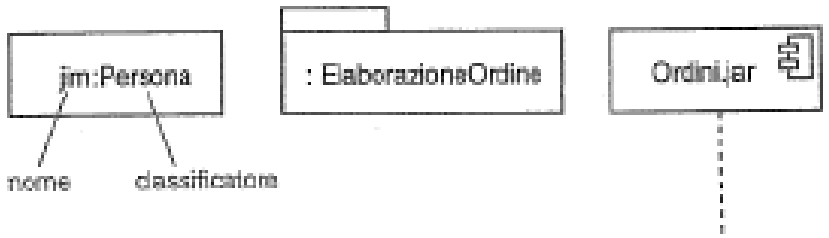
  - Il tipo rappresenta il nome del classificatore di cui la linea di vita rappresenta un'istanza.

Realizzazione dei casi d'uso – Linee di vita

- Il selettore indica una condizione booleana che può essere usata per selezionare una singola istanza che soddisfa la condizione.
  - Se non c'è alcun selettore la linea di vita fa riferimento a un'istanza arbitraria del classificatore.
  - I selettori sono validi solo se il tipo ha una molteplicità maggiore di uno in modo che ci siano più istanze tra cui scegliere.

- Le linee di vita sono disegnate con la stessa icona del loro tipo e quando vengono usati nei diagrammi di sequenza hanno una "coda" a forma di riga verticale "tratteggiata".

- Alcuni esempi di linea di vita vengono riportati nella seguente figura:



Realizzazione dei casi d'uso – Linee di vita

- Si pensi a una linea di vita come alla rappresentazione del modo in cui un'istanza del classificatore può partecipare all'interazione; tuttavia, essa non rappresenta una specifica istanza del classificatore.
- Si può, quindi, pensare alla linea di vita come alla rappresentazione di un ruolo che un'istanza del classificatore può assumere nell'interazione.
- Se si vuole, si possono mostrare le istanze direttamente su un diagramma di interazione.
- Questa distinzione tra linee di vita e istanze fa emergere due forme diverse di diagrammi di interazione.
- Un diagramma di interazione di forma generica mostra l'interazione tra le linee di vita che rappresentano istanze arbitrarie.
- Un diagramma di interazione di forma d'istanza mostra l'interazione tra specifiche istanze.
- Per completare l'interazione è necessario specificare i messaggi scambiati tra le linee di vita.

Realizzazione dei casi d'uso – Messaggi

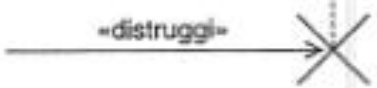
- Un messaggio rappresenta un tipo specifico di comunicazione tra due linee di vita in un'interazione.
- Questa comunicazione può implicare:
  - La chiamata di un'operazione.
  - La creazione o la distruzione di un'istanza.
  - L'invio di un segnale.
- Quando una linea di vita riceve un messaggio di chiamata vuol dire che viene invocata un'operazione che ha la stessa segnatura del messaggio.
- Quindi, per ogni messaggio di chiamata ricevuto da una linea di vita, ci deve essere un'operazione corrispondente nel classificatore di quella linea di vita.
- In realtà, UML consente che i messaggi nei diagrammi di interazione siano temporaneamente disallineati rispetto alle operazioni delle classi coinvolte.

Realizzazione dei casi d'uso – Messaggi

- Quando una linea di vita sta eseguendo un messaggio si dice che ha il focus di controllo o di attivazione.
- L'evolversi nel tempo dell'interazione provoca lo spostamento dell'attivazione tra le linee di vita; questo spostamento viene detto flusso di controllo.
- I messaggi vengono rappresentati come frecce tra linee di vita.

Realizzazione dei casi d'uso – Messaggi

- Esistono sette tipi di messaggio, come illustrato nella seguente figura:

Sintassi	Nome	Descrizione
	Messaggio sincrono	Il mittente aspetta che il destinatario ritorni dall'esecuzione del messaggio
	Messaggio asincrono	Il mittente invia il messaggio e continua l'esecuzione: *non* aspetta un ritorno dal destinatario
	Messaggio di ritorno	Il destinatario di un messaggio precedente restituisce il focus di controllo al mittente di quel messaggio
	Creazione dell'oggetto	Il mittente crea un'istanza del classificatore specificato dal destinatario
	Distruzione dell'oggetto	Il mittente distrugge il destinatario Se la sua linea di vita ha una coda, questa termina con una X
	Messaggio trovato	Il mittente del messaggio è al di fuori dell'ambito dell'interazione Lo si utilizza quando si vuole mostrare la ricezione di un messaggio, ma non si vuole mostrare da dove proviene
	Messaggio perso	Il messaggio non raggiunge mai la sua destinazione Può essere usato per indicare condizioni di errore in cui i messaggi vanno persi

Realizzazione dei casi d'uso – Messaggi – Messaggi sincroni, asincroni e di ritorno

- In una chiamata di un messaggio sincrono il mittente aspetta che il destinatario finisca l'esecuzione dell'operazione richiesta.
- In una chiamata di un messaggio asincrono il mittente non aspetta ma continua fino al passo successivo.
- Nei modelli di analisi, la distinzione tra messaggi sincroni e asincroni solitamente è troppo specifica.
- È consigliabile, però, visualizzare tutti i messaggi come sincroni perché questo è il caso più vincolante.
- Nella progettazione può essere importante fare una distinzione tra messaggi sincroni e asincroni per progettare flussi di controllo concorrente.
- È facoltativo mostrare i messaggi di ritorno nelle realizzazioni di caso d'uso a livello di analisi.
- Di solito i messaggi vengono mostrati solo se non affollano troppo il diagramma.

Realizzazione dei casi d'uso – Messaggi – Messaggi di creazione e distruzione

- Il messaggio di creazione degli oggetti è sempre disegnato come una linea continua con una punta di freccia aperta.
- Si può visualizzare la creazione degli oggetti semplicemente inviando un messaggio con lo stereotipo <<crea>> oppure mediante un messaggio specifico dotato di nome, anch'esso stereotipato con <<crea>>.
- In C++, C# o Java la creazione è un'operazione speciale, detta costruttore.
- Tuttavia, non tutti i linguaggi OO prevedono i costruttori; uno di questi è Smalltalk.
- La distruzione degli oggetti è rappresentata come una linea continua con una punta di freccia aperta stereotipata con <<distruggi>>.
- Se la linea di vita ha una coda questa deve terminare con una grossa croce nel punto di distruzione.
- Distruzione significa che l'istanza di classificatore a cui fa riferimento la linea di vita di destinazione non è più utilizzabile.
- I vari linguaggi hanno semantiche differenti per la distruzione degli oggetti.

Realizzazione dei casi d'uso – Messaggi – Messaggi di creazione e distruzione

- Per esempio, in C++, quando un oggetto viene eliminato viene automaticamente chiamato un metodo speciale chiamato distruttore.

- Nei linguaggi come Java e C# la distruzione degli oggetti è gestita dalla macchina virtuale tramite una strategia chiamata garbage collection.

  - Non si sa, però, quando si verificherà la distruzione.

  - Gli oggetti Java e C# possono avere un metodo "finale" che verrà eseguito dal garbage collector al momento della distruzione.

  - È rischioso però usare questo metodo perché non si sa quando verrà chiamato dal garbage collector.

Realizzazione dei casi d'uso – Messaggi – Messaggi trovati e persi

- Nell'analisi normalmente è possibile ignorare i messaggi trovati e persi.
- I messaggi trovati possono essere utili se si deve mostrare la ricezione di un messaggio da parte di una classe ma non si conosce l'origine del messaggio.
- I messaggi persi consentono di indicare che un messaggio è andato perso, cioè non raggiunge mai la sua destinazione.

Realizzazione dei casi d'uso – Diagrammi di interazione

- I diagrammi di interazioni di UML possono essere usati per modellare qualsiasi tipo di interazione tra istanze di classificatori.

- Nella realizzazione dei casi d'uso questi diagrammi vengono utilizzati per modellare interazioni tra oggetti che realizzano un caso d'uso o una parte di un caso d'uso.

- Ci sono quattro diversi tipi di diagrammi di interazione:

  - Diagrammi di sequenza: enfatizzano la sequenza temporale degli scambi di messaggi tra diverse linee di vita.

  - Diagrammi di comunicazione: enfatizzano le relazioni strutturali tra gli oggetti e sono utili per l'analisi, soprattutto per creare rapidamente una bozza di collaborazione tra gli oggetti.

  - Diagrammi di interazione generale: illustrano come il comportamento complesso viene realizzato da un insieme di interazioni più semplici; sono un caso speciale di diagramma di attività in cui i nodi si riferiscono ad altre interazioni.

  - Diagrammi di temporizzazione: enfatizzano gli aspetti in tempo reale di un'interazione; il loro scopo principale è aiutare l'utente a ragionare sul tempo.

Realizzazione dei casi d'uso – Diagrammi di sequenza

- I diagrammi di sequenza mostrano le interazioni tra linee di vita come una sequenza di eventi ordinati temporalmente.

- Nella modellizzazione, talvolta si inizia abbozzando una realizzazione dei casi d'uso mediante un diagramma di comunicazione.

- Tuttavia, quando occorre focalizzarsi sull'effettiva sequenza temporale degli eventi, è sempre molto più facile usare un diagramma di sequenza.

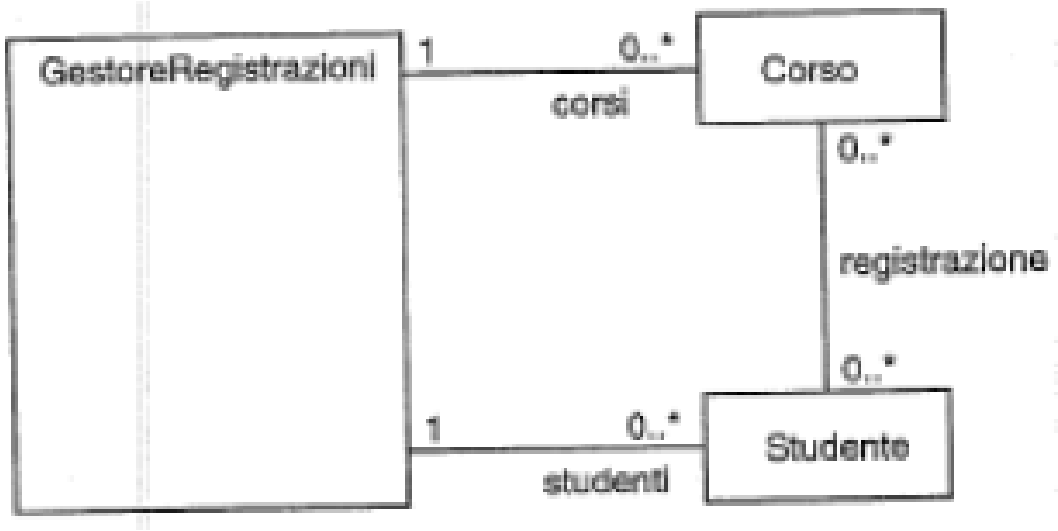
Realizzazione dei casi d'uso – Diagrammi di sequenza – Linee di vita e messaggi

- Consideriamo la realizzazione del caso d'uso `AggiungiCorso`, mostrato nella seguente figura:

Caso d'uso: AggiungiCorso
ID:8
Breve descrizione: Aggiunta di dettagli di un nuovo corso al sistema
Attori primari: Registrante
Attori secondari: Nessuno
Precondizioni: 1. Il Registrante si è collegato al sistema
Sequenza degli eventi principale: 1. Il Registrante seleziona "aggiungi corso" 2. I Registrante inserisce il nome del nuovo corso 3. Il sistema crea il nuovo corso
Postcondizioni: 1. Un nuovo corso è stato aggiunto al sistema
Sequenza degli eventi alternativa: CorsoEsiste

Realizzazione dei casi d'uso – Diagrammi di sequenza – Linee di vita e messaggi

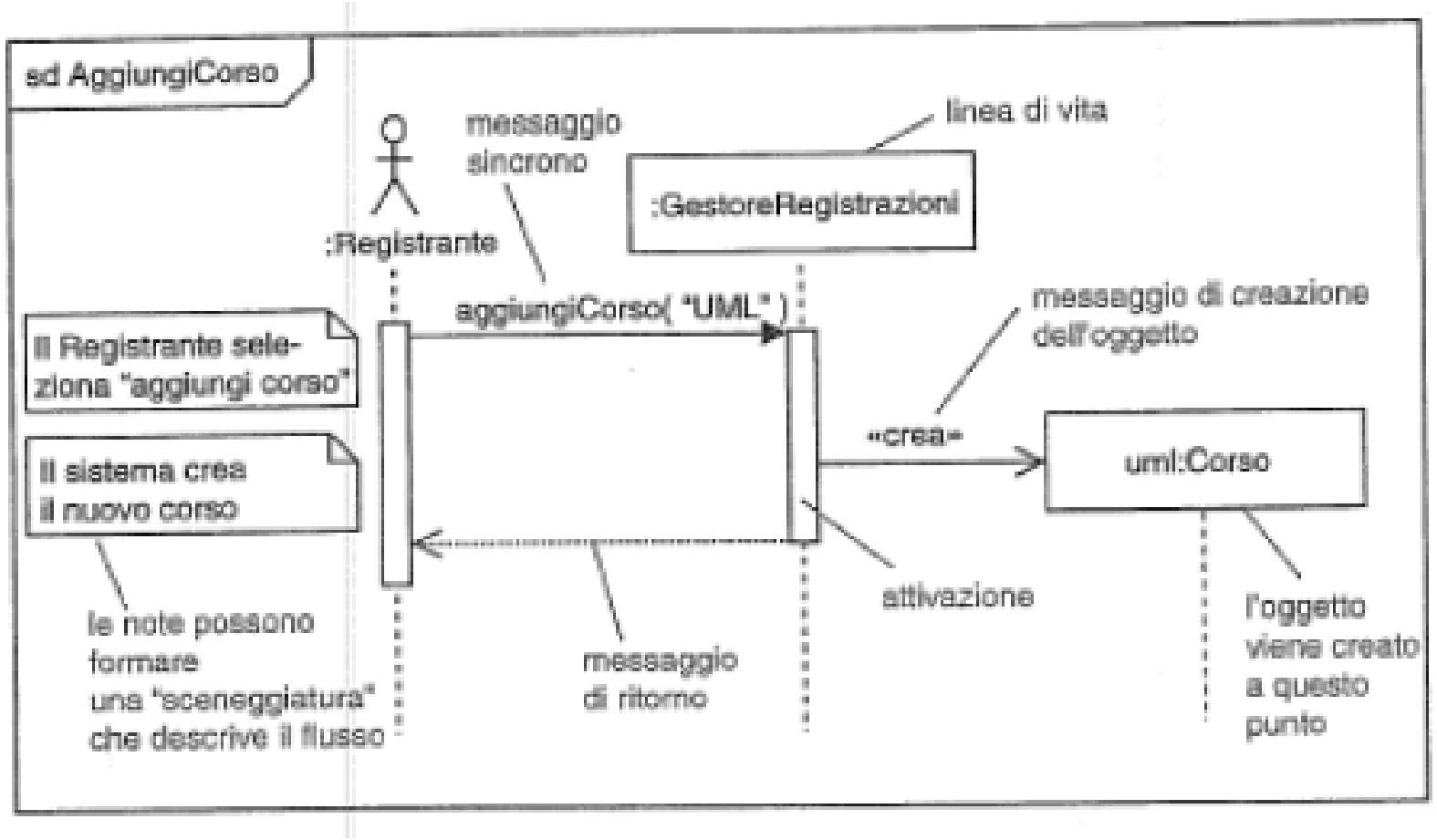
- L'analisi iniziale del caso d'uso ha realizzato il diagramma delle classi di analisi ad alto livello mostrato nella figura seguente:



- Dati la specifica del caso d'uso e il diagramma delle classi si hanno informazioni sufficienti per creare un diagramma di sequenza.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Linee di vita e messaggi

- Un diagramma di sequenza che realizza il comportamento specificato dal caso d'uso `AggiungiCorso` viene riportato nella seguente figura:



- Secondo la specifica di UML 2 i nomi dei diagrammi di interazione possono essere preceduti da `sd`.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Linee di vita e messaggi

- Quando si inizia a costruire diagrammi di sequenza come parte della realizzazione di un caso d'uso si può scoprire che bisogna modificare il diagramma delle classi di analisi o anche il caso d'uso.
- Nei diagrammi l'asse verticale viene utilizzato per riportare il tempo che scorre.
- L'asse orizzontale serve per riportare le linee di vita da sinistra a destra.
- Al di sotto di ogni linea di vita si traccia una linea tratteggiata che rappresenta l'esistenza della linea di vita nel tempo.
- La figura non è una rappresentazione esatta di ogni passo del caso d'uso.
- Il diagramma si può raffinare durante la fase di progettazione.

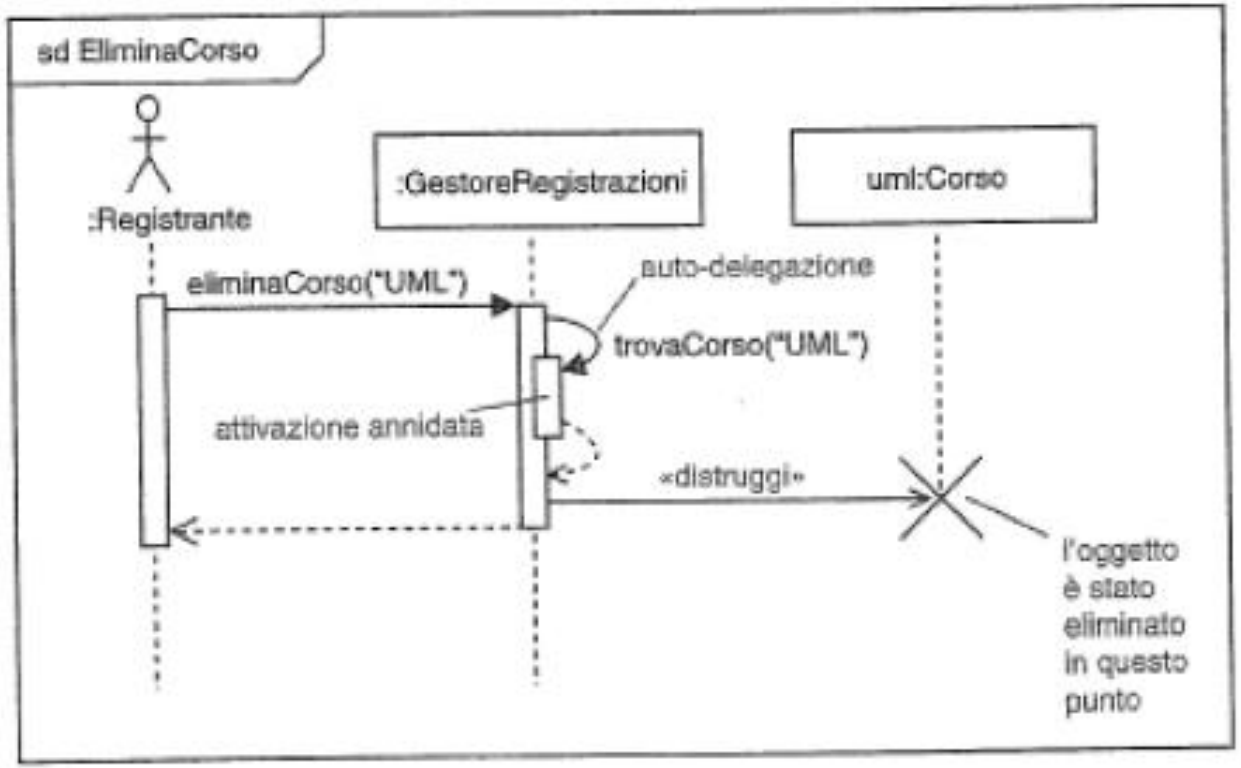
Realizzazione dei casi d'uso – Diagrammi di sequenza – Linee di vita e messaggi

- Si consideri il seguente caso d'uso `EliminaCorso`:

Caso d'uso:EliminaCorso
ID: 6
Breve descrizione Eliminazione di un corso dal sistema
Attori primari Registrante
Attori secondari Nessuno
Precondizioni 1. Il Registrante si è collegato al autenticato dal sistema
Sequenza degli eventi principale 1. Il Registrante seleziona "elimina corso" 2. Il Registrante inserisce il nome del corso 3. Il sistema elimina il corso
Postcondizioni 1. Un corso è stato eliminato dal sistema
Sequenza degli eventi alternativa CorsoNonEsiste

Realizzazione dei casi d'uso – Diagrammi di sequenza – Linee di vita e messaggi

- Il diagramma di sequenza per il caso d'uso `EliminaCorso` è il seguente:



- Per indicare la distruzione di un oggetto si termina la linea di vita con una grossa croce.

- Se non si sa, o non interessa sapere, quando un oggetto viene distrutto basta terminare la sua linea di vita normalmente, senza usare la croce.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Linee di vita e messaggi

- La figura precedente mostra anche l'auto-delega, cioè una linea di vita invia un messaggio a se stessa.
- L'auto-delega è molto comune nei sistemi OO; si utilizzano, a tal fine, i metodi privati.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Attivazioni

- Le attivazioni, o focus di controllo, consistono in rettangoli lunghi e stretti per indicare che una particolare linea di vita ha il focus di controllo.
- In UML 1 venivano chiamati anche "specifiche dell'esecuzione".
- Un focus di controllo può essere annidato all'interno di un altro focus di controllo.
- Le attivazioni non sembrano particolarmente apprezzate e molti modellatori non si preoccupano proprio di mostrarle.
- In un sistema OO in esecuzione le attivazioni vengono gestire implicitamente tramite la normale semantica di invocazione delle operazioni.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Documentare i diagrammi di sequenza

- È possibile aggiungere una "sceneggiatura" a un diagramma tramite note posto lungo il lato sinistro.
- Questo rende il diagramma molto più accessibile.
- La sceneggiatura può comprendere i passi della sequenza degli eventi al caso d'uso o può anche essere solo una descrizione testuale di ciò che sta succedendo nel diagramma.
- In entrambi i casi una sceneggiatura può essere un'aggiunta utile a un diagramma di sequenza.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Vincoli e invarianti di stato

- Quando un'istanza riceve un messaggio può provocare un cambiamento di stato.
- Uno stato è definito come:

> Una condizione o situazione durante la vita di un oggetto in cui esso soddisfa una condizione, esegue un'attività o aspetta un evento

- Ogni classificatore può avere una macchina a stati che descrive il ciclo di vita delle sue istanze in termine di stati ed eventi.
- Non tutti i messaggi comportano un cambiamento di stato.
- Aggiungere gli invarianti di stato a un diagramma di sequenza può essere una tecnica di analisi molto utile perché consente di rilevare gli stati fondamentali del ciclo di vita di una linea di vita.

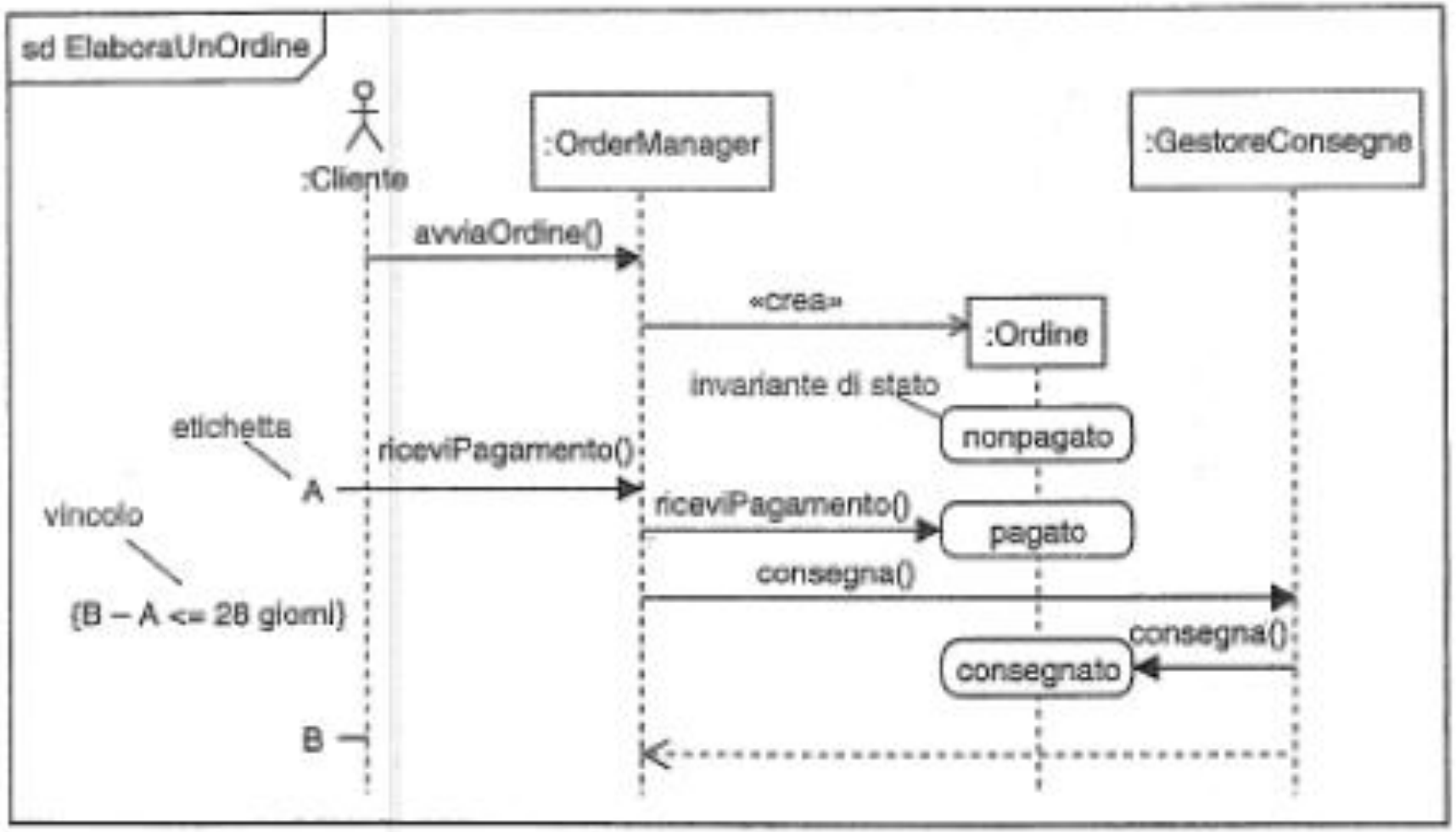
Realizzazione dei casi d'uso – Diagrammi di sequenza – Vincoli e invarianti di stato

- La seguente figura mostra un caso d'uso preso da un sistema di elaborazione ordini soggetto ai seguenti vincoli:
  - Il pagamento dell'ordine deve avvenire a saldo, tramite un unico pagamento.
  - I prodotti specificati nell'ordine possono essere consegnati solo dopo il pagamento.
  - I prodotti vengono consegnati al cliente entro 28 giorni dal pagamento.

Caso d'uso:ElaboraUnOrdine
ID:5
Breve descrizione Il Cliente avvia un ordine che viene poi pagato e consegnato
Attori primari Cliente
Attori secondari Nessuno.
Precondizioni Nessuna.
Sequenza degli eventi principale 1. Il caso d'uso inizia quando l'attore Cliente crea un nuovo ordine. 2. Il Cliente paga l'ordine a saldo. 3. I prodotti vengono consegnati al Cliente entro 28 giorni dalla data del pagamento finale.
Postcondizioni 1. L'ordine è stato pagato. 2. I prodotti sono stati consegnati entro 28 giorni dal pagamento finale
Sequenza degli eventi alternativa PagamentoInEccesso OrdineAnnullato ProdottiNonConsegnati ProdottiConsegnatiInRitardo PagamentoParziale

Realizzazione dei casi d'uso – Diagrammi di sequenza – Vincoli e invarianti di stato

- La seguente figura mostra un diagramma di sequenza per `ElaboraUnOrdine`:



Realizzazione dei casi d'uso – Diagrammi di sequenza – Vincoli e invarianti di stato

- Quando viene creata un'istanza `Ordine` passa immediatamente allo stato `nonpagato` (disegnato come un rettangolo con gli angoli arrotondati).

- Questo dimostra che tutti gli `Ordini` sono creati nello stato `nonpagato`.

- Alla ricezione del messaggio `riceviPagamento()`, che deve essere inviato per il pagamento a saldo, l'istanza `Ordine` passa allo stato `pagato`.

- Successivamente il messaggio `consegna()` viene inviato all'istanza `GestoreConsegne` che lo inoltra all'istanza `Ordine`, provocando la sua transizione allo stato `consegnato`.

- Se la classe `Ordine` ha anche una macchina a stati, gli stati in quella macchina devono corrispondere a tutti gli invarianti di stato che si trovano sui diagrammi di sequenza.

- Un vincolo posto su una linea di vita indica una condizione sulle istanze che deve essere vera da quel punto in avanti.

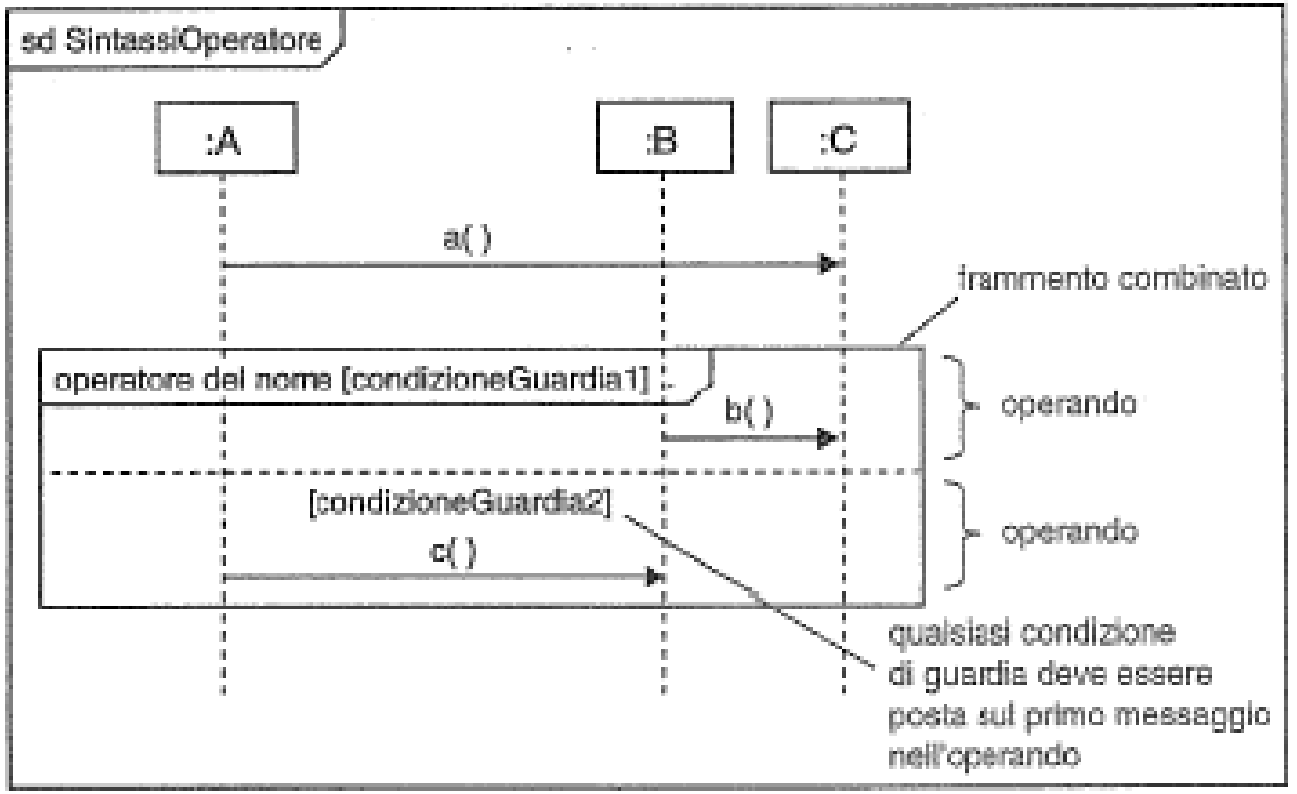
- I vincoli vengono spesso indicati usando un linguaggio informale, sebbene UML abbia un linguaggio formale, denominato OCL, per la loro specifica.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Vincoli e invarianti di stato

- Nella figura viene illustrato un vincolo di durata: `{B - A <= 28 giorni}`.
- Su una linea di vita può essere posto qualsiasi tipo di vincolo.
- Nella figura precedente non sono usate né attivazioni né ritorni di messaggi; questo perché nel diagramma si evidenziano già gli invarianti di stato e la temporizzazione.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame

- I diagrammi di sequenza possono essere divisi in aree chiamate frame.
- La seguente figura illustra la sintassi dei frame che è piuttosto ricca:



Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame

- Ogni frame ha un operatore, uno o più operandi e zero o più condizioni di guardia.
  - L'operatore determina come vengono eseguiti i suoi operandi.
  - Le condizioni di guardia stabiliscono se i loro operandi devono essere eseguiti; esse sono espressioni booleane.
  - A tutti gli operandi si può applicare una sola condizione di guardia oppure ciascun operando può avere la propria condizione di guardia univoca.

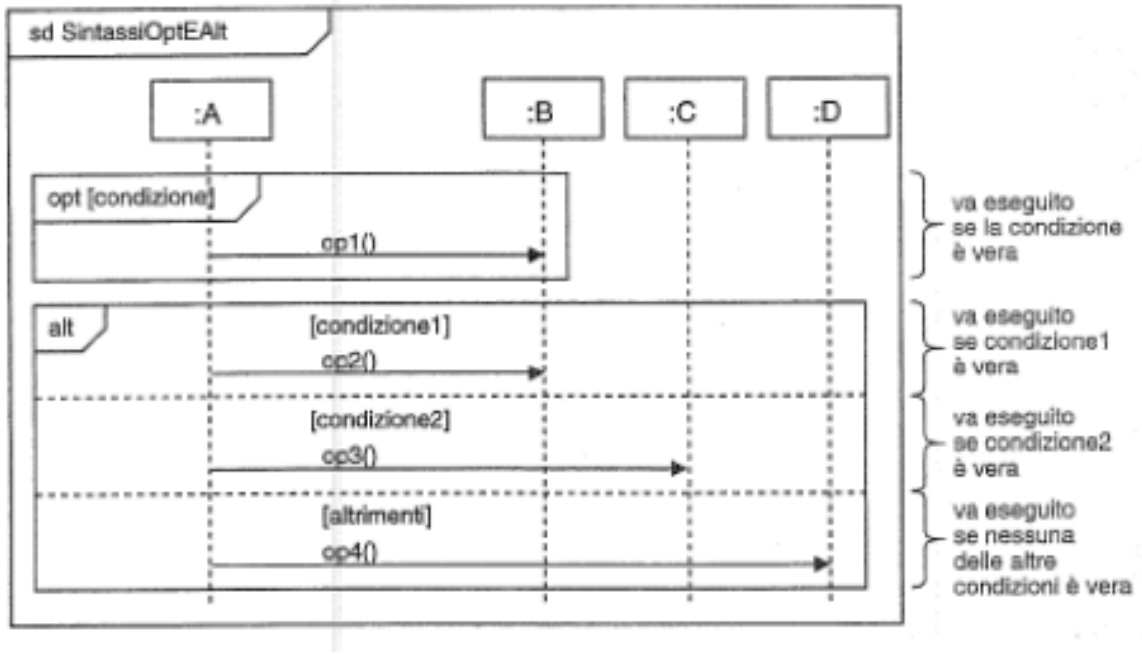
Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame

- La seguente tabella mostra l'elenco di tutti gli operatori:

Operatore	*Nome completo*	*Significato*
`opt`	option	C'è un singolo operando che viene eseguito se la condizione è vera (come il costrutto di programmazione if ... then)
`alt`	alternatives	Viene eseguito l'operando la cui condizione è vera. Si può usare la parola chiave `else` invece di un'espressione booleana (come il costrutto di programmazione select ... case)
`loop`	loop	Ha una sintassi speciale: `loop min, max [condizione]` la cui semantica è esegui `loop min` volte, poi mentre la condizione è vera, esegui `loop` altre `(max-min)` volte
`break`	break	Se la condizione di guardia è vera, viene eseguito l'operando, non il resto dell'interazione in cui è incluso
`ref`	reference	Il frammento combinato fa riferimento a un'altra interazione
`par`	parallel	Tutti gli operandi vengono eseguiti in parallelo
`critical`	critical	L'operando viene eseguito atomicamente senza interruzione
`seq`	weak sequencing	Tutti gli operandi vengono eseguiti in parallelo con il vincolo seguente: gli eventi che arrivano sulla stessa linea di vita da operandi diversi accadono nella stessa sequenza in cui si verificano gli operandi. Questo equivale a una forma debole di sequenza.
`seq`	strict sequencing	Gli operandi vengono eseguiti in sequenza stretta
`neg`	negative	L'operando mostra interazioni non valide. Questo operatore va usato quando si desidera mostrare le interazioni che non devono accadere
`ignore`	ignore	Elenca i messaggi che sono stati intenzionalmente omessi dall'interazione: i nomi dei messaggi ignorati vengono messi tra parentesi graffe in un elenco delimitato da virgole poste dopo il nome dell'operatore; per esempio, `{m1, m2, m3 }`. Per esempio, si usa se l'interazione rappresenta un caso di test in cui si sceglie di ignorare alcuni messaggi
`consider`	consider	Elenca i messaggi che sono intenzionalmente inclusi nell'interazione: i nomi dei messaggi vengono messi tra parentesi graffe in un elenco delimitato da virgole poste prima del nome dell'operatore. Per esempio, si usa se l'interazione rappresenta un caso di test in cui si sceglie di includere solo un sottoinsieme dell'insieme di messaggi possibili
`assert`	assertion	L'operando è l'unico comportamento valido a quel punto dell'interazione; qualsiasi altro comportamento sarebbe un errore. Questo operatore va usato come un modo per indicare che un comportamento deve avvenire a un certo punto dell'interazione

Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Ramificazione con opt e alt

- La seguente figura illustra la sintassi di `opt` e `alt`:



- L'operatore `opt` indica che il suo unico operando viene eseguito se e solo se la condizione di guardia è vera.

- `opt` equivale al costrutto di programmazione:

```
if (condizione1) then
   azione1
```

Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Ramificazione con opt e alt

- L'operatore `alt` rappresenta una scelta tra più alternative.
- Ciascuno dei suoi operandi ha la propria condizione di guardia.
- L'operando opzionale con una condizione di guardia di [`else`] viene eseguito se nessuna delle altre condizioni di guardia è vera.
- Può essere eseguito solo uno degli operandi `alt`.
- `alt` equivale al costrutto di programmazione:

```
if (condizione1) then
    operando 1
else if (condizione2) then
    operando 2
...
else if (condizioneN) then
    operando N
else
    operando M
```

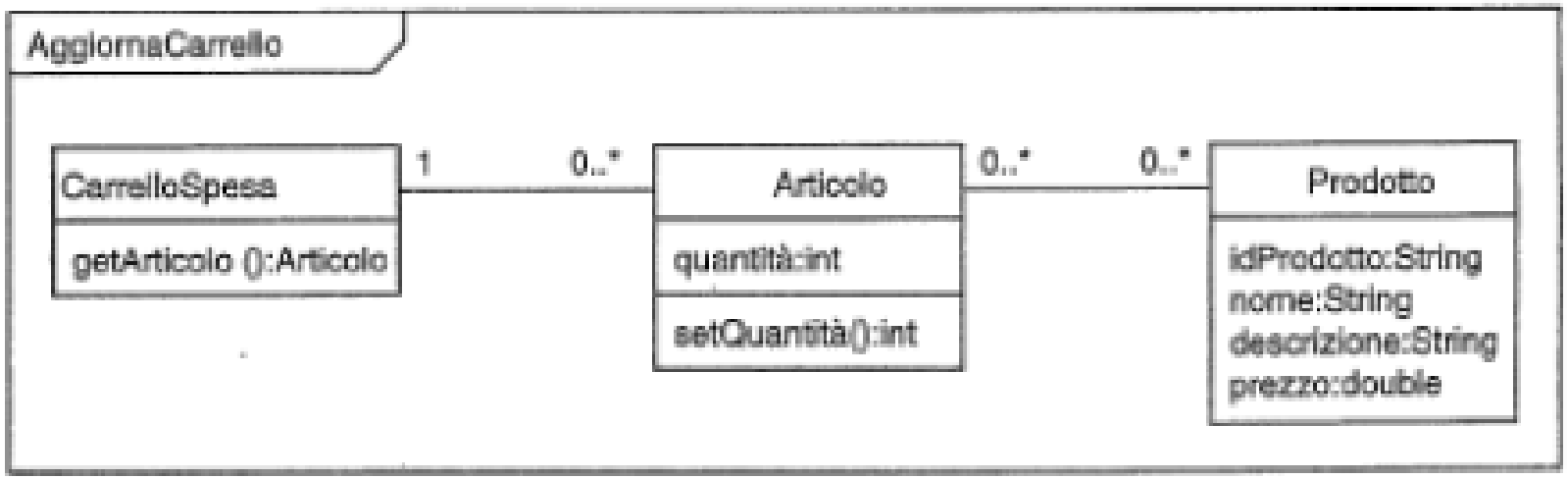
Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Ramificazione con opt e alt

- `opt` è semanticamente equivalente a un operatore `alt` con esattamente un operando.
- Come esempio di utilizzo di `opt` e `alt` si consideri il seguente caso d'uso `AggiornaCarrello`:

Caso d'uso:AggiornaCarrello
ID: 2
Breve descrizione Il Cliente cambia la quantità di un articolo nel carrello
Attori primari Cliente
Attori secondari Nessuno
Precondizioni 1. Il contenuto del carrello della spesa è visibile
Sequenza degli eventi principale 1. Il caso d'uso inizia quando il Cliente seleziona un articolo nel carrello 2. Se il Cliente seleziona "rimuovi articolo" 2.1 Il sistema elimina l'articolo dal carrello 3. Se il Cliente digita una nuova quantità 3.1 Il sistema aggiorna la quantità dell'articolo presente nel carrello
Postcondizioni Nessuna
Sequenza degli eventi alternativa Nessuna

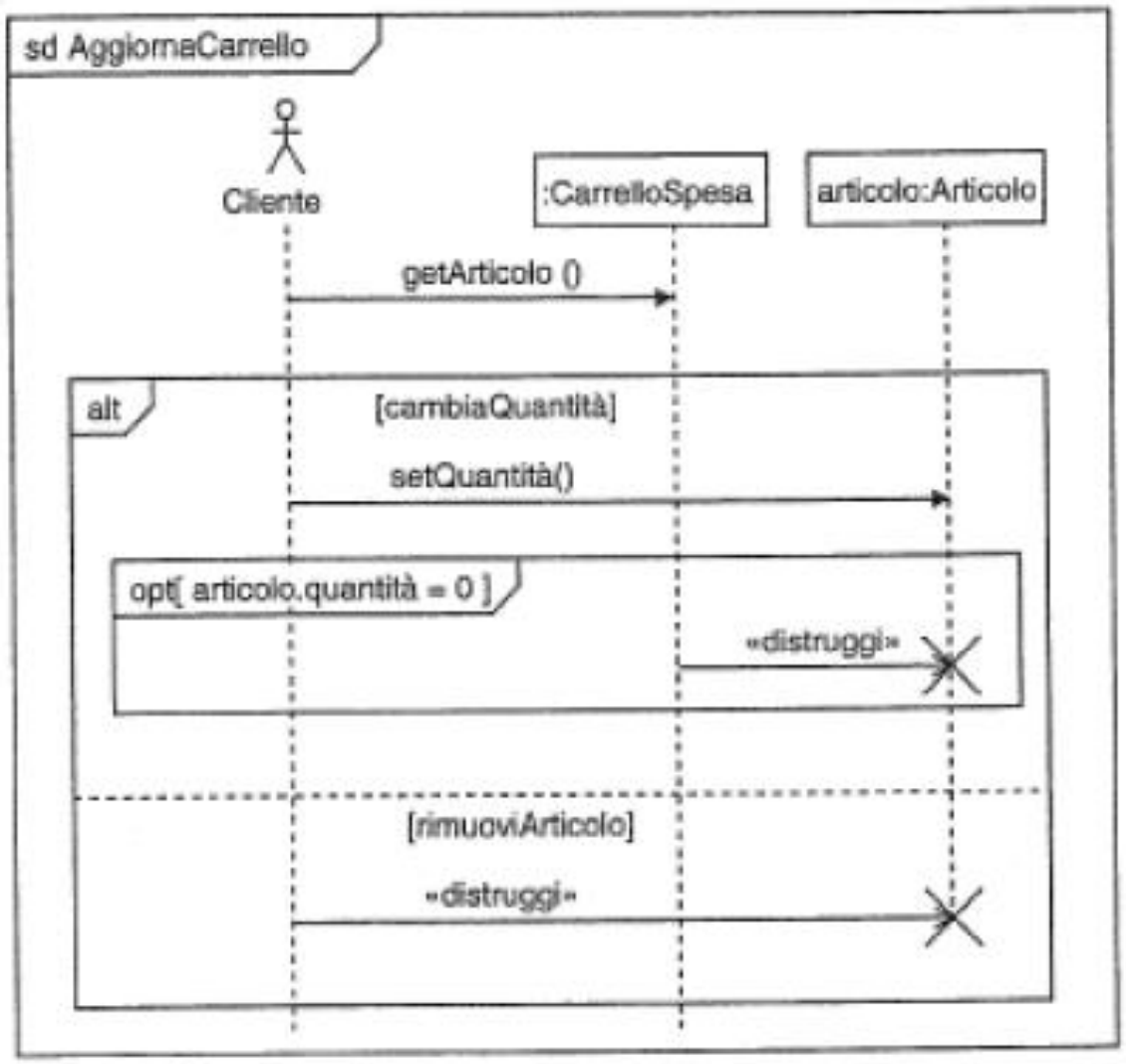
Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Ramificazione con opt e alt

- La seguente figura mostra il diagramma delle classi di analisi per questo caso d'uso:



Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Ramificazione con opt e alt

- La seguente figura mostra il diagramma di sequenza che realizza questo caso d'uso:



Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Ramificazione con opt e alt

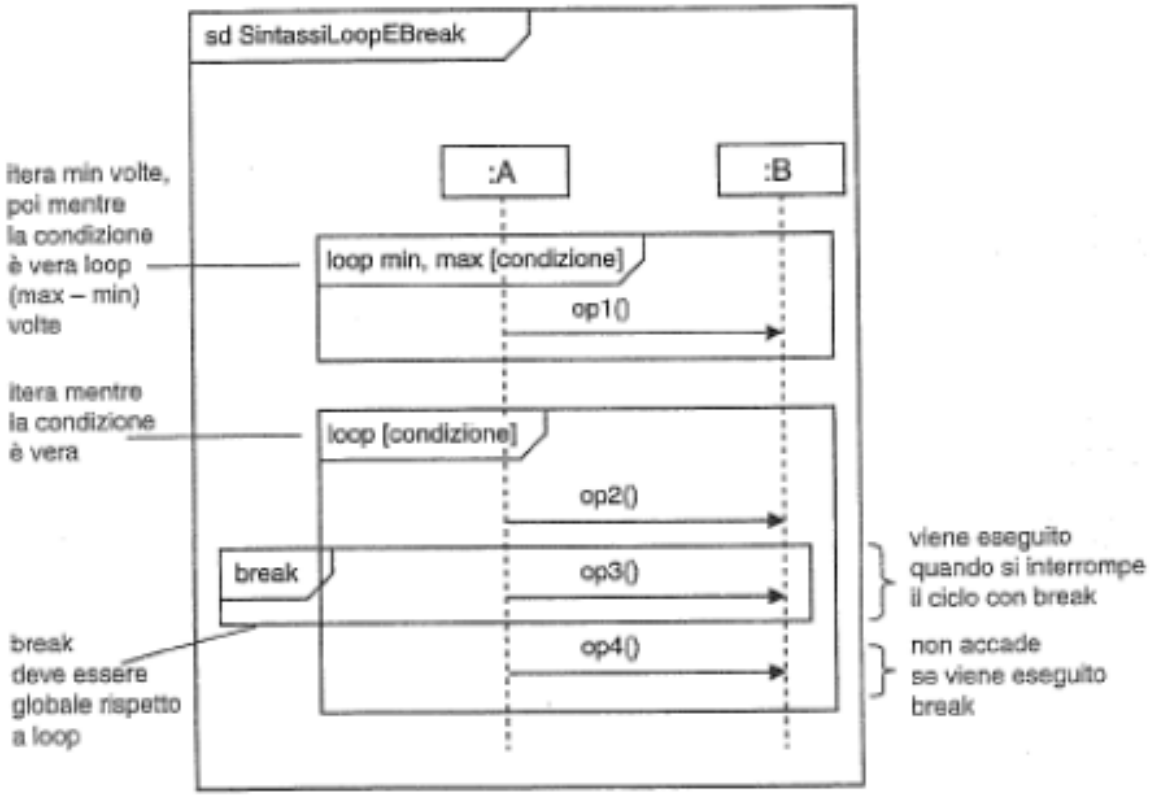
- Si osservi che in questo diagramma di sequenza viene eseguita la politica per cui quando la quantità di un articolo arriva a zero, l'articolo viene eliminato.

- Tuttavia si può ritenere che questo livello di dettaglio non sia necessario nell'analisi e che un diagramma di sequenza che non mostri l'eliminazione esplicita degli articoli sia ugualmente accettabile.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Iterazione con loop e break

- Molto spesso nei diagrammi di sequenza è necessario mostrare i cicli; per questo basta usare gli operatori `loop` e `break`.

- La sintassi di `loop` e `break` viene mostrata nella seguente figura:



Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Iterazione con loop e break

- L'operatore `loop` funziona nel modo seguente:

  `loop min` volte, poi

  `while` (condizione è vera)

  `loop (max-min)` volte

- Si notino i seguenti punti della sintassi di `loop`:
  - Un ciclo (`loop`) senza `max`, `min` o una condizione è un ciclo infinito.
  - Se viene specificato solo `min`, allora si considera `max = min`.
  - La condizione normalmente è un'espressione booleana ma può anche essere un testo purchè il suo significato sia chiaro.
- A prima vista `loop` potrebbe sembrare piuttosto complesso, ma può essere usato per supportare molti idiomi relativi ai cicli.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Iterazione con loop e break

- Nella seguente tabella vengono elencati alcuni degli idiomi più comuni:

Tipo di ciclo	*Significato*	*Espressioni del ciclo*
`while (true)` `{body}`	Continuare il ciclo per sempre	`loop` o `loop *`
`for i = n to m` `{body}`	Ripetere (`m-n`) volte	`loop n,m`
`while (espressioneBooleana)` `{body}`	Ripetere mentre `espressioneBooleana` è vera	`loop [espressioneBooleana]`

`repeat`
`{body}`
`while (espressioneBooleana)`
	Eseguire una volta, poi ripetere mentre `espressioneBooleana` è vera	`loop 1, *[espressioneBooleana]`

`forEach oggetto della collezione` `{body}`	Eseguire il corpo del ciclo una volta per ogni oggetto di una collezione di oggetti	`loop [per ogni oggetto di collezioneDiOggetti]`
`forEach oggetto della classe` `{body}`	Eseguire il corpo del ciclo una volta per ogni oggetto di una particolare classe	`loop [per ogni oggetto in NomeClasse]`

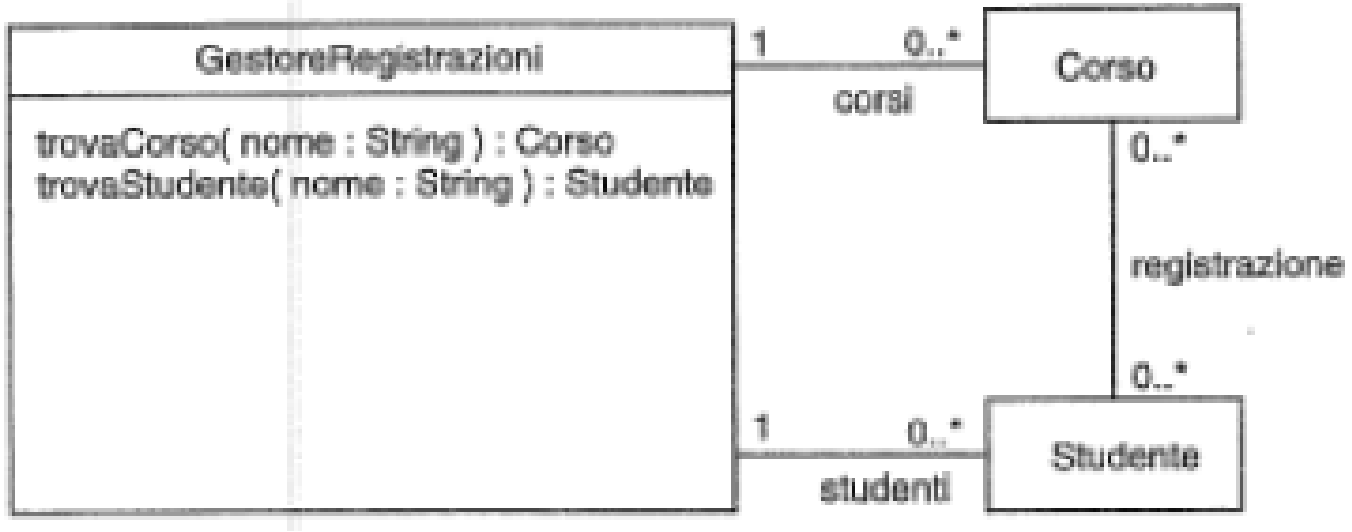
- È possibile usare `break` per indicare in quale condizione il `loop` viene interrotto e che cosa accade di seguito.
- L'operatore `break` ha una sola condizione di guardia e se questa è vera viene eseguito il corpo di `break` e il `loop` termina.
- Il punto fondamentale è che il resto del ciclo dopo l'interruzione (`break`) non viene eseguito.

Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Iterazione con loop e break

- Il frammento combinato `break` è logicamente correlato esterno al ciclo e non fa parte di esso; si deve disegnare il frammento `break` sempre all'esterno del `loop` ma sovrapponendolo.

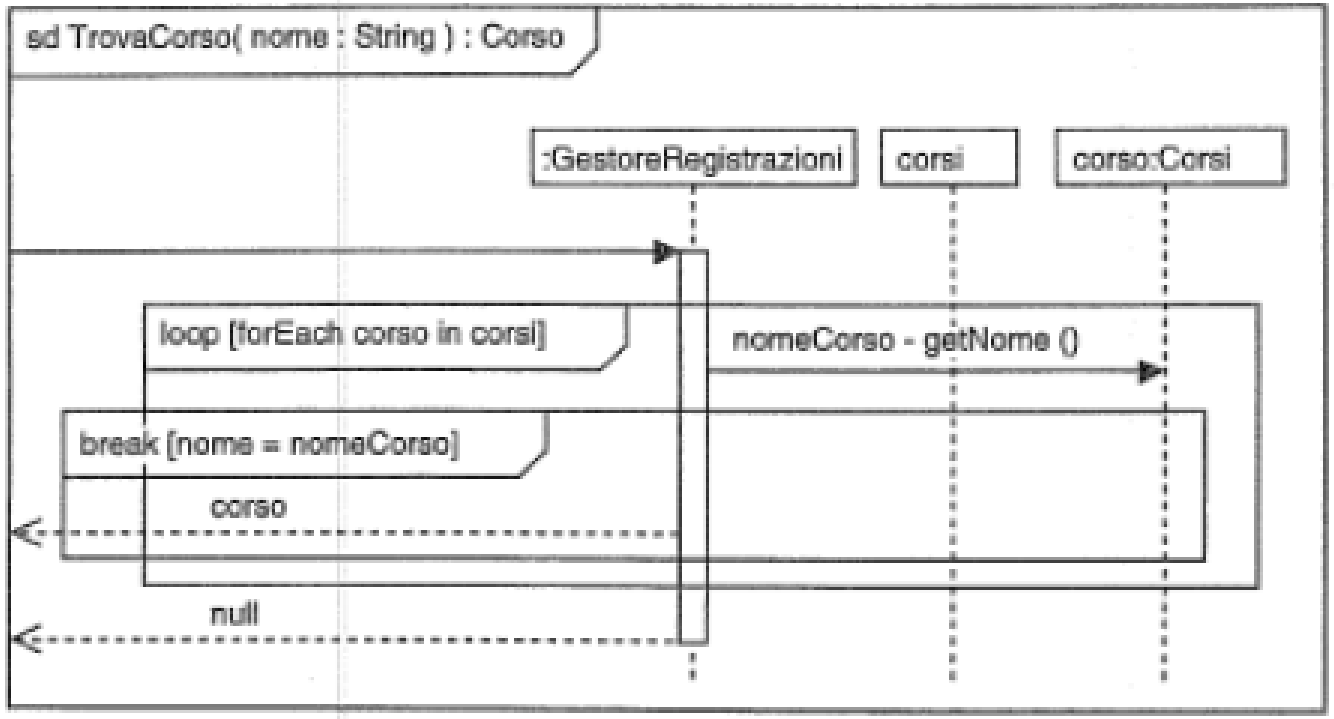
- Uno degli idiomi relativi ai cicli più comuni serve per scandire un insieme di oggetti.

- Per esempio si potrebbe usare un `loop` su un insieme come una possibile implementazione per l'operazione `trovaCorso`(…) della classe `GestoreRegistrazione`:



Realizzazione dei casi d'uso – Diagrammi di sequenza – Operatori e frame – Iterazione con loop e break

- L'operazione restituisce l'oggetto `Corso` con il nome esatto oppure `null`, come illustrato nella figura seguente:

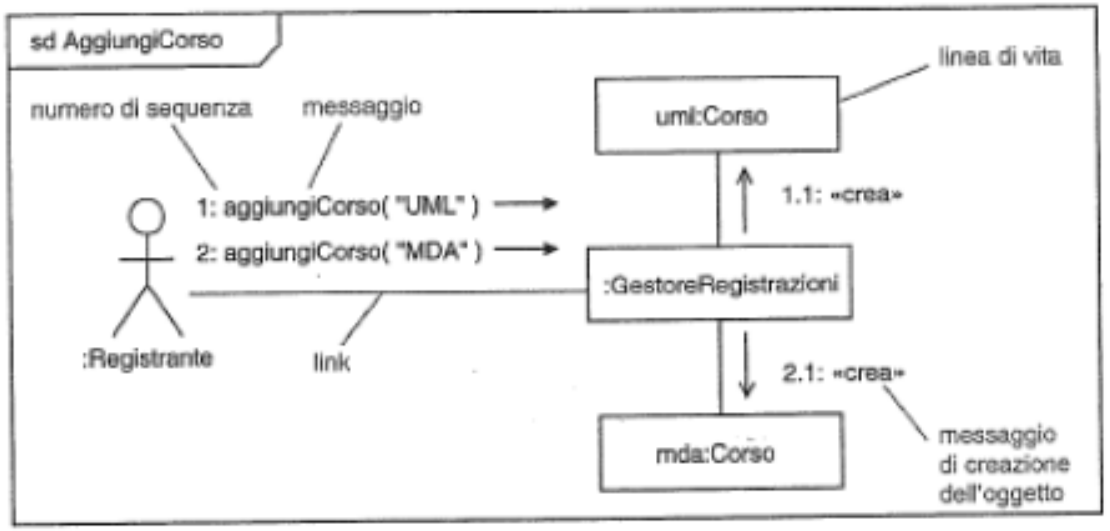


- Si può usare il nome dell'estremità di un'associazione con molteplicità maggiore di 1 come una collezione di oggetti.

- In questo caso si usa `corsi` per rappresentare una collezione di oggetti `Corso`.

Realizzazione dei casi d'uso – Diagrammi di comunicazione

- I diagrammi di comunicazione enfatizzano gli aspetti strutturali di un'interazione mostrando come si collegano le linee di vita.
- Questi diagrammi sono semanticamente deboli rispetto ai diagrammi di sequenza.
- I diagrammi di comunicazione hanno una sintassi simile a quella dei diagrammi di sequenza.
- La sequenza temporale è indicata dalla numerazione gerarchica di ogni messaggio.
- Un semplice diagramma di comunicazione per il caso d'uso `AggiungiCorso` viene riportato nella seguente figura:



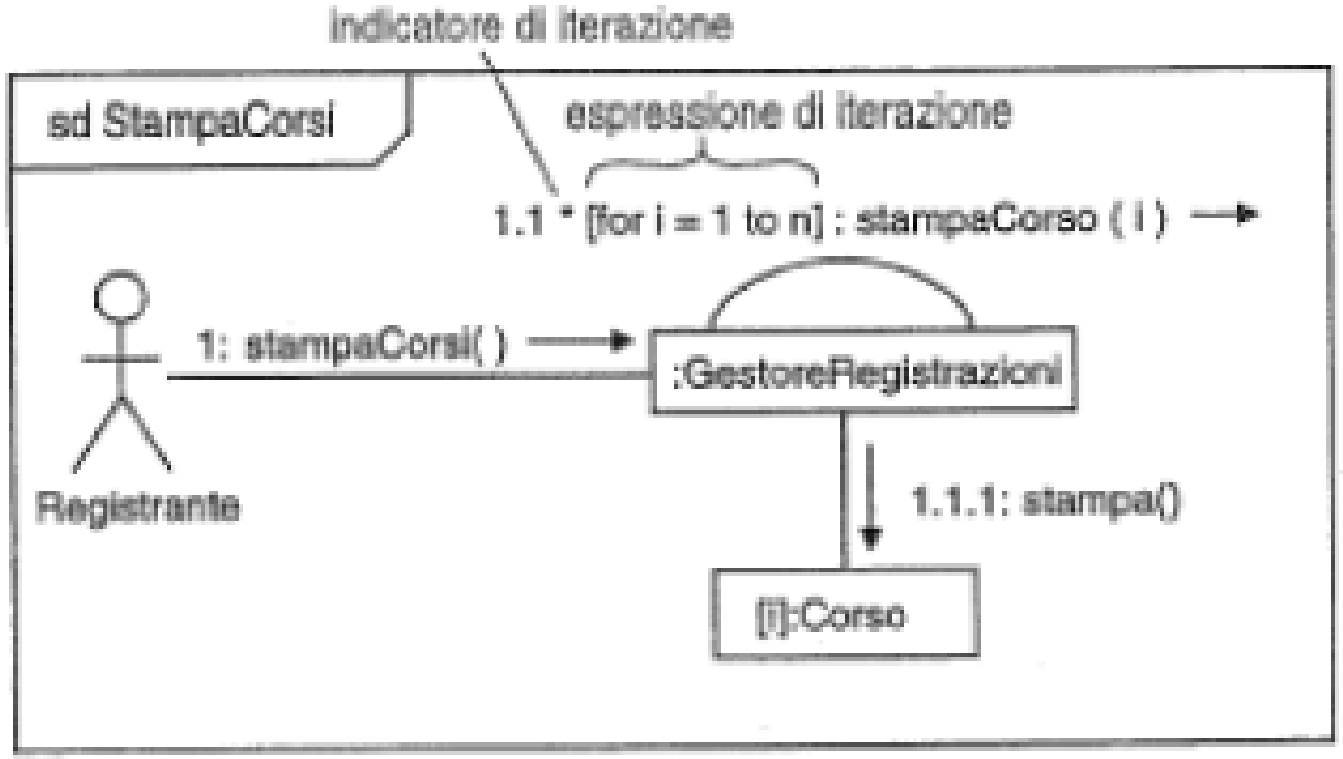
Realizzazione dei casi d'uso – Diagrammi di comunicazione

- `1. aggiungiCorso("UML")`
  - 1.1. `<<crea>>`: questo tipo di messaggi crea nuovi oggetti.
    - In seguito, durante l'analisi o la progettazione, si darà un nome a questo messaggio anonimo.
    - Dopo la creazione dell'oggetto non vi sono più messaggi nel focus di controllo per `aggiungiCorso(…)` e questo flusso restituisce il controllo al chiamante.
- `2. aggiungiCorso("MDA")`
  - 2.1 `<<crea>>`: il numero di sequenza 2.1 indica che si è nel focus di controllo di `aggiungiCorso(…)`.
- Vista la quantità di elementi diversi presenti in un diagramma di comunicazione, questi possono risultare inizialmente difficili da leggere.
- I numeri di sequenza dei messaggi indicano l'annidamento delle invocazioni di operazione al loro interno (cioè un focus di controllo annidato).

Realizzazione dei casi d'uso – Diagrammi di comunicazione – Iterazione

- È possibile mostrare l'iterazione sui diagrammi di comunicazione usando un'espressione di iterazione comprendente un indicatore di iterazione (*) e un'espressione di iterazione condizionale.

- Un esempio di iterazione nei diagrammi di comunicazione viene riportato nella seguente figura:



Realizzazione dei casi d'uso – Diagrammi di comunicazione – Iterazione

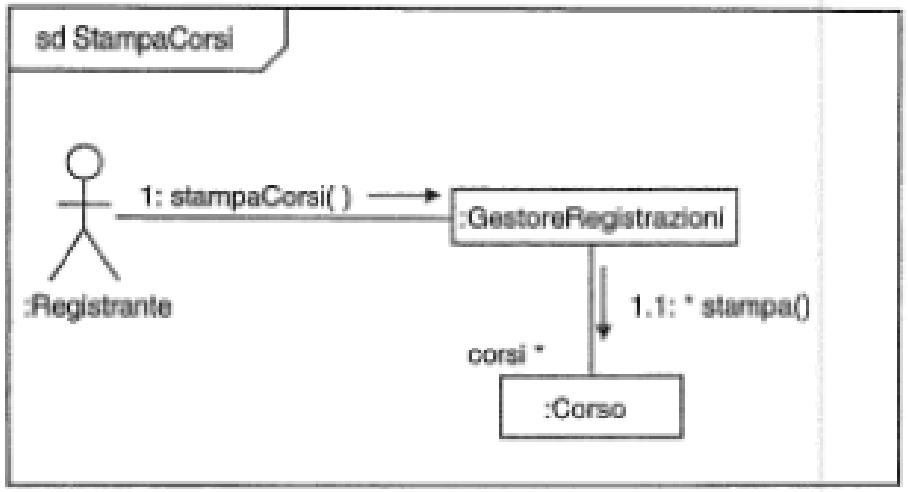
- UML 2 non stabilisce alcuna sintassi particolare per le espressioni di iterazione.
- Si potrebbe pensare che i diagrammi di comunicazione dovrebbero usare di default la sintassi di iterazione per i diagrammi di sequenza; tuttavia, la specifica di UML non lo menziona.
- Se si decide di usare la stessa sintassi di iterazione, un'espressione di iterazione potrebbe essere scritta nel modo seguente:

```
*[loop min, max [[condizione]]
```

- Questa espressione ha il vantaggio della consistenza ma ha una ridondanza sintattica poiché sia `loop` sia `*` sono indicatori di iterazione.
- Nell'esempio precedente è stato usato del pseudo-codice per indicare che l'espressione di iterazione viene ripetuta, incrementando `i` da `1` a `n`.

Realizzazione dei casi d'uso – Diagrammi di comunicazione – Iterazione

- Un approccio alternativo per l'utilizzo dell'interazione nei diagrammi di comunicazione viene riportato nella seguente figura:



- In questo approccio:
  - Viene mostrato il nome del ruolo e la molteplicità sul collegamento da `:GestoreRegistrazioni` a `:Corso`. Questo indica che `:GestoreRegistrazioni` è collegato a una collezione di oggetti di `:Corso` attraverso il ruolo chiamato `corsi`.

Realizzazione dei casi d'uso – Diagrammi di comunicazione – Iterazione

- Viene aggiunto l'indicatore di iterazione al messaggio `stampa()`; questo indica che il messaggio `stampa()` viene inviato a ogni oggetto della collezione.

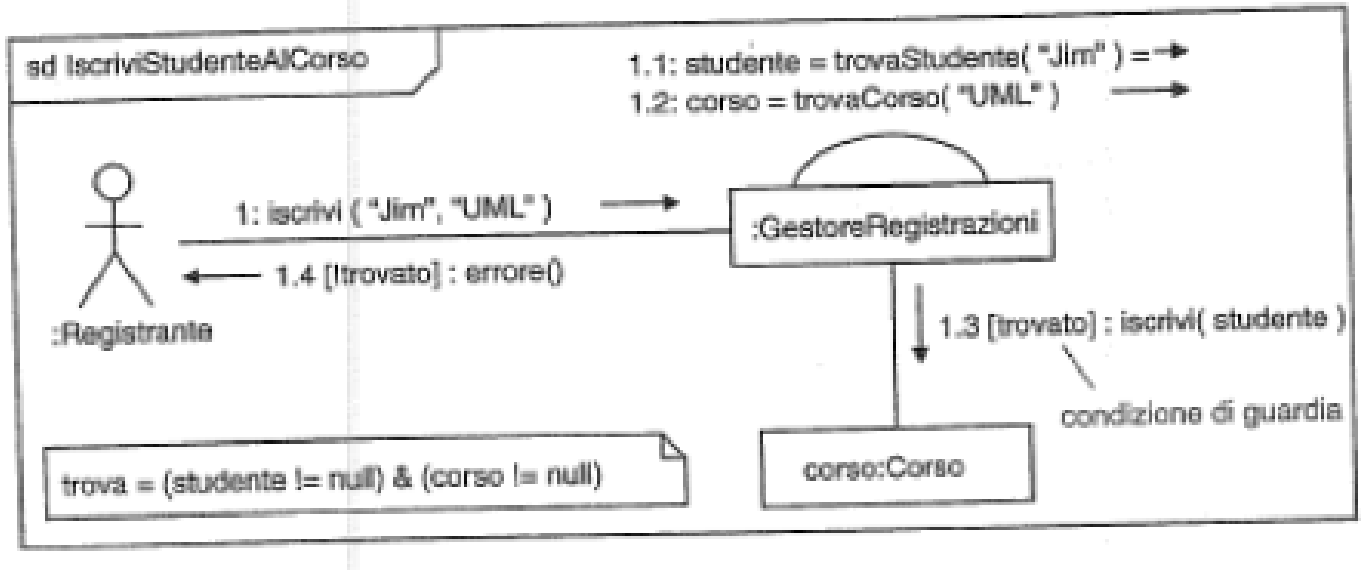
- L'indicatore standard di iterazione (`*`) indica che i messaggi verranno inviati sequenzialmente.

- Se si vuole indicare che i messaggi sono tutti eseguiti in parallelo, si deve usare l'indicatore parallelo di iterazione `*//`.

Realizzazione dei casi d'uso – Diagrammi di comunicazione – Ramificazione

- Le ramificazioni possono essere modellate aggiungendo condizioni di guardia ai messaggi; il messaggio viene inviato solo se la condizione di guardia risulta vera.

- Un esempio di ramificazione relativo al caso d'uso `IscriviStudenteAlCorso`:



- Non esiste una sintassi formale per le condizioni.

- Queste sono spesso espressioni che coinvolgono variabili temporanee nell'ambito del focus di controllo corrente o attributi delle classi coinvolte nell'iterazione.

Realizzazione dei casi d'uso – Diagrammi di comunicazione – Ramificazione

- Nella figura vengono memorizzati i risultati delle operazioni `trovaStudente`(…) e `trovaCorso`(…) in due variabili temporanee `studente` e `corso`.
- I valori di queste variabili vengono poi usati per calcolare il valore della variabile booleana temporanea `trovato`.
- Viene usato `trovato` per la ramificazione al Passo 1.3 e per decidere se attivare una condizione di errore al `:Registrante` al Passo 1.4.
- È difficile illustrare chiaramente le ramificazioni nei diagrammi di comunicazione.
- In generale, in questi diagrammi conviene usare solo ramificazioni molto semplici.

Diagrammi di attività – Cosa sono

- I diagrammi di attività consentono di modellare un processo come un'attività costituita da un insieme di nodi connessi da archi.
- In UML 1 i diagrammi di attività in realtà erano soltanto dei tipi speciali di macchine a stati.
- In UML 2 i diagrammi di attività hanno una semantica completamente nuova basata sulle reti di Petri.
  - Ciò ha due vantaggi:
    - Il formalismo delle reti di Petri fornisce maggiore flessibilità nei diversi tipi di flusso della modellazione.
    - In UML ora c'è una chiara distinzione tra diagrammi di attività e macchine a stati.
- È possibile associare un'attività a qualunque elemento di modellazione al solo fine di modellare il comportamento di tale elemento.

Diagrammi di attività – Cosa sono

- Le attività vengono tipicamente associate a:
  - Casi d'uso.
  - Classi.
  - Interfacce.
  - Componenti.
  - Collaborazioni.
  - Operazioni.
- È possibile usare i diagrammi di attività anche per modellare efficacemente i processi e i flussi di lavoro del business.
- È opportuno considerare che il codice sorgente di un'operazione, in un linguaggio di programmazione o pseudo-codice, potrebbe essere la rappresentazione migliore e più concisa.

Diagrammi di attività – Cosa sono

- L'essenza di un diagramma di attività è quella di comunicare uno specifico aspetto del comportamento dinamico di un sistema.

- A tal fine è necessario che il suo livello di astrazione sia quello più adatto a comunicare questo aspetto agli utenti e contenga la minore quantità di informazioni.

Diagrammi di attività – Relazione con UP

- A causa della loro flessibilità non c'è un unico punto del processo UP in cui collocare i diagrammi di attività.
- Noi li considereremo insieme al flusso di lavoro dell'analisi perché è la situazione in cui sono normalmente utilizzati.
- La caratteristica peculiare dei diagrammi di attività è quella di consentire di modellare un processo senza dover specificare la struttura statica di classi e oggetti che realizzano quel processo.
- L'esperienza insegna che i diagrammi di attività di solito sono usati nei casi seguenti:
  - Nel flusso di lavoro dell'analisi:
    - Per modellare graficamente il flusso di un caso d'uso affinché sia facilmente comprensibile dalle parti interessate.
    - Per modellare il flusso tra più casi d'uso; il flusso usa una forma speciale di diagramma di attività chiamato diagramma di interazione generale.

Diagrammi di attività – Relazione con UP

- Nel flusso di lavoro della progettazione:
  - Per modellare i dettagli di un'operazione.
  - Per modellare i dettagli di un algoritmo.
- Nella modellazione del business:
  - Per modellare un processo di business.

- Le parti interessate solitamente non hanno difficoltà a capire i diagrammi di attività.
- UML 2 introduce una nuova semantica e una sintassi potente per i diagrammi di attività ed è importante non farsi prendere la mano.

Diagrammi di attività – Attività

- Le attività sono reti di nodi connessi ad archi.
- Esistono tre categorie di nodi:
  - Nodi azione: rappresentano unità discrete di lavoro atomiche all'interno dell'attività.
  - Nodi controllo: controllano il flusso attraverso l'attività.
  - Nodi oggetto: rappresentano oggetti usati nell'attività.
- Gli archi rappresentano il flusso attraverso l'attività.
- Esistono due categorie di archi:
  - Flussi di controllo: rappresentano il flusso di controllo attraverso l'attività.
  - Flussi di oggetti: rappresentano il flusso di oggetti attraverso l'attività.

Diagrammi di attività – Attività

- Nella seguente figura viene riportato un semplice diagramma di attività per il processo di business `Invia lettera`:



- Si noti che le attività possono avere precondizioni e postcondizioni come i casi d'uso.

- Anche le azioni all'interno dell'attività possono avere le loro precondizioni e postcondizioni locali.

Diagrammi di attività – Attività

- Le attività spesso iniziano con un singolo nodo controllo, il nodo iniziale, che indica il punto in cui l'esecuzione inizierà quando l'attività viene invocata.
- Uno o più nodi finali indicano il punto in cui l'attività termina.
- Normalmente i diagrammi di attività vengono usati per modellare un caso d'uso come una serie di azioni.

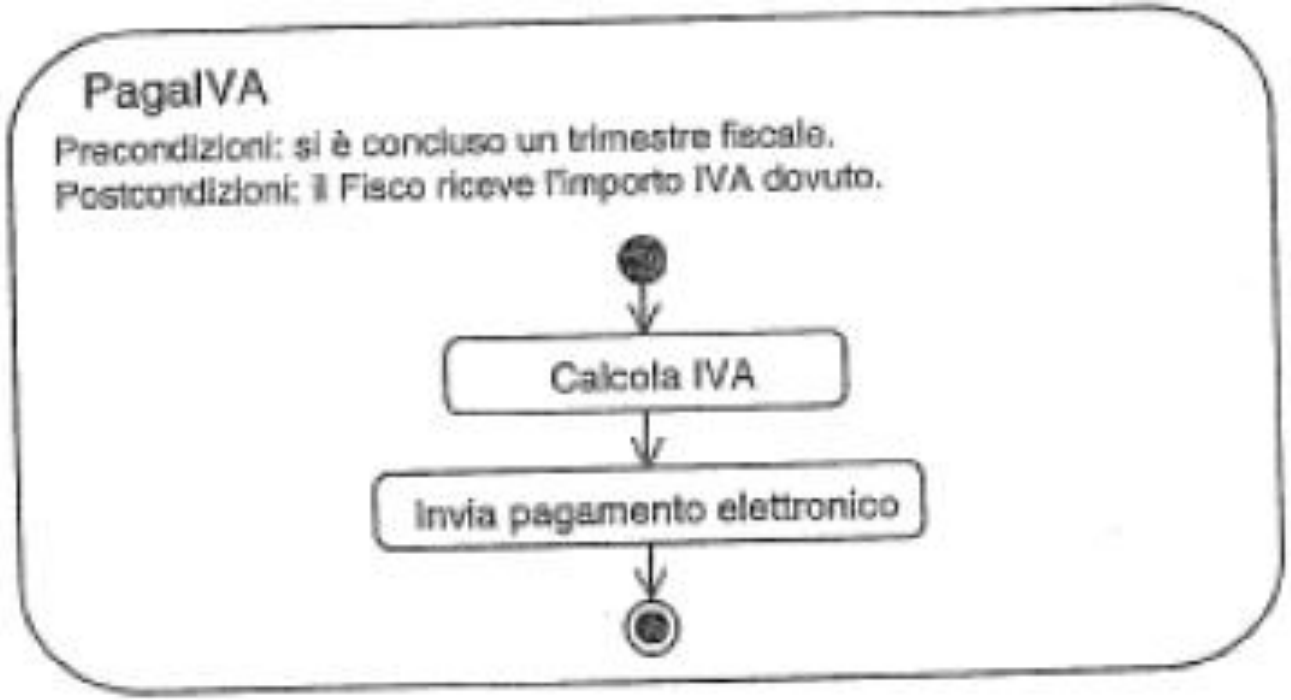
Diagrammi di attività – Attività

- Consideriamo il seguente caso d'uso `PagaIVA`:

Caso d'uso: PagaIVA
ID: 1
Breve descrizione: Pagamento dell'IVA alla fine del trimestre fiscale.
Attori primari: Tempo
Attori secondari: Fisco
Precondizioni: 1. Si è concluso un trimestre fiscale.
Sequenza degli eventi principale: 1. Il caso d'uso inizia quando si conclude un trimestre fiscale. 2. Il sistema calcola l'ammontare dell'IVA dovuta al Fisco. 3. Il sistema trasmette un pagamento elettronico al Fisco.
Postcondizioni: 1. Il Fisco riceve l'importo IVA dovuto.
Sequenza degli eventi alternativa: Nessuna.

Diagrammi di attività – Attività

- Questo caso d'uso può essere espresso mediante il diagramma di attività mostrato nella seguente figura:



- Si noti che il diagramma di attività fornisce una forma grafica più compatta del caso d'uso.

- Il diagramma di attività esprime il caso d'uso come due azioni: `Calcola IVA` e `Invia Pagamento Elettronico`.

Diagrammi di attività – Attività

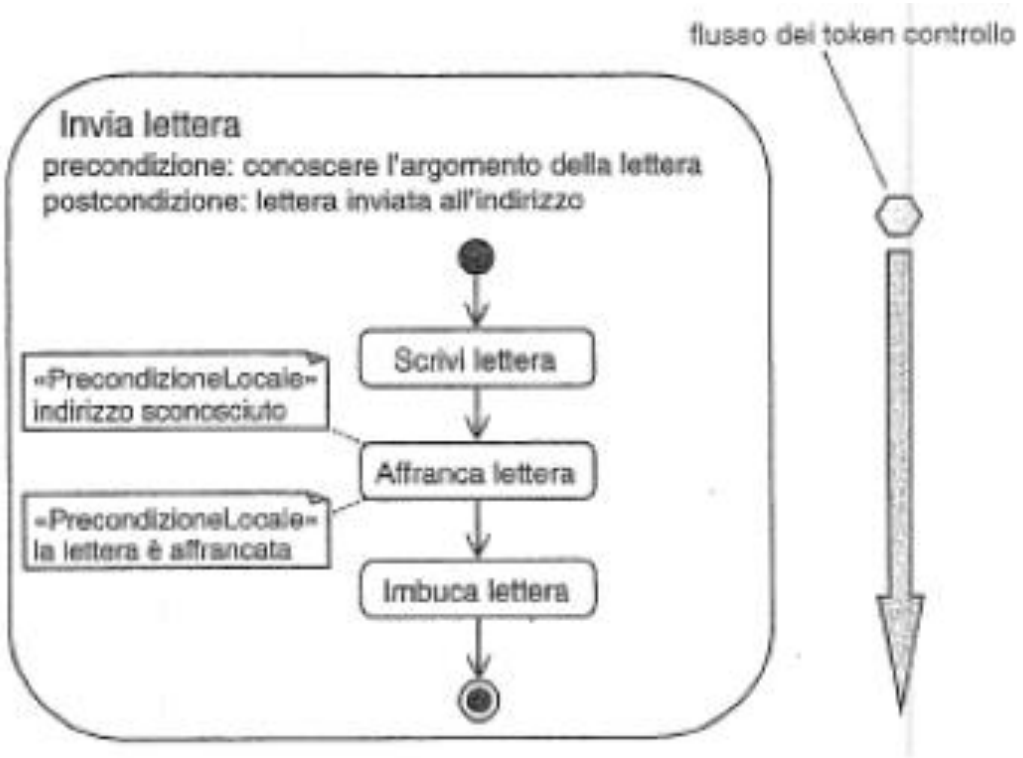
- Ognuna di queste azioni potrebbe a sua volta essere espressa come un diagramma di attività.
- Questo probabilmente andrà fatto nel flusso di lavoro della progettazione quando si deve specificare come sono implementate le azioni.
- L'attore e la sua interazione con il sistema sono caratteristiche strutturali che non sono riportate nel diagramma.
- I casi d'uso esprimono il comportamento del sistema come un'interazione tra un attore e il sistema, mentre i diagrammi di attività lo esprimono come una serie di azioni.
- Questi sono punti di vista complementari sullo stesso comportamento.

Diagrammi di attività – Semantica delle attività

- I diagrammi di attività hanno una semantica abbastanza intuitiva.
- I diagrammi di attività modellano il comportamento usando il cosiddetto token game che descrive il flusso di token attorno a una rete di nodi e archi in base a regole specifiche.
- Nei diagrammi di attività UML i token possono rappresentare:
  - Il flusso di controllo.
  - Un oggetto.
  - Alcuni dati.
- Lo stato del sistema è determinato in qualsiasi momento dalla disposizione dei token.
- Nel primo diagramma di attività visto il token è il flusso di controllo perché non ci sono oggetti o dati che devono passare tra i nodi in questo particolare caso.

Diagrammi di attività – Semantica delle attività

- I token si spostano da un nodo sorgente a un nodo destinazione attraverso un arco.
- Il movimento di un token è soggetto a condizioni e può verificarsi solo quando tutte le condizioni sono soddisfatte; le condizioni cambiano a seconda del tipo di nodo.
- Consideriamo il seguente ulteriore esempio di diagramma di attività:



Diagrammi di attività – Semantica delle attività

- Per i nodi della figura precedente (nodi azioni) queste condizioni sono:
  - Le postcondizioni del nodo sorgente.
  - Le condizioni di guardia sull'arco.
  - Le precondizioni del nodo destinazione.
- I nodi controllo hanno una semantica speciale che controlla come i token vengono passati dai loro archi di entrata ai loro archi di uscita; esempi di nodi controllo sono: il nodo iniziale, il nodo finale e il nodo di ricongiunzione.
- I nodi oggetto rappresentano oggetti che fluiscono nel sistema.
- Nel diagramma di attività precedente il flusso di controllo passa alternativamente attraverso ogni nodo azione, causandone l'esecuzione.
- Questa è la semantica dell'attività principale.

Diagrammi di attività – Semantica delle attività

- Lo stato del sistema in esecuzione può essere rappresentato in qualsiasi momento mediante la disposizione dei suoi token.
  - Per esempio, quando il token si trova nel nodo azione `Scrivi lettera` si potrebbe dichiarare che il sistema è nello stato `Scrittura lettera`.
- Non tutte le esecuzioni di azioni o i flussi di token costituiscono un cambiamento significativo nello stato del sistema.
- Ciò nonostante, la disposizione dei token fornisce un collegamento tra diagrammi di attività e macchine a stati.
- Si deve assicurare che i diagrammi di attività e le macchine a stati per un particolare elemento di modello siano coerenti tra loro.
- Un'attività è soltanto una specifica per cui ci possono essere numerose implementazioni.
- Per esempio, la figura precedente illustra un semplice processo di business più che un sistema software e l'implementazione di questo processo normalmente non richiede certo il passaggio di token.

Diagrammi di attività – Partizioni delle attività

- Le attività si possono dividere in partizioni usando linee verticali, orizzontali o curve.
- Ogni partizione rappresenta un raggruppamento ad alto livello di azioni correlate; le partizioni della attività sono talvolta chiamate corsie.
- Il partizionamento è una tecnica potente se usata bene.
- In UML 2 il modellatore definisce la semantica delle partizioni delle attività che non hanno alcuna semantica intrinseca; è quindi possibile usare le partizioni per suddividere i diagrammi delle attività come si preferisce.
- Le partizioni delle attività vengono comunemente usate per rappresentare:
  - Casi d'uso.
  - Classi.
  - Componenti.
  - Unità organizzative (nella modellazione di business).
  - Ruoli (nella modellazione dei flussi di lavoro).

Diagrammi di attività – Partizioni delle attività

- Nei modelli di progettazione di sistemi distribuiti si possono utilizzare le partizioni delle attività anche per modellare la distribuzione dei processi sulle macchine fisse.

- Ogni insieme di partizioni dovrebbe avere una singola dimensione che ne descrive la semantica di base; all'interno di questa dimensione le partizioni possono essere annidate gerarchicamente.

- La seguente figura mostra un'attività che ha un insieme di partizioni delle attività annidate gerarchicamente:



Diagrammi di attività – Partizioni delle attività

- Nella figura seguente la dimensione è `Locazione` e all'interno di questa dimensione c'è una gerarchia di partizioni:



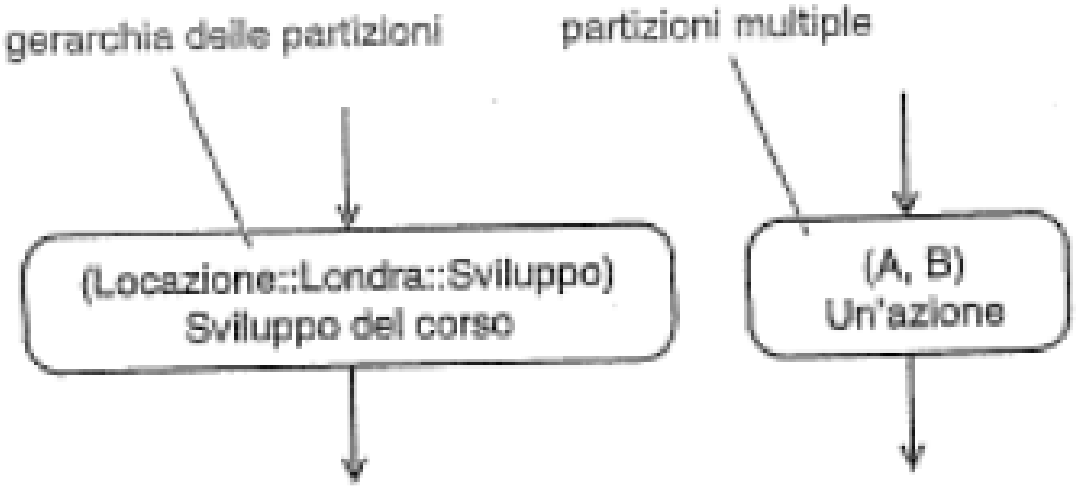
- Tra le partizioni delle attività e i flussi di controllo concorrenti spesso c'è una connessione.

- Talvolta non si riesce a ordinare i nodi in partizioni orizzontali o verticali senza rendere difficile la lettura del diagramma; in questo caso per creare delle partizioni irregolari si possono usare delle linee curve.

Diagrammi di attività – Partizioni delle attività

- La seguente figura illustra la notazione testuale per le partizioni delle attività.

- Essa viene tipicamente usata solo come ultima risorsa perché la notazione grafica di solito è molto più chiara.



- È possibile specificare la posizione di un'azione all'interno della gerarchia delle partizioni usando un pathname tra parentesi delimitato da una coppia di due punti sopra il nome dell'azione.

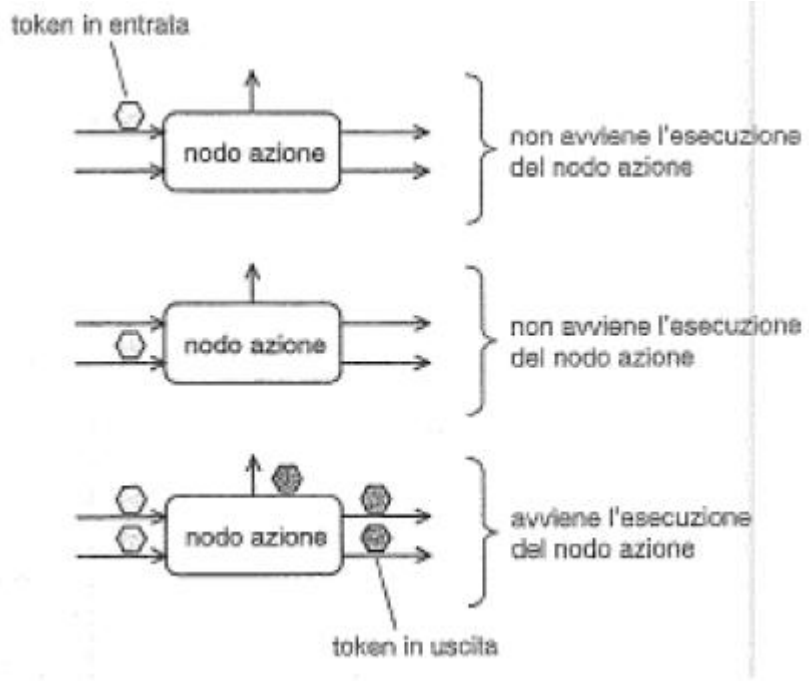
- Molto occasionalmente può essere necessario visualizzare in un diagramma di attività un comportamento che, in senso stretto, è al di fuori dello scopo del sistema.

Diagrammi di attività – Partizioni delle attività

- Il motivo potrebbe essere quello di mostrare come il sistema interagisce con un altro sistema esterno.
- I diagrammi di attività possono gestire questa situazione aggiungendo lo stereotipo <<esterna>> direttamente sopra il nome della partizione.
- Si osservi che la partizione esterna non fa parte del sistema e quindi non può essere inserita all'interno delle gerarchie di partizioni del modello.
- Tramite la scelta attenta di dimensioni e partizioni di attività è possibile aggiungere molte informazioni utili a un diagramma di attività.
- In pratica si deve cercare di usare non più di tre livelli gerarchici (tra cui la dimensione) e non più di due dimensioni per diagramma.

Diagrammi di attività – Nodi azione

- L'esecuzione dei nodi azione avviene quando queste condizioni sono simultaneamente soddisfatte:
  - Esiste un token simultaneamente su ciascun arco entrante.
  - I token in ingresso soddisfano tutte le precondizioni locali del nodo azione, come mostrato nella figura seguente:

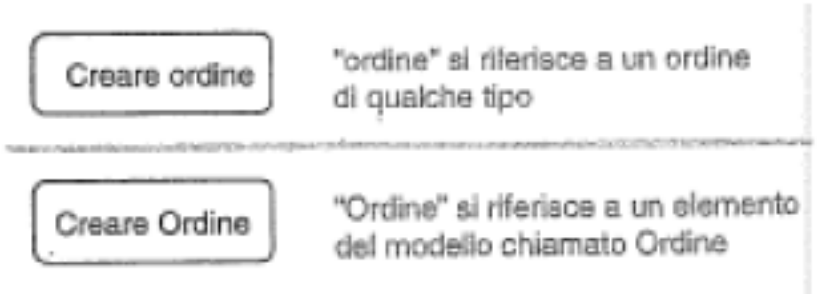


Diagrammi di attività – Nodi azione

- Il nodo non è pronto all'esecuzione finché non sono presenti token su tutti i suoi archi di entrata.
- Anche quando i token richiesti sono presenti l'esecuzione del nodo avviene solo se è soddisfatta la precondizione locale del nodo.
- Quando il nodo azione ha terminato l'esecuzione, la postcondizione locale viene controllata; se è soddisfatta il nodo emette simultaneamente i token su tutti i suoi archi uscenti.
- A differenza dei diagrammi di flusso i diagrammi di attività sono intrinsecamente concorrenti.
- Poiché i nodi di azione eseguono qualcosa i loro nomi normalmente sono dei verbi o delle frasi verbali.
- La specifica UML non fornisce indicazioni precise sulla denominazione dei nodi azione.
- La regola che si segue di solito prevede di assegnare un nome che inizia con una lettera maiuscola e continua con lettere minuscole e dove servono vengono inseriti degli spazi.
- L'unica eccezione si ha quando un nodo azione contiene un riferimento a un altro elemento del modello; in questo caso si usa sempre il nome dell'elemento del modello com'è senza modificare lettere maiuscole e minuscole e senza aggiungere spazi.

Diagrammi di attività – Nodi azione

- Quanto detto in precedenza viene mostrato graficamente nella seguente figura:



- La specifica del nodo azione cattura i dettagli dell'azione.

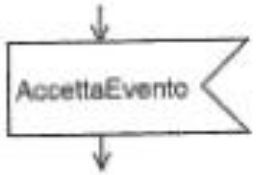
- Spesso è solo una descrizione testuale quale "Scrivere una lettera", ma nella progettazione potrebbe essere testo strutturato, pseudo-codice o codice vero e proprio.

- Se il diagramma di attività modella un caso d'uso la specifica potrebbe essere una o più fasi del flusso del caso d'uso.

- È necessario rendersi conto, però, che ciò può creare un problema di gestione perché bisogna mantenere allineati il caso d'uso e il diagramma di attività se uno dei due cambia.

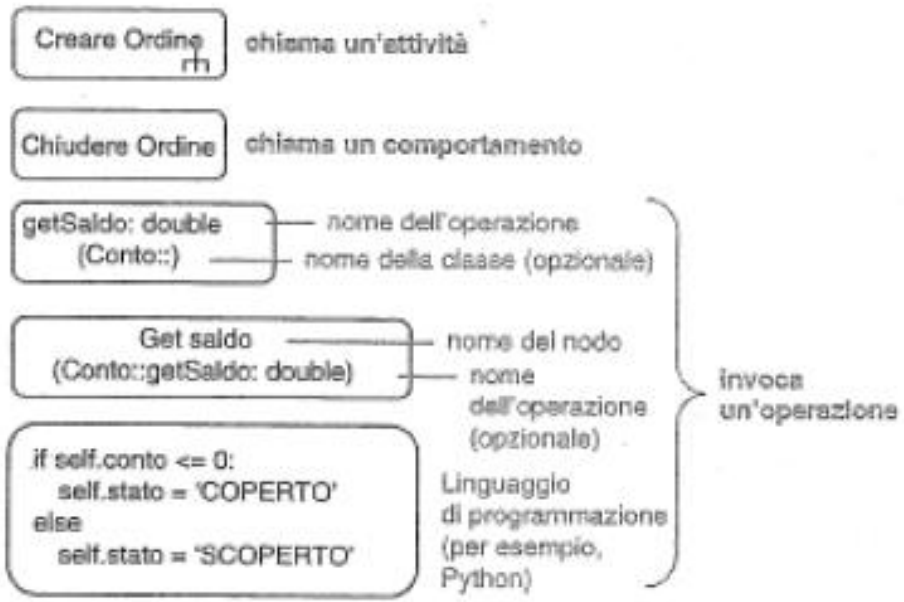
Diagrammi di attività – Nodi azione

- Esistono quattro tipi di nodi azione; essi sono riportati nella seguente figura:

Sintassi	Nome	Semantica	Paragrafo
	Nodo azione di chiamata	Invoca un'attività, un comportamento o un'operazione	13.7.1
	Inviare segnale	Invia segnale di azione: invia un segnale in modo asincrono (il mittente non aspetta la conferma di segnale ricevuto). Può accettare i parametri di input necessari per creare il segnale.	
	Nodo azione di accettazione evento	Accetta un evento: aspetta gli eventi individuati dal suo oggetto proprietario ed emette l'evento sul suo arco di uscita. È attivato quando riceve un *token* sul suo arco in entrata. Se *non* c'è alcun arco entrante, inizia quando la sua attività contenente inizia ed è sempre attiva.	
	Nodo azione di accettazione evento temporale	Accetta un evento temporale: risponde al tempo. Genera eventi temporali secondo la sua espressione temporale.	13.7.2

Diagrammi di attività – Nodi azione – Nodo azione di chiamata

- Il tipo più comune di nodo azione è il nodo azione di chiamata che può attivare:
  - Un'attività.
  - Un comportamento.
  - Un'operazione.
- La seguente figura illustra alcuni esempi di sintassi del nodo azione di chiamata:

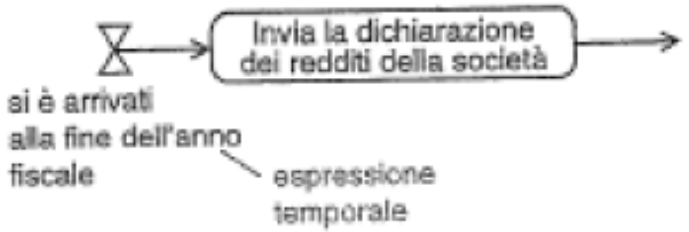


Diagrammi di attività – Nodi azione – Nodo azione di chiamata

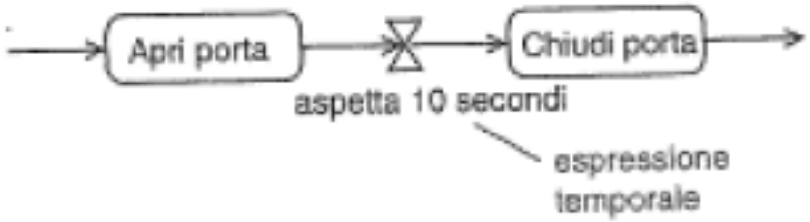
- Come si può vedere dalla figura, la sintassi del nodo azione di chiamata è molto flessible:
  - È possibile indicare che l'azione invoca un'altra attività usando il simbolo speciale del rastrello nell'angolo inferiore destro dell'icona del nodo.
  - È possibile chiamare un comportamento.
  - È possibile chiamare un'operazione usando la sintassi standard delle operazioni.
  - È possibile chiamare un'operazione specificando i dettagli dell'operazione in un particolare linguaggio di programmazione.
  - È possibile far riferimento alle caratteristiche del contesto dell'attività usando la parola chiave `self`.
- Quando si usano i nodi azione di chiamata nei diagrammi di attività a livello di analisi normalmente si chiamano i comportamenti.
- I nodi azione di chiamata che invocano operazioni specifiche tendono invece a essere usati nella progettazione per la modellazione più dettagliata delle attività.

Diagrammi di attività – Nodi azione – Nodo azione di accettazione evento temporale

- Questo tipo di nodo ha un'espressione temporale e genera un evento temporale quando questa espressione diventa vera.
- Il nodo si comporta in modo diverso a seconda se ha o meno un arco di ingresso.
- La figura seguente mostra un nodo di accettazione evento temporale senza alcun arco in ingresso:



- Nell'esempio della figura seguente l'azione di accettazione evento temporale ha un arco in ingresso e diventa attiva solo quando su quell'arco viene ricevuto un token:

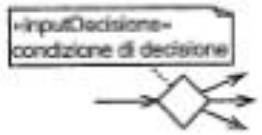
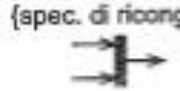


Diagrammi di attività – Nodi azione – Nodo azione di accettazione evento temporale

- Si noti che l'azione temporale può far riferimento a:
  - Un evento del tempo (per esempio, `fine dell'anno fiscale`).
  - Un punto nel tempo (per esempio, `11/03/1960`).
  - Una durata (per esempio, `aspetta 10 secondi`).

Diagrammi di attività – Nodi controllo

- I nodi controllo gestiscono il flusso di controllo all'interno di un'attività.
- La seguente figura riassume tutti i nodi di controllo in UML 2:

Sintassi	Nome	Descrizione		Paragrafo
	Nodo iniziale	Indica dove inizia il flusso quando viene invocata un'attività.		13.8.1
	Nodo finale dell'attività	Termina un'attività.	Nodi finali	13.8.1
	Nodo finale del flusso	Termina un flusso specifico all'interno di un'attività: gli altri flussi non vengono influenzati.		13.8.1
	Nodo decisione	Viene attraversato l'arco in uscita la cui condizione di guardia è vera.		13.8.2
	Nodo fusione	Copia *token* in ingresso nel suo unico arco in uscita.		13.8.2
	Nodo biforcazione	Divide il flusso in più flussi concorrenti.		13.8.3
	Nodo ricongiunzione	Sincronizza più flussi concorrenti. Opzionalmente può avere una specifica di ricongiunzione per modificare la sua semantica.		13.8.3

Diagrammi di attività – Nodi controllo – Nodi iniziali e nodi finali

- Il nodo iniziale è il punto in cui inizia il flusso quando viene invocata un'attività.
- Un'attività può avere più di un nodo iniziale; in questo caso i flussi incominciano simultaneamente in tutti i nodi iniziali e vengono eseguiti contemporaneamente.
- Un'attività può anche essere iniziata da un'azione di accettazione evento o da un nodo parametro di attività; quindi i nodi iniziali non sono obbligatori se esiste un altro modo per iniziare l'attività.
- Il nodo finale delle attività ferma tutti i flussi all'interno di un'attività; un'attività può avere molti nodi finali; il primo ad essere attivato fa terminare tutti gli altri flussi e l'attività stessa.
- Il nodo finale dei flussi ferma, invece, soltanto uno dei flussi all'interno dell'attività; gli altri continuano.

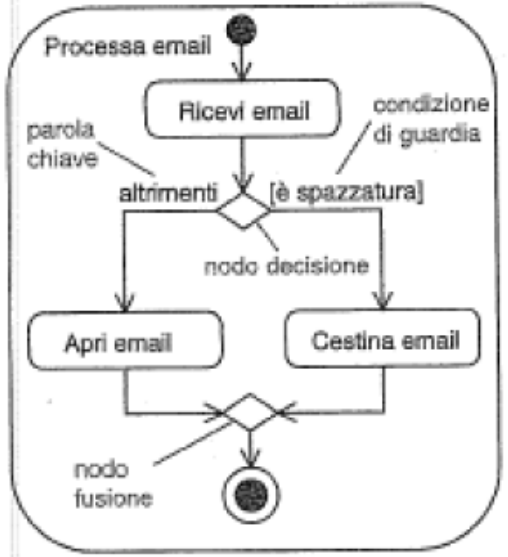
Diagrammi di attività – Nodi controllo – Nodi decisione e fusione

- Un nodo decisione ha un arco in ingresso e due o più archi alternativi in uscita; un token che arriva all'arco in ingresso viene offerto a tutti gli archi in uscita, ma ne attraversa al massimo uno.

- Ogni arco in uscita è protetto da una condizione di guardia tale che l'arco accetta un token se, e solo se, la condizione di guardia corrisponde al valore vero.

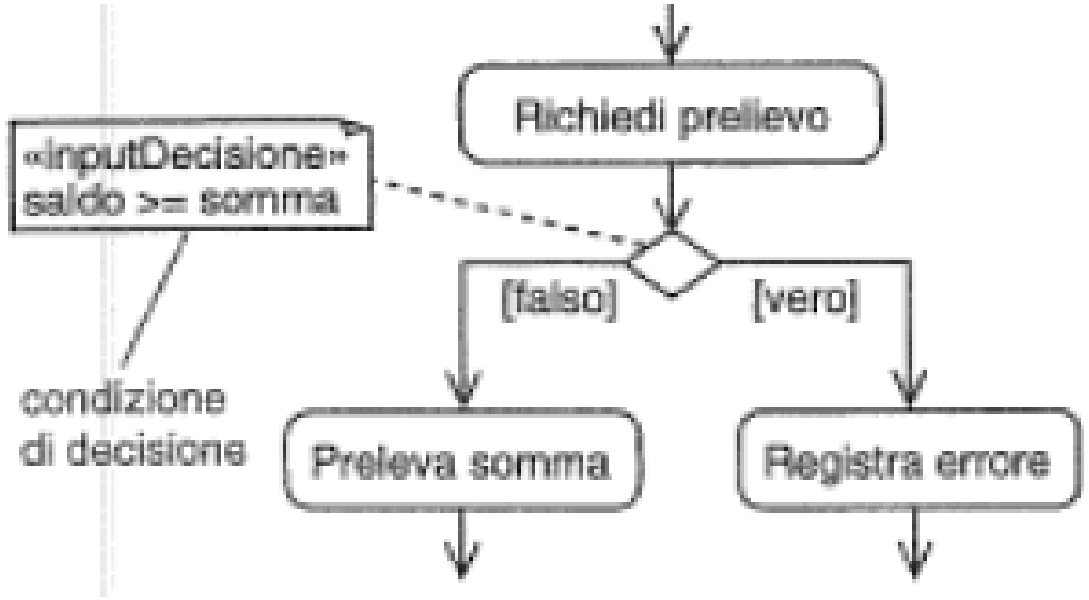
- La parola chiave `altrimenti` può essere usata per specificare l'arco da attraversare, se nessuna delle condizioni di controllo è vera.

- La figura seguente mostra un semplice esempio di nodo decisione:



Diagrammi di attività – Nodi controllo – Nodi decisione e fusione

- Un nodo con stereotipo <<`inputDecisione`>> fornisce una condizione di decisione per un nodo decisione.
- La figura seguente mostra un esempio di utilizzo di un nodo decisione:

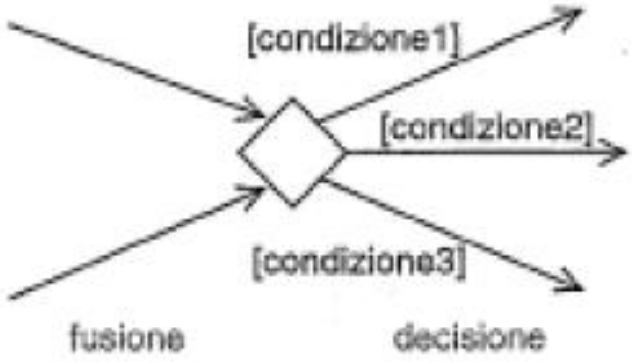


Diagrammi di attività – Nodi controllo – Nodi decisione e fusione

- La figura seguente presenta un nodo fusione; i nodi fusione hanno due o più archi in ingresso e un solo arco in uscita e fondono tutti i loro flussi entranti in un solo flusso uscente:

- La semantica della fusione è molto semplice; tutti i token offerti sugli archi entranti vengono inviati sull'arco uscente e non ci sono modifiche del flusso o dei token.

- Un nodo fusione e un nodo decisione posti uno dopo l'altro possono essere uniti in un singolo simbolo, come mostrato nella figura precedente.



Diagrammi di attività – Nodi controllo – Nodi biforcazione e ricongiunzione: concorrenza

- All'interno di un'attività è possibile creare flussi concorrenti usando un nodo biforcazione per dividere un singolo flusso in più flussi concorrenti.
- I nodi biforcazione e ricongiunzione vengono usati sia nel flusso di lavoro dell'analisi sia nella progettazione.
- Un nodo biforcazione ha un arco in ingresso e due o più archi in uscita; i token che arrivano all'arco in ingresso vengono duplicati e offerti simultaneamente su tutti gli archi in uscita.
- Ciascun arco in uscita può avere una condizione di guardia che deve essere vera affinchè il token possa attraversare l'arco.
- I nodi ricongiunzione hanno più archi in ingresso e un solo arco in uscita.
- Essi sincronizzano i flussi offrendo un token sul loro singolo arco in uscita quando c'è un token su tutti i loro archi in ingresso.
- Essi eseguono un'operazione logica `AND` sui loro archi in ingresso.

Diagrammi di attività – Nodi controllo – Nodi biforcazione e ricongiunzione: concorrenza

- Un esempio di utilizzo dei nodi di biforcazione e ricongiunzione viene riportato nella seguente figura:

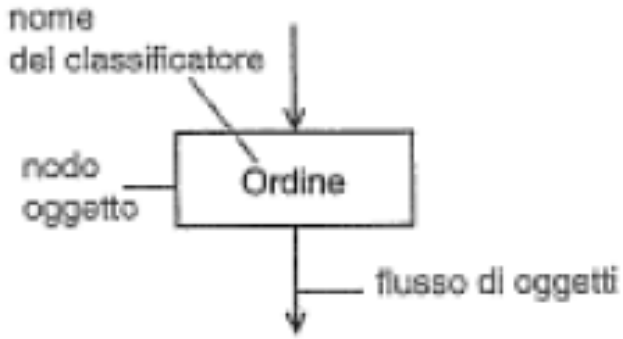


- Quando si modellano i nodi ricongiunzione è importante assicurarsi che tutti gli archi in ingresso nella ricongiunzione ricevano un token.

- Per esempio, se nella figura ci fossero condizioni di guardia mutuamente esclusive sui flussi in uscita della biforcazione alla ricongiunzione non potrebbero mai essere offerti sufficienti token da attivare e questo causerebbe il blocco delle attività.

Diagrammi di attività – Nodi oggetto

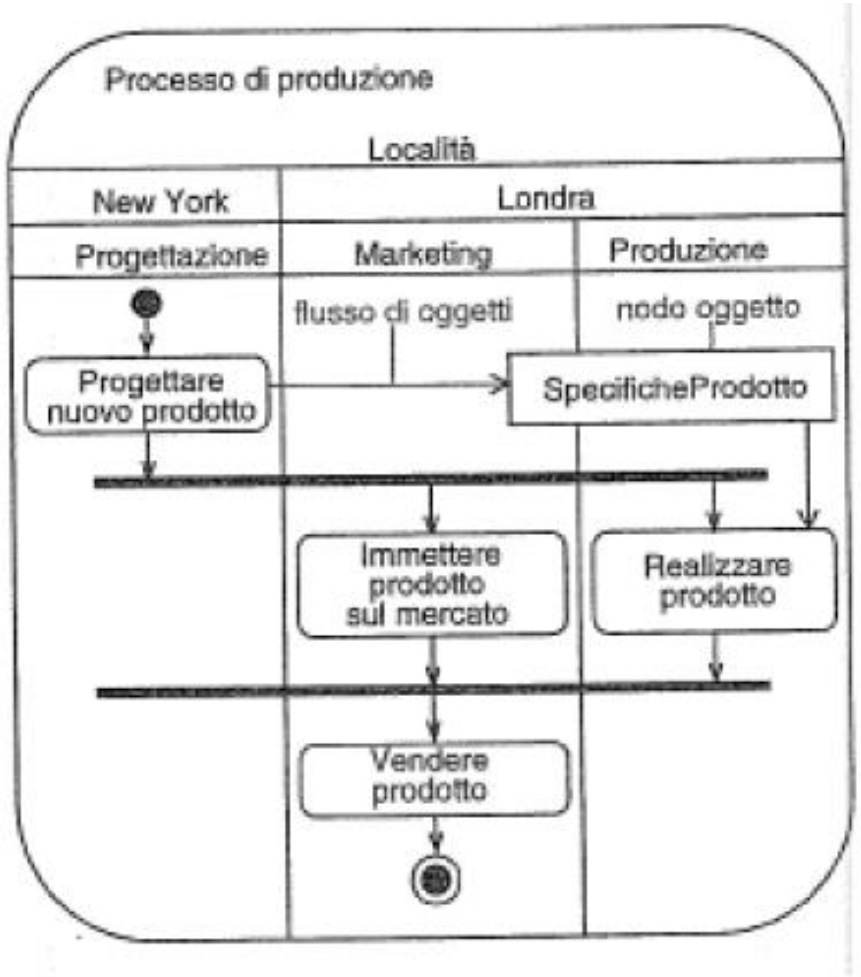
- I nodi oggetto sono nodi speciali che indicano che sono disponibili istanze di un particolare classificatore in un punto specifico dell'attività.
- Essi sono etichettati con il nome del classificatore e rappresentano le istanze di quel classificatore o delle sue sottoclassi.
- Un esempio di nodo oggetto viene riportato nella seguente figura:



- Gli archi in entrata e in uscita di nodi oggetto sono flussi di oggetti.
- Gli oggetti stessi vengono creati e consumati da nodi azione.

Diagrammi di attività – Nodi oggetto

- La seguente figura illustra la semplice attività `Processo di produzione` introdotta nella figura precedente, aggiornata per includere partizioni e per mostrare la creazione di un nodo oggetto `SpecificheProdotto` con l'azione `Progettare nuovo prodotto`.

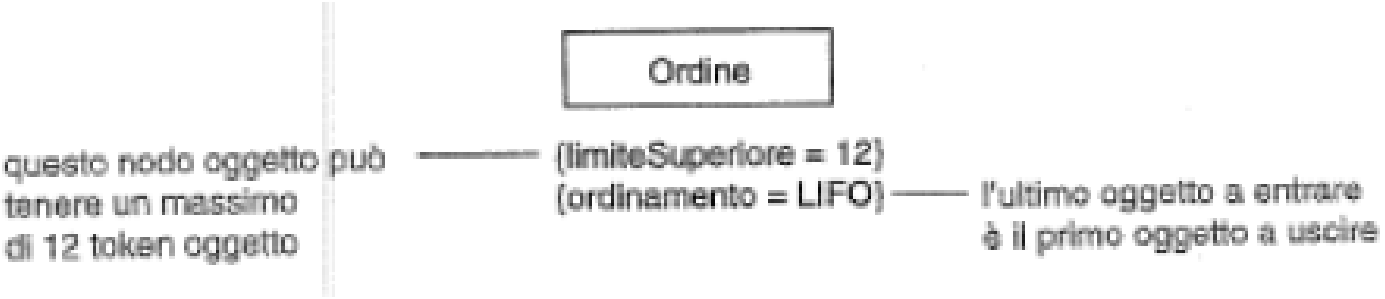


Diagrammi di attività – Nodi oggetto

- Quando un nodo oggetto riceve un token oggetto su uno dei suoi archi in ingresso lo offre simultaneamente su tutti i suoi archi in uscita e questi competono per il token.

Diagrammi di attività – Nodi oggetto – Semantica del buffer dei nodi oggetto

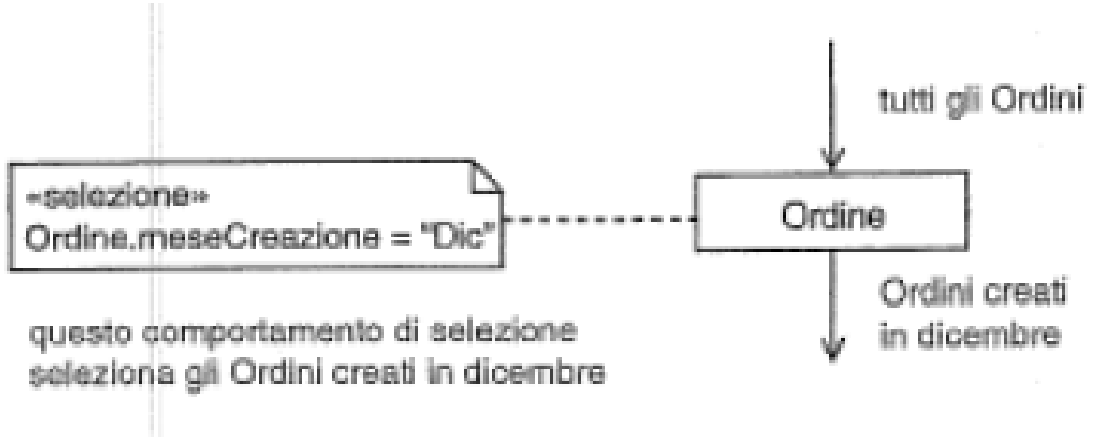
- I nodi oggetto fungono da buffer in cui i token oggetto possono risiedere mentre aspettano di essere accettati da altri nodi.
- Per default ogni nodo oggetto può tenere un numero infinito di token oggetto.
- Tuttavia, a volte bisogna specificare che il buffer ha una dimensione finita.
- La seguente figura mostra un esempio di un nodo oggetto con un limite superiore specificato:



- Per i nodi oggetto è possibile specificare due aspetti della semantica del buffer:
  - I nodi oggetto hanno un ordinamento che specifica come si comporta il buffer; l'ordinamento predefinito è quello FIFO.

Diagrammi di attività – Nodi oggetto – Semantica del buffer dei nodi oggetto

- I nodi oggetto possono avere un comportamento di selezione.
  - La selezione è specificata da una nota con stereotipo <<selezione>>, come mostrato nella seguente figura:



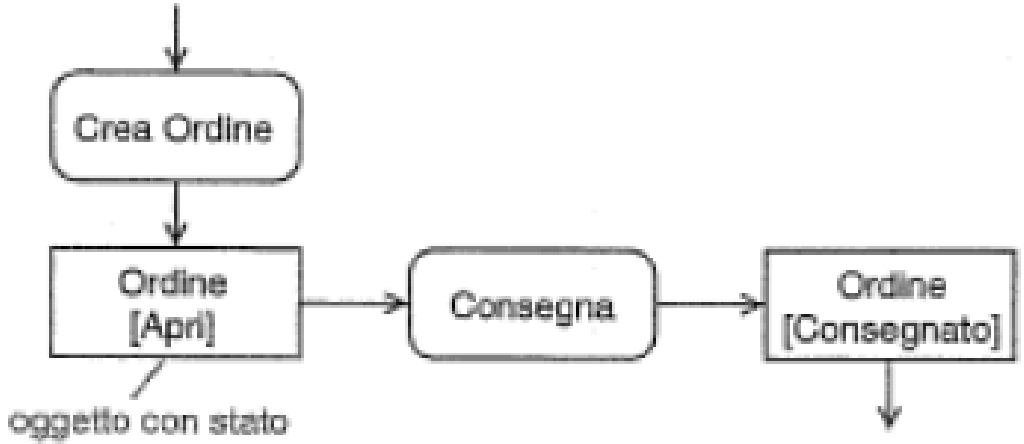
  - In questo esempio il nodo oggetto seleziona solo quegli oggetti ordine che sono stati creati nel mese di dicembre e li offre ai suoi archi in uscita usando l'ordinamento predefinito (FIFO).
  - Nella figura precedente viene mostrata la specifica della selezione in un nodo oggetto.

Diagrammi di attività – Nodi oggetto – Semantica del buffer dei nodi oggetto

- Un nodo oggetto può essere usato per riunire oggetti da più flussi o distribuire oggetti in più flussi di oggetti in uscita.
- In questo caso si può utilizzare lo stereotipo <<`bufferCentrale`>>.
- Oltre alla bufferizzazione di singoli oggetti, i nodi oggetto possono memorizzare nel buffer anche insiemi di oggetti.
- Per far risaltare ciò basta semplicemente porre `Insieme di` come prefisso al nome del classificatore.

Diagrammi di attività – Nodi oggetto – Rappresentazione di oggetti con stato

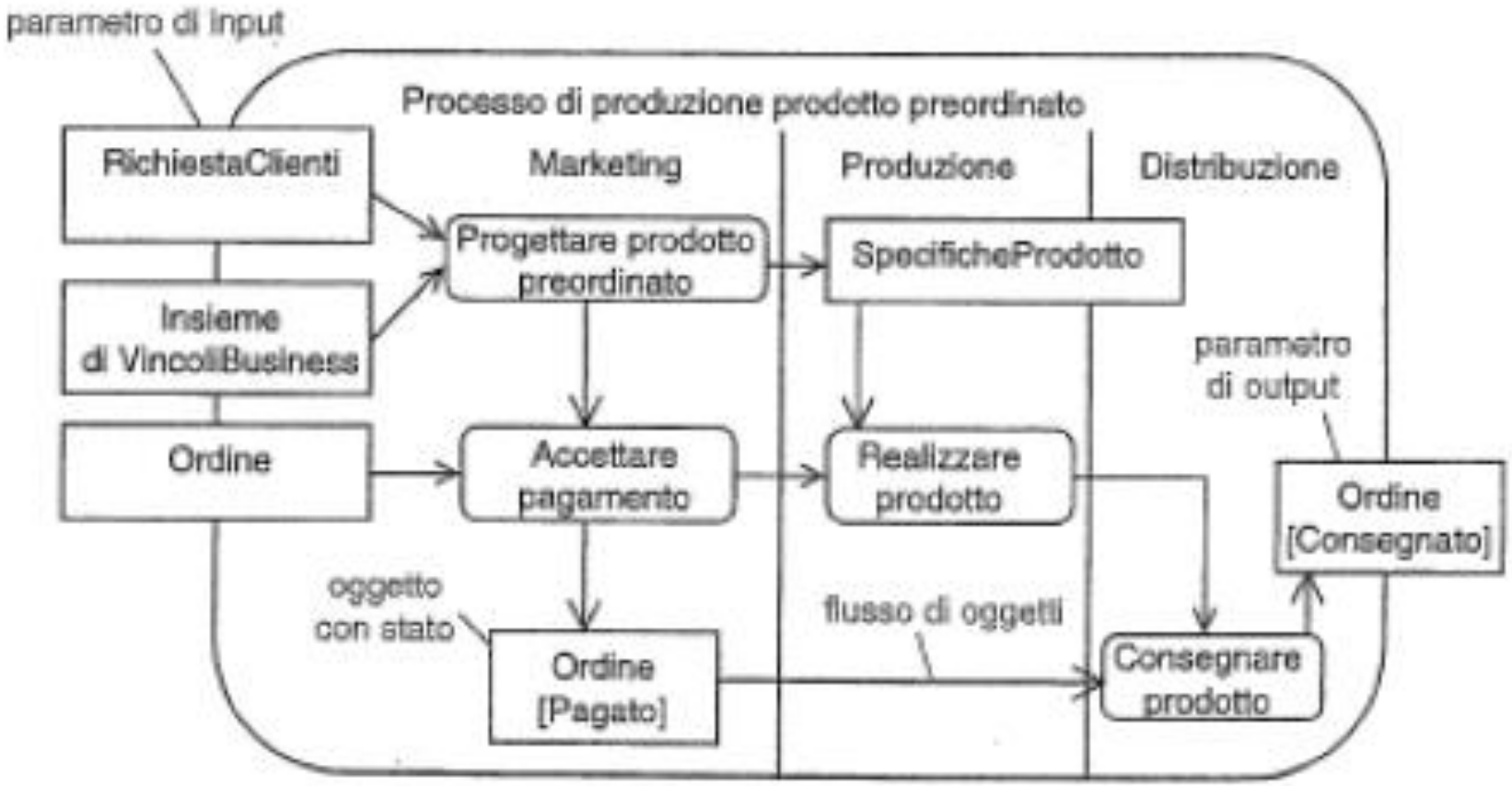
- I nodi oggetto possono rappresentare gli oggetti in un particolare stato.
- Un esempio di rappresentazione degli oggetti con stato viene mostrato nella seguente figura:



- Gli stati degli oggetti a cui si fa riferimento con i nodi oggetto possono essere modellati con le macchine a stati.

Diagrammi di attività – Nodi oggetto – Parametri di attività

- La seguente figura mostra come è possibile usare nodi oggetto per fornire input alle attività:



- I nodi oggetto in ingresso e in uscita dovrebbero essere disegnati in modo che si sovrappongano al riquadro che delimita l'attività.

Diagrammi di attività – Nodi oggetto – Parametri di attività

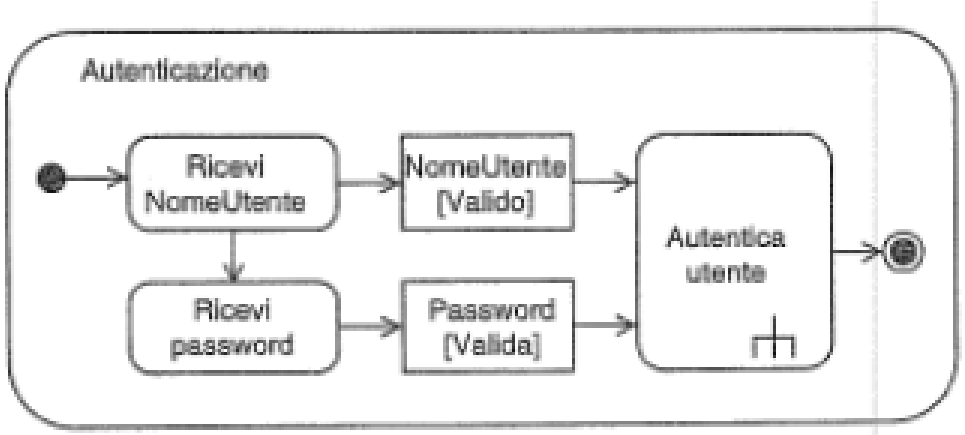
- Nella figura precedente l'attività `Processo di produzione prodotto preordinato` ha tre parametri di input `RichiestaClienti`, `Insieme di VincoliBusiness` e `Ordine` e un parametro di output, `Ordine`. Il nodo `Insieme di VincoliBusiness` contiene un insieme di oggetti `VincoliBusiness`.
- Ci sono molti requisiti di business per questo processo:
  - I prodotti sono progettati in base a una `RichiestaClienti` e ciò crea `SpecificheProdotto`.
  - Il progetto del prodotto tiene conto dei `VincoliBusiness`.
  - Il pagamento è richiesto solo dopo che il prodotto è stato progettato.
  - Il prodotto non può essere realizzato finchè non è stato ricevuto il pagamento e c'è `SpecificheProdotto`.
  - La consegna non può avvenire finchè il prodotto non è stato realizzato.
- Si può vedere che si è riusciti a soddisfare abbastanza facilmente i requisiti di business.
- Questo esempio illustra in parte la potenza dei diagrammi di attività: possono modellare processi complessi in un modo conciso e preciso.

Diagrammi di attività – Pin

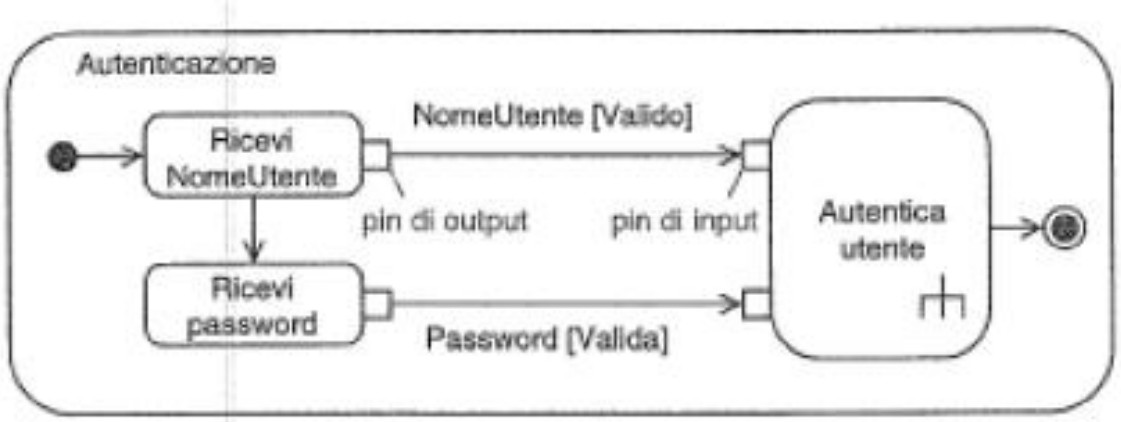
- Per ovviare al fatto che un'attività con molti flussi di oggetti può risultare caotica si possono usare i pin.
- Un pin è un nodo oggetto che rappresenta un input in un'azione o un output da un'azione.
- I pin di input hanno esattamente un arco in entrata e i pin di output hanno esattamente un arco di uscita.
- I pin hanno la stessa sintassi dei nodi oggetto.
- Poiché i pin sono molto piccoli, bisogna scrivere tutte le informazioni, per esempio il nome del classificatore, fuori dal pin ma il più vicino possibile.

Diagrammi di attività – Pin

- La seguente figura mostra un'attività di autenticazione espressa senza i pin:



- Nella seguente figura, invece, viene mostrata la stessa attività di autenticazione, ma espressa tramite i pin:



Diagrammi di attività – Pin

- Confrontando le due figure è possibile vedere che il nodo oggetto è esattamente equivalente alla combinazione del pin di output su `Ricevi NomeUtente` e il pin di input su `Autentica utente`.

- A causa di ciò, talvolta, un nodo oggetto viene definito «pin in stile autonomo».